分　类　号：U461.1　　单位代码：10183
研究生学号：2022422009　　密　　级：公开

吉 林 大 学

硕士学位论文

（学术学位）

面向城市场景的智能车辆轨迹规划算法研究

Research on Intelligent Vehicle Trajectory Planning Algorithm for Urban Scenarios

作者姓名：陈俊毅

专　　业：车辆工程

研究方向：汽车智能化技术

指导教师：何磊 教授

培养单位：汽车工程学院

2025 年 05 月

中国知网　https://www.cnki.net

面向城市场景的智能车辆轨迹规划算法研究

Research on Intelligent Vehicle Trajectory Planning Algorithm for Urban Scenarios

作者姓名：陈俊毅

专业名称：车辆工程

指导教师：何磊 教授

学位类别：学术学位

答辩日期：2025 年 05 月 23 日

中国知网 https://www.cnki.net

摘　要

面向城市场景的智能车辆轨迹规划算法研究

随着人工智能、化学材料等领域的快速发展，汽车行业正加速向电动化、智能化转型，探索绿色智能出行新模式已成为学术界和工业界的共识。在这一进程中，智能车辆的轨迹规划模块如何在道路结构复杂、行驶工况多变的城市场景中，为车辆快速生成安全、高效的最优行驶轨迹，对推动自动驾驶规模化应用具有重要意义。

本文以城市道路场景为研究对象，针对轨迹规划模块中对求解质量和算法实时性的要求，提出了一套适用于城市场景的时空联合轨迹规划算法框架。本文的具体研究内容如下：

（1）本文首先从城市道路场景的特点出发，对自动驾驶轨迹规划中常用的 Frenet 坐标系和 Cartesian 坐标系进行介绍，并阐述了二者在轨迹规划任务中的优缺点以及它们之间的转换关系。随后，阐述并论证了贝塞尔曲线在轨迹规划领域的相关性质，在复杂特殊场景中使用五阶贝塞尔曲线进行参考线生成，并将其与一般道路中心线有机结合，共同构成平滑连续的行驶参考线。最后，以参考线为基础建立 Frenet 坐标系构建三维规划空间，在该规划空间内表征不同的交通信号和交通参与者的时空信息，从而构建出车辆的可行驶空间。

（2）本文针对三维规划空间内的时空参考轨迹生成算法进行研究。考虑到传统混合 A*算法对速度因素考虑不足的局限性，本文引入时间维度，对节点的扩展方式和代价函数进行了改进，将传统混合 A*算法改进为时空混合 A*算法，以生成包含路径和速度信息的可行驶轨迹。为了避免高维空间带来的耗时严重问题，满足算法的实时性要求，本文又将其改进为时空混合 ARA*算法，使算法能够在规定的时间范围内完成搜索任务，从而在效率和性能之间取得平衡。最后，本文融合了可达集和边界框的思想，提出了一种根据时空参考轨迹构建道路边界约束和障碍物边界约束的方法，为后续时空联合轨迹优化中安全性约束的构建奠定了基础。

（3）本文针对笛卡尔坐标系下的时空联合轨迹优化算法进行研究。首先，在笛卡尔坐标系中对时空联合轨迹优化问题进行了简单定义，详细介绍了 ILQR 算法求解优化问题的步骤，并引入正则化等方法对求解过程进行改进。为了弥补 ILQR 算法的缺点，本文综合分析了 CILQR 算法实现方式的优缺点，最终采用改进后的增广拉格朗日

中国知网　https://www.cnki.net

方法来处理不等式约束。之后，本文针对时空联合轨迹优化问题的细节进行了深入设计与补充，提出了一种更高阶的车辆运动学模型，以增强计算结果的平滑性和准确性。同时，设计了多种初始轨迹生成方式，以加快求解速度。从安全性、舒适性等多个维度出发，构建了合理的目标函数。最后，结合车辆的性能限制和功能安全要求，将不等式约束纳入目标函数中，进一步完善了时空联合轨迹优化问题的建模与求解过程。

（4）本文针对前文提出的时空联合轨迹规划算法框架进行仿真验证。基于 Ubuntu 系统和 ROS1 构建了仿真实验平台，并采用 C++语言进行代码开发。针对城市道路环境，本文精心设计了换道超车、交叉路口左转以及大曲率 U 形弯三种典型场景，用以全面评估算法性能。实验结果表明，本文提出的时空联合轨迹规划算法框架在各个场景中均能生成合理且安全的轨迹，同时展现出高效性和良好的实时性，充分证明了其在复杂城市道路场景下的适用性。

关键词：

自动驾驶，城市场景，改进混合 A*，时空联合规划，数值优化

中国知网 https://www.cnki.net

Abstract

Research on Intelligent Vehicle Trajectory Planning Algorithm for Urban Scenarios

With the rapid advancement in areas such as artificial intelligence and chemical materials, the automotive industry is undergoing accelerated transformation toward electrification and intelligent systems. Developing new paradigms of green and intelligent mobility has become a shared focus in both academia and industry. In this context, effectively enabling the trajectory planning module of intelligent vehicles to swiftly generate safe and efficient driving paths under complex urban road structures and dynamic driving conditions is crucial for advancing the large-scale deployment of autonomous driving systems.

This paper focuses on urban road scenarios and proposes a spatiotemporal joint trajectory planning algorithm framework tailored for these environments, addressing the requirements for both solution quality and real-time performance in the trajectory planning module. The specific research topics explored in this paper are outlined as follows:

(1) This paper begins by exploring the characteristics of urban road scenarios and introduces the commonly used Frenet and Cartesian coordinate systems in autonomous driving trajectory planning. It discusses the advantages and disadvantages of these two systems in trajectory planning tasks and the conversion process between them. The paper then examines the relevant properties of Bezier curves in trajectory planning and demonstrates the use of a fifth-order Bezier curve to generate reference lines in complex and special scenarios. These reference lines are then integrated with the general road centerline to create a smooth and continuous reference line. Finally, based on the reference line, a Frenet coordinate system is established within a three-dimensional planning space. In this space, the spatiotemporal information of various traffic signals and participants is represented, which defines the vehicle's feasible driving space.

(2) This paper investigates the generation algorithm of spatiotemporal reference trajectories within a three-dimensional planning space. Considering the limitation of the traditional hybrid A* algorithm, which insufficiently accounts for the speed factor, we introduce the time dimension to improve the node expansion strategy and cost function. As a result, the traditional hybrid A* algorithm is enhanced into a spatiotemporal hybrid A* algorithm, capable of generating feasible trajectories that incorporate both path and speed information. To mitigate the time-consuming challenges posed by high-dimensional spaces

中国知网 https://www.cnki.net

and meet the algorithm's real-time requirements, we further refine the algorithm into a spatiotemporal hybrid ARA* algorithm. This refinement ensures the algorithm completes the search task within the specified time frame, achieving a balance between efficiency and performance. Finally, by integrating the concepts of reachable sets and bounding boxes, we propose a method to construct road boundary constraints and obstacle boundary constraints based on spatiotemporal reference trajectories. This method establishes a foundation for the development of safety constraints in subsequent spatiotemporal joint trajectory optimization.

(3) This paper investigates the spatiotemporal joint trajectory optimization algorithm in the Cartesian coordinate system. The spatiotemporal joint trajectory optimization problem is first defined, and the ILQR algorithm's steps for solving it are detailed, with improvements using techniques such as regularization. To address the limitations of ILQR, a comparative analysis of the CILQR algorithm implementation is presented, followed by the adoption of an improved augmented Lagrangian method to handle inequality constraints. The paper further refines the optimization problem by introducing a higher-order vehicle kinematic model to improve the smoothness and accuracy of the results. Several initial trajectory generation algorithms are also proposed to accelerate the solving process. A comprehensive objective function is designed, considering safety, comfort, and other factors. Finally, inequality constraints, reflecting vehicle performance limitations and functional safety requirements, are integrated into the objective function to enhance the modeling and solution of the optimization problem.

(4) This paper verifies the proposed spatiotemporal joint trajectory planning algorithm through simulation. A simulation platform based on the Ubuntu system and ROS1 was developed, with C++ used for code implementation. Three typical urban road scenarios: lane-borrowing overtaking, left turns at intersections, and large-curvature U-turns, were designed to evaluate the algorithm's performance. Experimental results show that the proposed framework generates safe and feasible trajectories in all scenarios, demonstrating both efficiency and strong real-time performance, confirming its suitability for complex urban road environments.

Key Words:

Autonomous driving, Urban scenarios, Improved hybrid A*, Spatio-temporal joint planning, Numerical optimization

中国知网 https://www.cnki.net

目 录

第 1 章 绪论.....1
1.1 研究背景与意义.....1
1.2 自动驾驶车辆研究现状.....2
1.2.1 国外自动驾驶车辆研究现状.....3
1.2.2 国内自动驾驶车辆研究现状.....4
1.3 自动驾驶轨迹规划方法研究现状.....5
1.3.1 基于图搜索的规划方法.....5
1.3.2 基于采样的规划方法.....8
1.3.3 基于曲线的规划方法.....10
1.3.4 基于数值优化的规划方法.....12
1.3.5 基于学习的规划方法.....14
1.4 本文主要研究内容.....15
第 2 章 参考线生成与规划空间构建.....19
2.1 引言.....19
2.2 Frenet 坐标系.....19
2.2.1 Frenet 坐标系介绍.....19
2.2.2 坐标系转换关系.....22
2.3 参考线生成.....25
2.3.1 贝塞尔曲线基本性质.....26
2.3.2 特殊场景参考线生成.....28
2.4 规划空间构建.....31
2.5 本章小结.....33
第 3 章 时空参考轨迹生成算法研究.....35
3.1 引言.....35
3.2 时空混合 A*算法设计.....35
3.2.1 时空节点扩展方案设计.....35
3.2.2 时空节点启发式函数设计.....42
3.2.3 时空节点成本函数设计.....43
3.2.4 时空混合 A*算法流程.....45
3.3 时空混合 ARA*算法设计.....47
3.4 边界信息匹配.....50
3.5 本章小结.....53
第 4 章 时空联合轨迹优化算法研究.....55
4.1 引言.....55

中国知网 https://www.cnki.net

4.2 基于 CILQR 算法的优化问题求解 55
4.2.1 ILQR 问题定义 55
4.2.2 ILQR 算法求解流程 57
4.2.3 CILQR 算法问题定义与求解.....61
4.3 时空联合轨迹规划优化问题构建 64
4.3.1 高阶车辆状态方程.....64
4.3.2 初始轨迹生成.....65
4.3.3 目标函数设计.....69
4.3.4 车辆性能约束.....71
4.4 本章小结.....72
第 5 章 城区场景轨迹规划算法仿真验证 75
5.1 引言.....75
5.2 仿真实验平台搭建与参数设置 75
5.3 多场景仿真实验结果分析 78
5.3.1 借道超车场景分析.....78
5.3.2 交叉路口左转场景分析.....81
5.3.3 大曲率 U 形弯场景分析.....84
5.4 本章小结.....87
第 6 章 总结与展望 89
6.1 工作总结.....89
6.2 研究展望.....90
参考文献.....91

中国知网 https://www.cnki.net

第1章 绪论

1.1 研究背景与意义

随着我国经济社会的持续发展，汽车行业也迎来了快速增长的阶段。凭借便捷性、舒适性以及经济性等显著优势，汽车已成为我国居民中短距离交通出行的首选方式。截止到 2024 年 5 月，全国机动车保有量达到 4.4 亿辆，驾驶人总量达到 5.3 亿人[1]。在机动车保有量不断提高的同时，国内的城市道路建设和公共交通能力也在不断提升，但由于城区规划限制等诸多原因，无法及时对老旧道路进行扩建重修，很多城区道路在面对快速增长的车流量时显得十分吃力，道路拥堵问题日益严重，严重降低了人民群众的通行效率。在驾驶人总量的不断提高的同时，城区中各类非机动车的数量也在不断增长，相较于机动车来说，非机动车相关的法律法规尚未完善，同时还存在着监管困难、处罚困难、使用者安全意识淡薄等问题。在日常生活中，各种“老头乐”、自行车、电动车等非机动车辆和在道路上十分常见，非机动车的杂乱通行增加了道路拥堵程度，进一步降低了通行效率。除此之外，当驾驶员驾驶经验匮乏，交通环境混乱时，出现交通事故的可能性也会大大提高。

随着人工智能、信息安全、传感技术、化学材料等多个领域的发展，汽车行业在新能源动力系统、智能驾驶、线控底盘等领域，正进行着如火如荼的变革。其中，智能驾驶技术包含环境感知与定位、决策规划、控制执行等方面，具备智能驾驶技术的智能汽车，在将人类驾驶员从驾驶任务中解放出来的同时，还在提升交通安全性、降低驾驶疲劳、提高通行效率、减少交通事故等方面相较于传统汽车展现出明显优势。智能驾驶技术凭借其显著优势和广阔前景，不仅对社会经济发展和保障人民群众生命财产安全具有重要意义，更是国家前沿科技创新与数字化建设的重要代表。因此，该技术长期以来在学术界与工业界备受瞩目，并已成为众多研究者与从业者竞相探索的热点领域。

关于智能驾驶功能，美国汽车工程学会 SAE 发布的 J3016 智能驾驶标准[2]和中国工信部发布的《汽车驾驶自动化分级》(GB/T 40429-2021)[3]中都将智能驾驶等级分为 6 个等级，即从无驾驶自动化（0 级）到完全驾驶自动化（5 级）。各个等级的具体内容如表 1.1 所示。值得说明的是，0-2 级为辅助驾驶功能，在这些级别中，智能驾驶技术作为一项辅助功能帮助人类驾驶员驾驶车辆，人类驾驶员在使用时仍需要观察道路交通情况并在必要时刻及时接管控制车辆，倘若出现交通事故，人类驾驶员应对交通事故

中国知网 https://www.cnki.net

的结果负责。3-5 级被定义为自动驾驶功能，此级别功能启用时，人类驾驶员无需关注驾驶环境，可以将注意力转移至其他事物，只有在系统发出接管提醒时才需要接管控制车辆，在开启此级别的自动驾驶功能时，如若发生交通事故，人类驾驶员通常无需承担责任。

在整个自动驾驶系统中，轨迹规划模块的作用是生成一条平滑连续的目标行驶轨迹，供自动驾驶系统的控制执行模块进行跟踪参考，因此轨迹规划在自动驾驶系统中具有不可替代的作用。轨迹规划算法需要综合考虑车辆运动学约束、道路边界约束、避障约束和舒适性约束等约束条件，实时处理环境感知与定位模块产生的道路环境和车辆状态等信息，同时还需要考虑预测模块生成的动态、静态交通参与者的交通意图与未来轨迹来生成自车行驶轨迹，提供给控制执行模块使用[4]。总的来说，设计一个鲁棒性强、适用于多交通场景、能够实时高效生成满足系统需求的轨迹规划算法，对于自动驾驶系统来说是一个具有挑战性且十分关键的任务。

表 1.1 智能驾驶等级划分

分级	名称	车辆控制	目标和事件探测与响应	动态驾驶任务接管	设计运行条件
0 级	应急辅助	驾驶员	驾驶员及系统	驾驶员	有限制
1 级	部分驾驶辅助	驾驶员和系统	驾驶员及系统	驾驶员	有限制
2 级	组合驾驶辅助	系统	驾驶员及系统	驾驶员	有限制
3 级	有条件自动驾驶	系统	系统	动态驾驶任务接管用户(接管后成为驾驶员)	有限制
4 级	高度自动驾驶	系统	系统	系统	有限制
5 级	完全自动驾驶	系统	系统	系统	无限制

1.2 自动驾驶车辆研究现状

自动驾驶技术因其具有广泛的商业前景，已经成为全球范围内的研究热点。目前，针对自动驾驶技术的研究主要集中在美国、欧洲、新加坡、韩国、日本、中国等国家和

中国知网 https://www.cnki.net

地区。接下来，将对自动驾驶车辆的研究现状进行简要说明。

1.2.1 国外自动驾驶车辆研究现状

Waymo 公司作为专注于研发自动驾驶功能的公司，自 2015 年成立以来，在美国亚利桑那州、日本东京等多个国家和地区开展了自动驾驶功能测试。2020 年 10 月，其成为第一家在车辆未配备安全驾驶员的情况下向公众提供服务的公司。截止到 2024 年 10 月，Waymo 已经在亚利桑那州、加利福尼亚州等多个城市开展了付费乘车服务，每周提供约 15 万次服务，行驶里程超过 100 万英里，其自动驾驶功能在道路上累计行驶里程已经超过了 3300 万英里[5]。同时，Waymo 也在积极开拓自动驾驶功能的使用场景，将自动驾驶功能应用在无人驾驶出租车、物流运输、城市功能交通、乘用车等领域。图 1.1 展示的是搭载了 Waymo 公司自动驾驶功能的无人驾驶出租车。

图 1.1 Waymo 公司无人驾驶出租车

特斯拉作为全球电动汽车行业的领导者，在自动驾驶领域也颇有建树。其 L2 级别的 Autopilot 辅助驾驶系统，可以帮助用户实现自动辅助转向、自适应巡航、自动变道等功能，已在美国、欧洲等多个国家和地区投入市场使用。2024 年 3 月，特斯拉正式向美国用户推送了 FSD V12 全自动驾驶功能，FSD V12 使用了来自全球各地数百万辆特斯拉的车身视频作为训练材料来模仿人类司机的驾驶决策，因其良好的实际表现和非分层式的端到端架构引来了从业人员和消费者的关注和认可。2024 年 10 月特斯拉正式发布 RobotTaxi 车型 CyberCab，去除了传统的方向盘和踏板。该车型有望降低出行成本和整车成本，预计 2026 年大规模量产[6]。

德国汽车厂商梅赛德斯-奔驰发布的 Drive Pilot 系统，已经在德国和美国市场投入

中国知网 https://www.cnki.net

使用。这是欧洲经济委员会认定的首个，也是唯一一个 L3 级自动驾驶系统。此系统在满足系统启用条件时，由系统全权接管车辆，在开启 Drive Pilot 系统过程中如果出现安全问题，责任完全由企业承担，这对自动驾驶功能的实际落地使用有着重要意义。

1.2.2 国内自动驾驶车辆研究现状

相较于国外，我国关于自动驾驶技术的相关研究起步相对较晚。随着国内产业升级和相关政策的推动，近些年自动驾驶功能受到了大量科研机构和相关企业的青睐。哈尔滨工业大学和国防科技大学于上世纪八十年代研发出的无人驾驶车辆，标志着我国自动驾驶技术研究的开始[7]。百度公司作为中国最早一批研究自动驾驶功能的公司，在 2015 年发布了 Apollo 自动驾驶平台，2017 年在北京进行了自动驾驶功能测试，2018 年将 Apollo 系统的相关代码进行开源，并且每年对其进行迭代更新，这大大加快了我国自动驾驶技术的研究速度。如图 1.2 所示，百度公司在 2021 年在武汉、上海、北京等地发布了萝卜快跑服务平台以开展 RoboTaxi 业务。自开展业务以来，平均每周提供出行服务 7.5 万次；截至 2024 年 7 月，萝卜快跑已提供 700 万次安全出行服务，有力地推动了国内自动驾驶功能的落地与发展[8]。

图 1.2 百度公司萝卜快跑无人驾驶出租车

除百度公司外，小马智行、滴滴等公司重点研发 L4 级别的自动驾驶技术，搭载其自动驾驶系统的产品纷纷落地，已经在北京、上海等地开展 RoboTaxi 业务。相较于 L4 级别的高级别自动驾驶，Momenta、小鹏、华为、理想、大疆等主机厂和供应商将主要精力投入到 L2 级别的自动驾驶技术研究上，其中 Momenta 公司开发的自动驾驶解决方案覆盖城市和高速等主要使用场景，实现了有路就能开的目标，截止到 2024 年 7 月，

中国知网 https://www.cnki.net

已经有近十万辆汽车搭载 Momenta 公司提供的解决方案。新石器、九识、美团等公司研究的自动驾驶技术主要聚焦于物流运输领域，其中新石器与顺丰、中通等公司合作，使用无人驾驶物流小车在开放道路及园区、社区等地进行快递运输、物资配送等，与相关从业人员一起解决了末端配送问题。文远知行、酷哇科技等公司研究开发的自动驾驶功能搭载在城市无人环卫车、城市无人小巴等城市功能交通车辆上，极大地降低了相关行业人员的劳动强度，降低了公共交通运行成本[9]。

目前，全国主流的汽车厂商都推出了具备 L2 级别自动驾驶功能的量产车型，自动驾驶功能在人们的日常生活中越来越常见。相信随着自动驾驶功能的不断完善和交通参与人员接受程度的不断提高，未来会有越来越多、越来越好的自动驾驶车辆出现在道路上，提高通行效率，降低出行成本，减少交通事故的发生。

1.3 自动驾驶轨迹规划方法研究现状

在自动驾驶领域，轨迹规划作为核心环节，受到了学界与业界的重点关注。该模块的相关算法通过实时解析交通参与者的动态信息，并结合自车的运动状态，在复杂交通场景中实现多维度平衡：首要任务是确保行驶安全性，通过精确的碰撞预测与规避机制来保障行车安全；同时，综合考虑驾乘人员的舒适性、轨迹的平顺性及通行效率等多重指标，最终生成符合车辆动力学约束的最优行驶轨迹。目前，常用的轨迹规划方法主要有：基于图搜索的方法、基于采样的方法、基于曲线的方法、基于数值优化的方法以及基于学习的方法。

1.3.1 基于图搜索的规划方法

图搜索算法及其衍生算法作为最常用的规划算法之一，会将可行驶区域在空间或时间维度上离散化为晶格或栅格地图。在地图中，离散节点代表车辆的运动状态，连接离散节点的边则代表相应的可执行操作。算法的核心思想在于通过设计相应的搜索策略和评估方法，在图中找到具有最低成本的从起点到终点的全局最优路径，在成本计算过程中，不仅会考虑沿路径或轨迹的行驶距离，还会综合评估平滑性和能耗等其他相关因素。这类算法通常应用于道路交通环境中的轨迹规划问题，特别是在那些具有明确规则和确定性状态的场景中。

Dijkstra 算法作为最为经典且使用最为广泛的图搜索算法，其原理是通过逐步扩展

中国知网 https://www.cnki.net

边来找到从起始节点到目标节点的所需路径。在搜索过程中，该算法会保持一组待扩展的节点，并在每一步中，从待扩展节点中选择成本最小的节点来进行扩展，算法效果如图 1.3 所示。

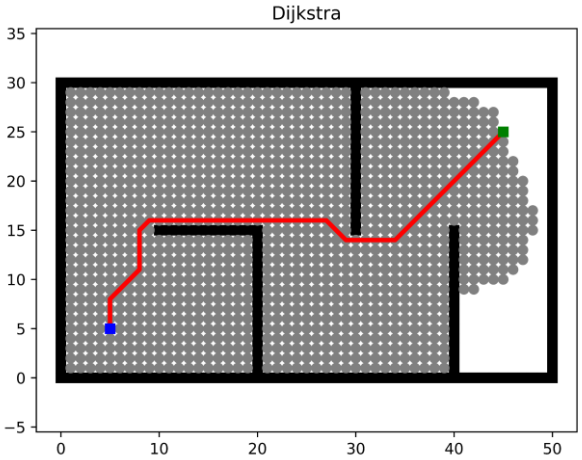


图 1.3 Dijkstra 算法示意图

成本函数设计的灵活性使得该算法能够有效地解决多目标的轨迹规划问题，而不需要优化目标具有可微性或连续性。该算法的主要局限性在于它无法处理具有负权重边的图，并且在探索的过程中对待探索的边不存在倾向性，会忽视新节点或边与目标节点之间的关系。这些限制可能会导致计算稠密问题，在非最优的节点和边上耗费较多的时间[10]。Ruinan Chen 等人将 Dijkstra 算法和快速探索随机树(Rapidly-exploring random tree, RRT)算法结合起来提出了一种旨在解决半结构化道路的自动驾驶车辆路径规划策略。该策略通过 RRT 算法缩小规划区域，并将路径规划问题转化为加权有向图的多源成本优化问题，使用 Dijkstra 算法进行优化，该策略在静态和动态障碍物场景中均表现出较高的效率和准确性，且实时性能有了较大提升[11]。Sundarraj 等人提出了一种基于权重控制粒子群优化的 Dijkstra 算法。该算法通过动态调整惯性权重优化路径规划，结合 Dijkstra 算法的最短路径搜索能力与粒子群优化的全局搜索能力，显著提升了自主机器人车辆在动态复杂环境中的路径规划效率，在最优路径选择、路径平滑度、避障及计算时间等方面都有不错的表现[12]。

为了解决 Dijkstra 算法的计算效率问题，A*算法在搜索成本中融入启发式函数来引导算法搜索方向，效果如图 1.4 所示。启发式函数通过提供当前节点和目标节点之间的估计成本信息，显著减少了搜索过程中边和节点的访问次数，从而加速了搜索过程，提高了搜索效率[13]。启发式函数作为节点之间成本信息的估计项，当启发式函数的数值始终低于实际成本的下限时，可以确保 A*算法的搜索结果是最优的；当启发式函数

中国知网 https://www.cnki.net

的数值可能超过实际成本时，获得的结果可能是次优的。值得一提的是，在优化目标和成本类似于最短距离这种较为简单的形式时，A*算法及其衍生算法解决问题非常方便，其启发式函数的设计也较为简单。但当图搜索规划问题具有更加复杂的优化目标时，启发式函数的设计具有相当的挑战性，此时更适合使用无启发式的穷举搜索算法和 Dijkstra 类算法[14]。

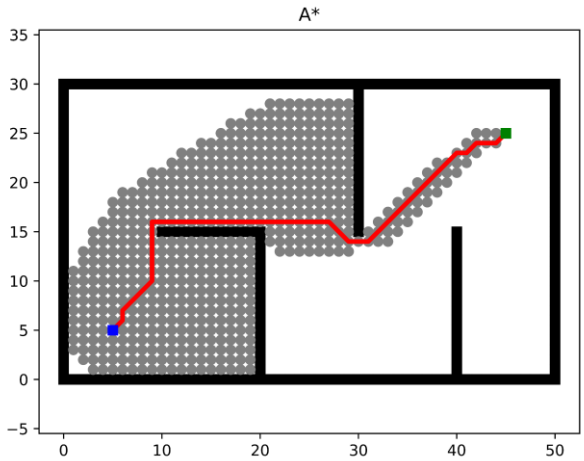


图 1.4 A*算法示意图

为了进一步提升 A*算法的效率，降低搜索耗时，Maxim Likhachev 等人提出了 ARA*(Anytime Repairing A*)算法。该算法在任务开始时，使用较大的启发式函数来快速找到可行路径，并在剩余时间内不断降低启发式函数以优化路径，这使得该算法能够在规定的时间内完成路径规划任务[15]。此外，仍有大量的研究人员针对 A*算法的启发式函数和搜索方式进行了改进，如从起点和目标同时进行搜索的双向 A*(A*-connect)算法[16]、引入启发式函数权重以增加搜索过程贪心性的加权 A*(Weighted A*, WA*)算法[17]、运行时动态更新启发式函数的动态 A*(Dynamic A*, D*)算法[18]、结合增量思想的终身规划 A*(Lifelong Planning A*, LPA*)算法[19]等。以上的 A*及其变种算法，大多在离散空间内使用四联通或八联通的方法来进行节点扩展，这使得此类算法不能很好地满足车辆的运动学约束，无法直接应用于自动驾驶车辆的路径规划问题。为了更好地在车辆上进行算法应用，Dmitri Dolgov 等人提出了混合 A*(Hybrid A*)算法。混合 A*算法相较于 A*算法，在节点的扩展方式上引入了 θ 维度，即车辆的航向角，使得算法能够按照车辆运动学公式来扩展节点，满足车辆运动学约束，增加了算法的适用性[20]。

混合 A*算法在扩展节点时，会通过在单位时间内对车辆的速度和前轮转角进行采样，结合当前节点的车辆位姿状态，进行前向模拟，从而得到一段符合车辆运动学特性的行车路径，路径末端位姿对应的节点即为当前节点的子节点，算法效果如图 1.5 所

中国知网 https://www.cnki.net

示。启发式函数一般需要结合要解决的规划问题来进行具体定义，通常需要考虑实际距离、避障条件等。相较于传统 A*算法，混合 A*算法生成的路径更加符合车辆的实际运动能力，同时也更好地满足避障约束和车辆运动学约束，近年来在自动驾驶车辆的落地应用中，特别是在非结构化道路场景中得到了广泛使用。Tianchuang Meng 等人在自动驾驶停车系统的路径规划问题中，结合维诺图来改进混合 A*算法以增强路径规划结果的安全性，并采用多阶段动态优化策略提高搜索效率[21]。Yichen Zhang 等人在无人驾驶矿用卡车的全局路径规划中，设计了一种改进的两阶段二分法混合 A*算法，通过地图预处理和两阶段二分法优化转向角和步长，该算法在障碍物稀疏和密集的环境中均显著减少了计算时间和路径长度，提升了路径规划的效率和质量[22]。Kangle Hu 等人将铰接式履带车辆模型与混合 A*算法结合，并通过最小快照平滑法对搜索结果进行优化，解决了复杂环境中铰接式履带车的轨迹规划问题[23]。Weitian Sheng 等人将多阶段混合 A*算法和改进的安全行驶走廊优化方法结合，以解决狭窄通道环境下的自动驾驶泊车轨迹规划问题，该算法通过识别并分段搜索狭窄通道中的可行路径，显著提高了自动驾驶车辆在复杂环境中的规划效率和轨迹平滑性，并在处理狭窄通道时展现出更优越的性能[24]。

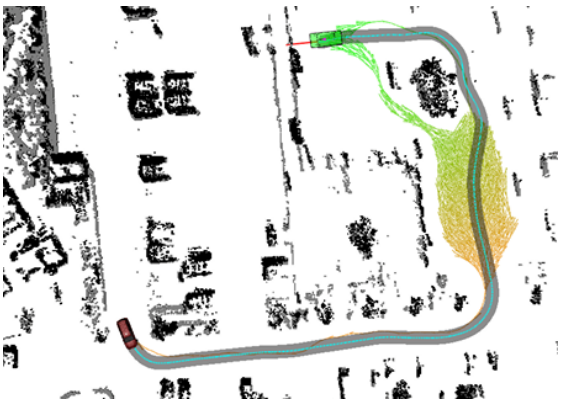


图 1.5 混合 A*算法示意图

1.3.2 基于采样的规划方法

相较于需要预先离散化规划状态空间的图搜索规划算法，基于采样的规划算法通过增量地探索状态空间或控制空间并在对应空间内采样来执行规划任务，根据采样方式的不同可以分为确定性采样方法和随机采样方法。

确定性采样方法通过离散格点在状态空间或控制空间内按照一定的规则或策略进行采样，进而完成规划任务。确定性采样的代表性工作是 Apollo 规划模块中的动态规

中国知网 https://www.cnki.net

划方法，该方法在固定的位置进行采样，根据预先设计的成本计算方法计算出各点的成本并更新父节点，然后在路径的末端选取成本最低的点回溯，从而得到整条路径[25]。

随机采样方法是以概率完备性来完成规划任务的，它通过在空间中随机生成采样点来探索路径。常见的随机采样方法有：概率路线图法(Probabilistic Road Map, PRM)和随机扩展树法(Rapidly-exploring Random Tree, RRT)。PRM 算法及其衍生算法通过在自由空间中随机采样构建一个可行的路径图，再在这个图中进行搜索，以找到一条从起点到终点的路径。在应用 PRM 算法时，需要着重注意障碍物碰撞检测和计算时间，因此可以使用“懒惰”方法，即首先搜索轨迹然后验证其可行性，以此来推迟碰撞检测步骤[26]。相比之下，RRT 算法通过随机采样快速探索整个空间，并构建一棵从起点出发、逐步扩展的随机树，直到找到一条从起点到目标的路径，因此得到了更广泛的应用。RRT 算法的效果如图 1.6 所示。

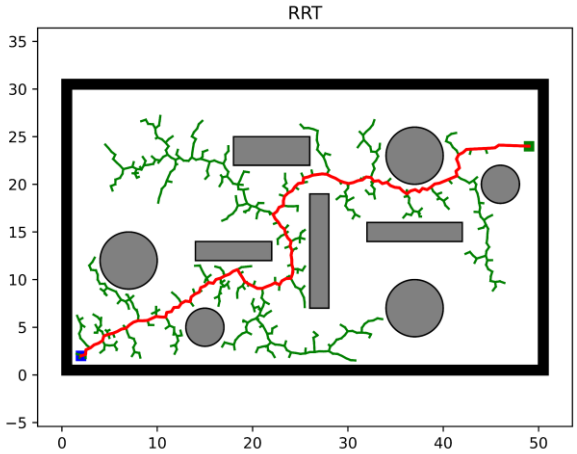


图 1.6 RRT 算法示意图

在 RRT 算法的基础上，许多研究人员对其进行深入研究，衍生出了一些改进算法，如具有渐进最优性且复杂程度固定的 RRT*算法[27]、从起始节点和结束节点同时生成两棵搜索树，搜索效率更高的 RRT-connect[28]或 Bi-RRT 算法[29]、引入前向可达集或运动学约束改进节点生成，考虑受控对象的运动学约束的运动学 RRT*[30]和运动学双向 RRT*[31]算法等。James Swedeen 等人向 RRT*算法中加入基于圆弧和基于贝塞尔曲线的两种圆角概念，在规划时考虑了路径的最大曲率约束和曲率连续性要求，同时引入路径抽样启发式算法，显著提升了路径规划的收敛速度和解的质量[32]。Guanghao Huang 等人通过采用循环采样策略生成随机点、基于成本函数筛选随机点、考虑车辆转向范围选择邻近点的方法来改进 RRT 算法，并结合 B 样条曲线简化和优化路径，减少了路径的曲率变化和规划时间，显著提高了路径规划的质量[33]。Hamada Esmaiel 等人使用

中国知网 https://www.cnki.net

双层 RRT*算法，第一层通过目标偏差采样策略和分段前向剪枝处理快速生成初始路径，第二层利用第一层的结果，结合反向维护策略对路径进行优化，在计算成本、路径长度等方面有着明显优势[34]。Bai Li 等人提出了一种基于时空 RRT*的轨迹规划方法，通过改进后的 RRT*算法灵活地搜索粗轨迹，并利用二次规划对轨迹进行优化，确保轨迹平滑且满足碰撞避免约束，但 RRT*算法的随机性在结构化道路中使用时并不能保证耗时可控[35]。

虽然基于随机采样的算法具有概率完备、计算效率高等优点，但基于随机采样的实现会将不确定性引入最终的规划结果中。在环境状态相对确定且对规划轨迹质量要求较高的封闭道路场景等轨迹规划任务中，通常期望得到确定性的轨迹规划结果。因此，这类算法的应用不如基于图搜索、曲线和数值优化等算法广泛[36]。

1.3.3 基于曲线的规划方法

基于图搜索或采样的算法一般会生成离散的点，由这些点连接得到的路径，通常会存在不符合车辆运动特性、运动不连续不平滑等缺点。而基于曲线的规划算法的使用，则可以在其他算法的基础上更进一步，提升规划结果的平滑性、安全性，并提高行驶效率。常见的用于解决规划任务的曲线有：Reeds-Shepp 曲线、Dubins 曲线、回旋曲线、三阶螺旋线、Sigmoid 曲线、多项式曲线、多项式样条曲线、贝塞尔(Bézier)样条曲线、B 样条曲线等，接下来将进行相关说明。

Reeds-Shepp 曲线由一系列直线和圆弧组成，描述车辆的前向直线运动、后向直线运动和恒定曲率转弯运动。Dubins 曲线是 Reeds-Shepp 曲线的简化版，只能描述车辆的前向直线运动和恒定曲率转弯运动。作为由直线和圆弧组成的参数曲线，Reeds-Shepp 曲线和 Dubins 曲线因其简单高效的性质，在具有曲率约束、起始位置约束和航向角约束等约束条件的最短路径规划问题中表现出良好的效果。在不考虑避障因素的前提下，Reeds-Shepp 曲线能够以基本符合车辆动力学约束的方式衔接始末位姿。因此，在自动驾驶泊车规划任务中，通常和混合 A*算法结合使用，以提高搜索效率。当扩展新的节点时，可以生成一条连接新节点到目标节点的相应曲线，如果这条曲线不会与障碍物发生碰撞，就代表规划任务完成，可以输出混合路径。Shi Y 等人对混合 A*算法和 Reeds-Shepp 曲线进行了改进，提高了泊车功能局部路径规划的效率[37]。然而，这两种曲线均无法保证规划结果的曲率连续性，因此难以适应车辆高速行驶的场景。

回旋曲线也被称作欧拉螺线或羊角螺线，其特性是曲线上点的曲率与沿曲线移动

中国知网 https://www.cnki.net

的距离成正比，即曲率随着曲线长度的增加而线性改变。这一特性对于限制车辆的横向加速度、确保沿轨迹行驶时的舒适性至关重要，因此回旋曲线被广泛应用于各种轨迹规划任务中。然而，回旋曲线缺乏明确的数学解析表达式，其构造依赖于积分运算和迭代求解，因此耗时较长。Kemeng Ran 等人设计了基于回旋曲线的路径变形策略，以调整 RRT*算法规划出的路径的曲率和弧长，帮助机器人在平滑路径的同时避开障碍物，确保路径的可行性和满足运动学约束[38]。Dae Jung Kim 等人则提出了一种基于回旋曲线的局部路径规划方法，该方法通过近似回旋曲线模型，能够在计算效率和路径平滑性之间取得平衡，在自动垂直泊车系统上表现出色[39]。

三阶螺旋线也是轨迹规划中的常见数学曲线形式[40]，其特点是曲率随曲线长度呈三次多项式变化，需要采用数值优化方法来求解未知参数。Sigmoid 曲线适用于传统封闭道路场景中的简单变道轨迹规划任务[41]，可以避免加速度和速度发生突变，从而减少机械系统的冲击和振动。由于其特殊的性质，这两种曲线适合在特殊场景下使用[36]。

除了上述简单的插值曲线外，带分段表达式和多控制点的样条曲线也常用于轨迹规划算法中。当需要在规划任务中考虑起始位置与状态约束时，由于多项式曲线具有明确的数学形式，可以保证规划结果中速度和曲率变化的连续性，因此得到了广泛的应用。然而，多项式曲线在拟合过程中可能会出现龙格现象，所以在轨迹规划任务中，更常见的是将曲线进行分段，在每个分段中分别使用多项式样条曲线。当使用五阶或七阶多项式样条曲线时，可以在给定的控制点下最小化轨迹的加加速度或加加速度变化率，这有助于提升驾乘人员的舒适性或降低车辆制动器的能耗[42]。除此之外，相较于高阶多项式曲线，多项式样条曲线可以使用更少的参数来拟合更长距离和更加复杂的车辆运动轨迹，并且计算效率更高[43]。Zhe Zhang 等人利用多项式曲线在 Frenet 坐标系中分别规划纵向和横向轨迹，通过将多项式曲线的系数转化为二次规划问题求解，避免了耗时的碰撞检测，从而显著提高了轨迹规划的实时性和鲁棒性[44]。Yang Li 等人基于五次多项式曲线设计了一种用于自动驾驶车辆换道的轨迹规划算法，该算法将多项式曲线参数优化问题转化为考虑安全性、舒适性和效率的优化问题，利用序列二次规划算法求解最优轨迹参数，并在实车实验中验证了算法的有效性[45]。

贝塞尔曲线和 B 样条曲线是多项式样条曲线的特殊形式，同样也具有解析表达式明确、计算效率高、导数计算方便等优点。贝塞尔曲线的形状由一系列有顺序的控制点决定，可以显式地确定曲线参数。如果控制点构成的特征多边形是凸多边形，还可以保证曲线被包围在凸多边形的范围内，因此被广泛应用于轨迹规划任务当中。B 样条曲

中国知网 https://www.cnki.net

线是贝塞尔曲线的一般化形式，除可以指定阶次和可以局部修改曲线形状外，其余特性与贝塞尔曲线基本相同，所以在使用时可以通过优化对应的特征多边形来间接优化曲线本身[46]。在速度规划任务中，Jialun Li 等人巧妙地利用梯形走廊来约束贝塞尔曲线的形状，不仅显著拓展了求解空间，还确保了轨迹规划的安全性，同时还在最优性和可靠性方面取得了显著提升，为自动驾驶车辆的速度规划提供了一种高效且安全的解决方案[47]。Wenchao Ding 等人针对复杂交通环境提出了一种创新的轨迹生成方法。他们将多种语义元素（如动态障碍物、交通信号、速度限制等）抽象为统一的时空语义走廊，并利用分段贝塞尔曲线的凸包性质和速度图性质，在走廊内高效生成安全、连续且满足动态约束的轨迹，为自动驾驶车辆在复杂场景下的安全导航提供了理论保障[48]。

1.3.4 基于数值优化的规划方法

数值优化方法是在满足特定约束的前提下，寻找使轨迹的评价函数达到最小（或最大）值的方法，本质上是将轨迹规划问题转换为最优控制问题。车辆轨迹可以表示为系统状态变量和控制变量的函数，而评价函数则是轨迹函数的函数。数值优化方法通过在当前轨迹附近进行轻微扰动，使轨迹代价下降，最终寻找到代价最低的轨迹。在利用数值优化方法求解规划问题时，通常要把无限维连续优化问题转化为有限维常规非线性优化问题。根据转化方法的不同，轨迹优化方法又可大致分成打靶法(shooting method)和配点法(collocation method)，两者的主要区别在于对状态方程的处理，如图 1.6 所示。

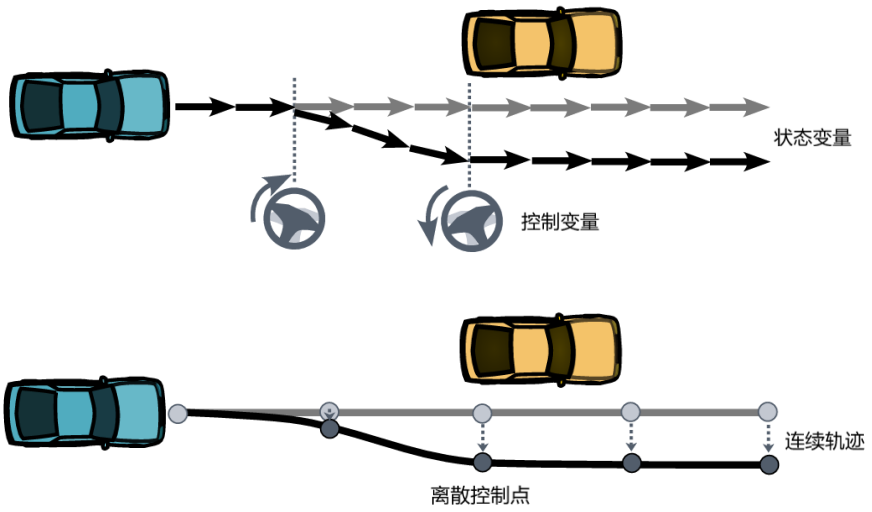


图 1.6 打靶法（上）和配点法（下）

中国知网 https://www.cnki.net

打靶法只优化控制变量，通过初始状态和对状态方程进行前向积分得到状态变量，其优点是能够得到非常精确的最优轨迹，缺点是对于有强约束或动力学不稳定的系统，优化过程不容易收敛；配点法则是直接优化表示车辆运动的参数化轨迹，得到一系列表示轨迹形状的离散控制点，在优化过程中，将约束应用于轨迹以确保其可行性。

打靶法中常见的优化算法包括迭代线性二次调节器(Iterative Linear Quadratic Regulator, ILQR)[49]、微分动态规划(Differential Dynamic Programming, DDP)[50]和模型预测控制(Model Predictive Control, MPC)[51]等。这些算法不仅适用于轨迹规划，而且在跟踪和执行规划结果时也有良好的表现。OPTPAP 算法也是基于最优控制解决轨迹规划问题的典型算法[52]。

MPC 算法在规划过程中利用车辆运动学或动力学模型来预测车辆的状态，有效地考虑了复杂的约束条件，具体实现细节灵活多样。其中，使用了非线性模型约束的非线性预测控制(nonlinear MPC, NMPC)算法，虽然能够准确地预测系统状态的变化，但在处理非线性模型时需要消耗大量的计算资源，相对而言实时性较差[53]。为了确保轨迹规划和跟踪任务的实时性能，需要对模型进行线性化。具体的方法包括使用线性时不变预测控制(Linear Time Invariant MPC, LTI-MPC)算法[54]，该算法直接采用基于车辆动力学平坦度特性的等价线性模型；线性时变预测控制(Linear Time Variant MPC, LTV-MPC)算法[55]，该算法在每个控制输出处对非线性模型线性化；以及线性变参数预测控制(linear parameter-varying MPC, LPV-MPC)算法[56]，该算法在状态空间中使用线性模型而在参数空间中使用非线性模型。

MPC 算法在设置优化目标和约束方面具有很强的灵活性。许多研究人员在 MPC 算法中引入人工势场方法，将其作为风险优化目标的一部分，构建道路环境风险势场，通过风险指标取代了避障约束[57]。Tianqi Qie 等人针对不确定交通环境，提出了一种基于管状模型预测控制的轨迹规划方法，该方法通过构建收紧约束，有效考虑了轨迹预测的不确定性，并在静态与动态场景中均表现出色[58]。

与 MPC 算法相比，ILQR 算法通常具有更简洁的数学形式，但它没有明确考虑复杂的车辆动力学和交通环境的约束，只能处理线性化的状态模型。为了处理这些约束，可以通过罚函数等方法将其添加到优化目标函数中，从而变为CILQR(Constrained ILQR)算法来进行迭代求解[59]。另一方面，ILQR 算法不能像 MPC 算法那样通过采用与预测时间窗不一致的控制时间窗来加速求解过程，必须在预测时间窗口内求解整个控制序列。虽然有这些限制，但 ILQR 算法的简洁数学形式使得其具有更高的计算效率[60]，因

中国知网 https://www.cnki.net

此在各种轨迹规划任务中具有更广泛的实际应用。Fang Da 等人通过构建和优化策略树，提出了一种 Reactive ILQR 算法，该算法允许车辆根据未来环境的变化动态调整行为，从而在复杂环境中实现更高的安全性和灵活性，这为自动驾驶中的安全规划提供了一种新的思路[61]。Tong Li 等人通过构建基于场景的动态决策树和优化轨迹树，利用 ILQR 算法在固定风险权重下处理多种未来场景，优化轨迹树的控制序列，从而生成最优轨迹[62]。这些算法考虑了周围车辆行为的不确定性，将车辆未来轨迹的多模态特性纳入优化过程，生成考虑预测结果概率分布的规划方案，并确定最优动作序列。由于其对不确定性的处理，这些方法也被称为应急规划算法[63]。

配点法使用稀疏离散的轨迹控制点来代替打靶法中的完整控制序列，从而形成非线性问题并求解相关参数。Yajia Zhang 等人在充分考虑车辆运动学模型和静态障碍物避让需求的基础上，直接采用 IPOPT 求解器来求解非线性路径规划问题，为自动驾驶领域的工业应用提供极具价值的解决方案[64]。Zhichao Han 等人提出了一种面向非结构化环境的时空轨迹优化方法，该方法借助车辆的微分平坦特性，简化了轨迹规划问题的表达形式，并通过拟牛顿法进行求解，不仅显著提升了计算效率，还确保了生成轨迹的高质量[65]。

当优化目标以基于控制点参数的二次形式表示时，配置轨迹规划任务可以转化为二次规划(Quadratic Programming, QP)问题。由于二次规划问题的凸性，保证了解决方案的唯一性，可以使用二次规划求解器进行求解，并且与一般非线性优化求解器相比，其计算效率更高，这使得 QP 方法在轨迹规划中被广泛采用。Yajia Zhang 等人提出了一种基于 Frenet 坐标系的路径规划方法。该方法将路径分解为一系列离散点，并通过恒定的横向偏移量三阶导数连接这些离散点，从而实现路径的平滑表示。通过将路径规划问题转化为二次规划问题，该方法能够快速高效地求解出平滑且安全的路径[66]。

1.3.5 基于学习的规划方法

随着计算机硬件性能的快速发展和人工深度神经网络技术的不断进步，端到端的轨迹规划方法正逐渐展现出巨大的潜力。现有的端到端轨迹规划算法主要分为两类：模仿学习(Imitation Learning, IL)和强化学习(Reinforcement Learning, RL)。

模仿学习直接从专家演示数据中学习自动驾驶规划策略，而不依赖于与实际环境交互的反馈，其目标是使训练模型的行为尽可能接近驾驶员数据的行为[67]。模仿学习可以进一步细分为三类：行为克隆(Behavior Cloning, BC)，直接策略学习(Direct Policy

中国知网 https://www.cnki.net

Learning, DPL)和逆强化学习(Inverse Reinforcement Learning, IRL)。作为最广泛使用的模仿学习方法，行为克隆是一种被动的离线规划策略学习方法，它将驾驶员数据拟合到端到端模型中，然后生成模仿人类驾驶的行为[68]。Zeyu Zhu 等人提出了一种基于行为克隆的多任务条件模仿学习框架，用于解决自动驾驶在拥挤路口的导航任务。该方法将横向控制和纵向控制任务整合到一个多任务学习框架中，并引入任务依赖的不确定性权重来动态平衡任务的重要性。通过这种方式，该方法在复杂交互场景中显著提高了任务的成功率和控制质量，展现出更强的适应性和鲁棒性[69]。直接策略学习是一种基于行为克隆的迭代在线策略学习方法，它通过有效地消除过去不正确的策略来不断改进驾驶策略[70]。Chengzhen Yan 等人提出了一种基于直接策略学习的地图无关导航方法，该方法专注于在未知环境中安全地训练导航策略，通过设计一种受控探索策略来收集演示数据，并引入一种增强安全性的损失函数来优化网络训练，同时利用对称数据增强技术显著提高训练效率[71]。逆强化学习通过处理专家轨迹数据来学习从输入到输出的基本因素，并基于推断的奖励函数优化实际轨迹规划策略，最终得到近似驾驶员行为的驾驶策略。Tung Phan-Minh 等人提出了一种基于最大裕度算法的逆强化学习规划方法，该方法在复杂的城市交通中生成多样化的候选轨迹，使用安全过滤器和由专家数据训练的模型对轨迹进行评分并选择最优轨迹，在密集交通、车辆加塞、急刹车和酒店接送区等复杂场景下均有不错的表现[72]。

与模仿学习不同，强化学习依赖车辆与驾驶环境的交互来获得学习和训练所需的奖励和惩罚反馈数据。强化学习算法的最终目标是学习一种规划策略，以最大化随着时间的推移所积累的反馈。常见的强化学习规划算法包括深度 Q 网络(Deep Q-networks, DQN)[73]、深度确定性策略梯度(Deep Deterministic Policy Gradient, DDPG)[74]和邻近策略优化(Proximal Policy Optimization, PPO)[75]。尽管强化学习算法通常需要大量的训练步骤和计算资源，采样效率低，收敛速度慢，但它们能够学习复杂的规划策略并适应不断变化的环境。Yuanqing Wu 等人提出了一种基于深度强化学习的端到端自动驾驶策略学习方法，通过结合近端策略优化和随机网络蒸馏好奇心机制，并引入辅助评论家网络，有效提高了在复杂交通场景下的训练效率和控制性能[76]。

1.4 本文主要研究内容

根据现阶段轨迹规划算法的研究现状，本文提出一套适用于城市道路场景的安全

中国知网 https://www.cnki.net

高效的时空联合轨迹规划算法框架。本文各部分研究内容如下：

第一章，绪论。主要介绍了自动驾驶车辆目前的研究现状和背景意义，对现有的轨迹规划方法进行梳理总结，并对本文的主要研究内容和章节分布进行介绍。

第二章，参考线生成与规划空间构建。首先介绍轨迹规划中常用的 Frenet 坐标系和笛卡尔(Cartesian)坐标系，并阐述二者在轨迹规划任务中的优劣势，还推导了两种坐标系之间的坐标转换公式。其次，根据结构化道路的特点，使用五阶贝塞尔曲线在特殊场景生成参考线，并与一般结构化道路场景的车道中心线组合成完整的参考线，以此参考线作为 Frenet 坐标系的基础，构建三维规划空间。最后，分析了不同的交通信号和交通参与者的特性，并将它们的时空信息在三维规划空间内表示出来，最终构建出车辆的可行驶空间。

第三章，时空参考轨迹生成算法研究。首先，将原本用于二维空间路径搜索的混合 A*算法进行改进，引入加速度信息和时间信息，改变节点扩展方式，使其变为能够进行轨迹搜索的时空混合 A*算法。其次，对时空混合 A*算法的成本函数和启发式函数进行设计，并引入局部不一致的概念，将算法进一步改进为时空混合 ARA*算法，以满足轨迹规划实时性的要求。最后，基于时空参考轨迹进行交通信息匹配，以便构建安全性约束并进行后续的轨迹优化。

第四章，时空联合轨迹优化算法研究。本章首先给出了一般道路下时空联合轨迹优化问题的定义，详细阐述了如何使用 ILQR 算法求解该问题。针对求解过程中出现的问题，本文引入相关方法对传递过程和不等式约束的添加方式进行改进。之后，本文提出一种高阶车辆运动学模型以提升轨迹平滑性，并结合时空参考轨迹设计了多种初始解的生成方式。最后，结合车辆性能以及对优化结果的要求，对目标函数进行了设计。

第五章，城市场景轨迹规划算法效果的仿真验证。为了验证本文提出的轨迹规划算法的效果，本文利用 Ubuntu 系统和 ROS1 搭建了仿真实验平台，并使用 C++语言进行算法开发。针对城市道路场景，本文设计了换道超车场景、交叉路口左转场景以及大曲率 U 形弯场景进行算法验证。实验数据分析表明，本文提出的时空联合轨迹规划算法框架在各场景中均保证了轨迹规划结果的合理性和安全性，同时还具有高效性和良好的实时性。

第六章，总结与展望。本章对本文提出的面向城市交通场景的时空联合轨迹规划算法框架进行总结，并对本研究后续可以改进和提升的地方进行分析与展望。

本文主要研究内容的组织结构如图 1.7 所示。

中国知网 https://www.cnki.net

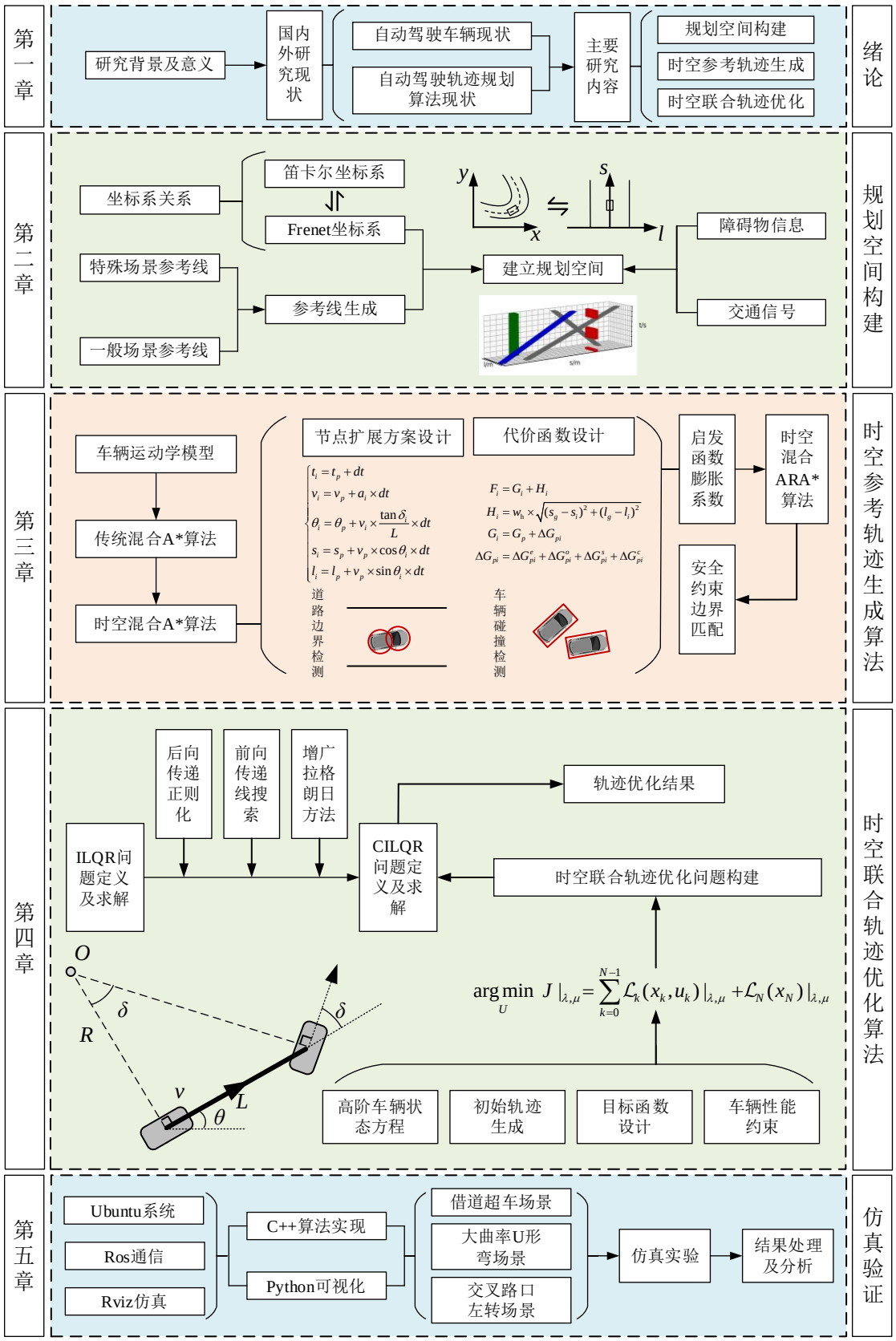


图 1.7 本文组织架构图

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第2章 参考线生成与规划空间构建

2.1 引言

从本章节开始，本文将从城市道路场景的特点出发，对自动驾驶车辆中的轨迹规划算法进行研究。作为后续章节的基础，本章主要解决一些前置任务。首先，将介绍轨迹规划领域常用的 Frenet 坐标系和笛卡尔坐标系，并阐述二者在轨迹规划任务中的优缺点以及它们之间的转换关系。其次，根据道路的自身特点并结合贝塞尔曲线，生成车辆行驶时使用的参考线，以该参考线作为 Frenet 坐标系的基础构建三维规划空间。最后，将道路中不同的交通标志和交通参与者根据自身特点投影到规划空间中，在规划空间中确定车辆的可行驶区域。

2.2 Frenet 坐标系

2.2.1 Frenet 坐标系介绍

在现实世界中，人们对于物体二维位置的描述一般基于笛卡尔坐标系。笛卡尔坐标系使用两个垂直的轴（通常是 x 轴和 y 轴）来表示空间位置，物体可以通过向这两个坐标轴投影来确定唯一坐标。同时，在投影的过程中，物体仍会保持原有的性质，不会发生变形与失真等情况，这极大地方便了坐标的标定和理解。在车辆轨迹规划中，笛卡尔坐标系主要用于描述车辆在现实世界的位置。基于车辆运动学或车辆动力学，它还可以用来表示车辆的航向角、横摆角速度等状态。通过记录不同时间的车辆位置坐标，人们可以清晰地知晓车辆的运动轨迹。然而，现实世界中道路并不总是横平竖直的，通常存在着各式各样的弯曲道路，笛卡尔坐标系难以直接描述这些弯曲道路的曲线形状。在狭窄空间或复杂地形中，笛卡尔坐标系的局部特征捕捉能力有限，无法简单清晰地描述道路边缘形状，这会影响规划结果的精度和问题的求解效率。

Frenet 坐标系，也被称为曲线坐标系，是一种专门用于描述曲线或路径的局部坐标系。它以一条参考曲线为基础，通过引入曲线的切线和法线的概念，将沿曲线切线的方向定义为 s(station)方向，表示沿曲线的行驶距离；将沿曲线法线的方向定义为 l (lateral)方向，表示相对于曲线的横向偏离程度。在 Frenet 坐标系中，物体的位置可以通过两

中国知网 https://www.cnki.net

个参数简洁地描述：s 值表示物体从起点开始沿曲线行驶的距离，l 值表示物体相对于曲线上最近投影点的横向偏离程度。这种表示方式无需考虑物体的绝对位置，仅通过 s 值和 l 值即可直观地描述物体在曲线上的相对位置。与笛卡尔坐标系相比，Frenet 坐标系能够更直观、更方便地处理与曲线相关的运动问题，在路径跟踪、轨迹规划和障碍物规避等场景得到了广泛应用。

Frenet 坐标系能够将复杂多变的道路几何形状标准化为具有左右边界的道路形式。如图 2.1 所示，当道路出现弯曲时，以道路的中心线作为参考线，通过 Frenet 坐标系的变换，原本弯曲的道路被“拉直”，同时，原本复杂的弯曲道路边界也随着坐标系的调整变得简单清晰。这种变换不仅在视觉上简化了道路的几何结构，更在数学处理上带来了极大的便利。首先，各式各样的交通参与者和交通标志物都可以被投影到参考线上，进而获得对应的横纵坐标值。这种投影方式使得车辆能够更加直观地获取周围环境信息，进而为车辆对周围环境进行决策提供了便利。其次，轨迹规划中的非线性边界约束在 Frenet 坐标系下可以被转换为线性边界约束，这可以极大地降低问题的复杂性，使得后续的求解过程更加高效。再次，Frenet 坐标系的一个显著优势在于其能够实现车辆运动的解耦。在传统的笛卡尔坐标系中，车辆的运动轨迹是涉及 x、y 坐标和时间 t 一个三维问题。通过坐标系之间的转换，可以将原有的三维问题分解为两个二维问题：S-L 和 S-T。其中，S-L 描述了车辆在道路平面上的位置，S-T 则描述了车辆的行驶距离与时间的关系。这种解耦不仅简化了问题的复杂程度，而且还提高了轨迹规划问题的实时性。综上所述，在设计轨迹规划算法时，合理使用 Frenet 坐标系能够有效简化问题，降低算法开发的难度。

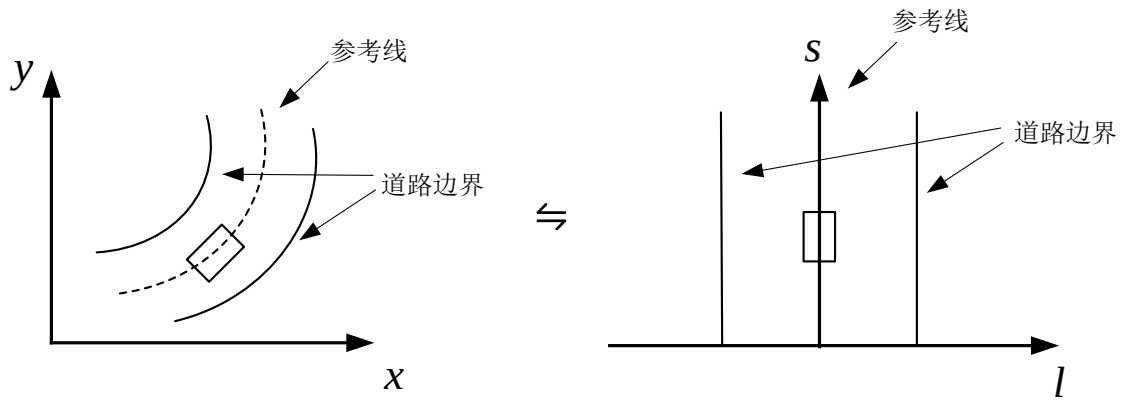


图 2.1 笛卡尔坐标系 Frenet 坐标系相互转换示意图

尽管 Frenet 坐标系在轨迹规划领域具有上述诸多显著优点，但它同时也存在一些

中国知网 https://www.cnki.net

不容忽视的缺点。首先，在道路曲率较大的区域，Frenet 坐标系可能会出现失真现象。如图 2.2 所示，当车辆处于弯曲道路时，经过坐标系转换后，车辆的形状可能会变得不规则，甚至可能出现两个顶点之间以曲线连接的情况。此外，如果采用轴对齐膨胀的方法处理障碍物形状，可能会进一步压缩求解空间，从而影响轨迹规划结果的准确性。

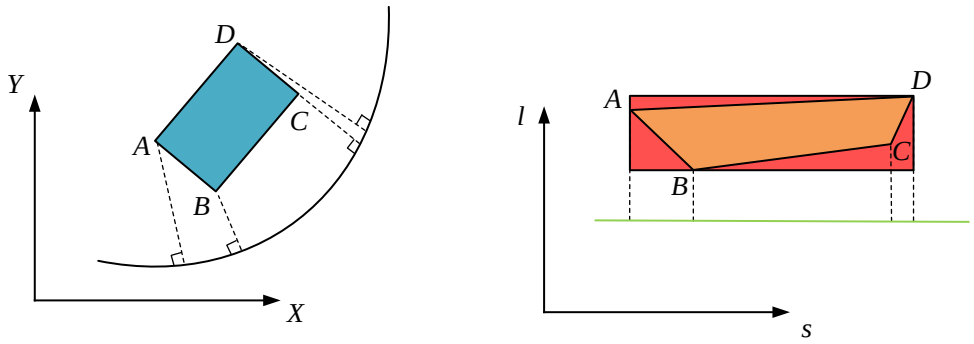


图 2.2 障碍物投影示意图

其次，Frenet 坐标系还存在坐标值映射不唯一的情况。假设参考线由一组点 $\mathrm{ref}=\{(x_i^w,y_i^w)\}$ 表示，对于二维平面上的点 $P=(x_p,y_p)$，定义匹配点 P_{ref} 为沿参考线最接近点 P 的点，具体可以用下面的公式对应表示：

$$P_{\mathrm{ref}}=\arg\min_{(x_i^w,y_i^w)\in\mathrm{ref}}\left\|\left(x_i^w-x_p,y_i^w-y_p\right)\right\| \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots(2.1)$$

如图 2.3 所示，当参考线的一部分由圆弧段构成时，对于图中的点 P_0 来说，红色圆弧内存在无穷多个位于参考线上的点，这些点均能满足公式(2.1)的要求，此时匹配点 P_{ref} 的 s 坐标也随之无法唯一确定。

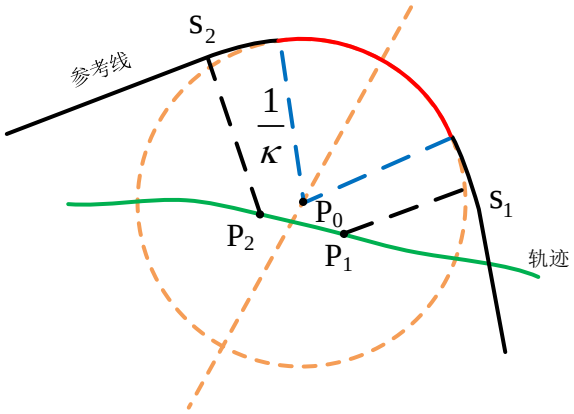


图 2.3 坐标值映射关系示意图

最后，即使不考虑坐标值映射问题，Frenet 坐标系仍存在其他问题。在图 2.3 中，P_1、P_2 两个点在笛卡尔坐标系中相距不远，但转换后对应的 s 值分别为 s_1、s_2，两点在

中国知网 https://www.cnki.net

s 维度上的差距比实际情况要大很多，这说明在两个坐标系坐标转换的过程中还可能出现轨迹不连续的情况。

2.2.2 坐标系转换关系

如图 2.4 所示，假设在笛卡尔坐标系下存在一条轨迹和一条参考线，以参考线为基础建立 Frenet 坐标系，现在想要将轨迹上的点由笛卡尔坐标系转换至 Frenet 坐标系进行表示。

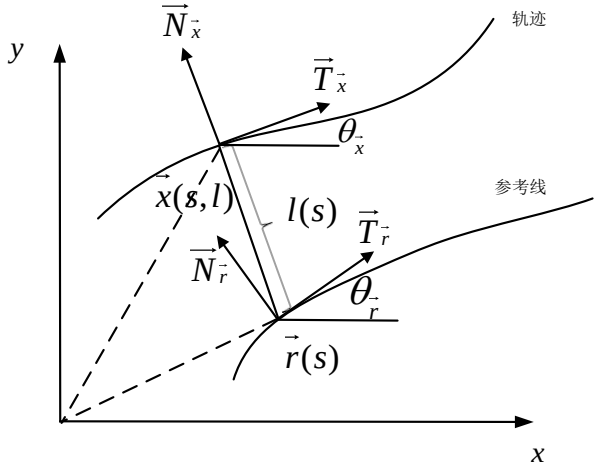


图 2.4 笛卡尔坐标系与 Frenet 坐标系转换关系示意

假设轨迹上某一点表示为 $\vec{x}(s,l)$，该点沿轨迹的切向量表示为 $\vec{T}_{\vec{x}}$，法向量表示为 $\vec{N}_{\vec{x}}$，$\vec{T}_{\vec{x}}$ 的水平夹角表示为 $\theta_{\vec{x}}$，该点在参考线上的投影点表示为 $\vec{r}(s)$，该投影点的切向量表示为 $\vec{T}_{\vec{r}}$，法向量表示为 $\vec{N}_{\vec{r}}$，$\vec{T}_{\vec{r}}$ 的水平夹角表示为 $\theta_{\vec{r}}$。$\vec{x}(s,l)$ 和 $\vec{r}(s)$ 的距离表示为 $l(s)$。根据转换过程需要，本文在笛卡尔坐标系中定义变量如下：v_x 表示自车在该点的速度，方向沿 $\vec{T}_{\vec{x}}$，a_x 表示自车在该点的加速度，κ_x 表示该点的曲率。同样在 Frenet 坐标系中定义变量如下：定义 $\dot{s}=\dfrac{ds}{dt}$ 和 $\ddot{s}=\dfrac{d\dot{s}}{dt}$ 分别表示自车在该点的纵向速度和纵向加速度，同理，定义 $\dot{l}$ 和 $\ddot{l}$ 分别表示自车在该点的横向速度和横向加速度。定义横向坐标对纵向坐标的一阶导数和二阶导数分别为 $l'=\dfrac{dl}{ds}$ 和 $l''=\dfrac{dl'}{ds}$。定义车辆航向偏差角为 $\Delta\theta=\theta_{\vec{x}}-\theta_{\vec{r}}$。

为方便后续推导，首先求取坐标轴对弧长的导数。如图 2.5 所示，假设在曲线中存在两个点 s_1 和 s_2 无限接近，分别用 $\vec{T}_1$、$\vec{N}_1$ 和 $\vec{T}_2$、$\vec{N}_2$ 表示这两个点对应的切向量和法向量。根据前文定义可以表示为：

中国知网 https://www.cnki.net

$$\frac{d\vec{T}}{ds}=\lim_{\Delta\to 0}\frac{\vec{T}_1-\vec{T}_2}{\Delta s} \quad\cdots\cdots(2.2)$$

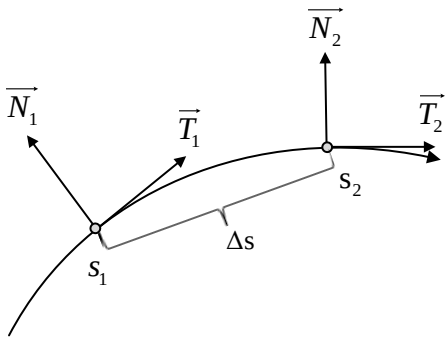


图 2.5 坐标系微分图

设向量$\vec{T}_1$和$\vec{T}_2$间的夹角为$\Delta\theta$，根据假设条件有Δs足够小，所以$\vec{T}_1$和$\vec{T}_2$足够接近，又因为向量$\vec{T}_1$和$\vec{T}_2$均为单位向量，所以$\Delta\theta$趋近于 0，此时$\sin(\Delta\theta)\approx\Delta\theta$，有公式如下：

$$\frac{d\vec{T}}{ds}=\lim_{\Delta\to 0}\frac{\vec{T}_1-\vec{T}_2}{\Delta s}=\lim_{\Delta\to 0}\frac{\Delta\theta\vec{N}}{\Delta s}=\kappa\vec{N} \quad\cdots\cdots(2.3)$$

同理可得公式如下：

$$\frac{d\vec{N}}{ds}=-\kappa\vec{T} \quad\cdots\cdots(2.4)$$

因为切向量与法向量都是单位向量，所以有公式如下：

$$\begin{cases}\vec{T}_{\vec{r}}=\left[\cos\theta_r,\sin\theta_r\right]^T\\ \vec{N}_{\vec{r}}=\left[-\sin\theta_r,\cos\theta_r\right]^T\\ \vec{T}_{\vec{x}}=\left[\cos\theta_x,\sin\theta_x\right]^T\\ \vec{N}_{\vec{x}}=\left[-\sin\theta_x,\cos\theta_x\right]^T\end{cases} \quad\cdots\cdots(2.5)$$

基于向量运算法则，有公式如下：

$$\vec{x}(s,l)=\vec{r}(s)+l(s)\vec{N}_{\vec{r}} \quad\cdots\cdots(2.6)$$

简写为：

$$\vec{x}=\vec{r}+l\vec{N}_r \quad\cdots\cdots(2.7)$$

因此和l关系有公式如下：

$$l=[\vec{x}-\vec{r}]^T\vec{N}_r \quad\cdots\cdots(2.8)$$

因此l的长度在笛卡尔坐标系下可如下表示：

$$l=\sqrt{(x-x_r)^2+(y-y_r)^2} \quad\cdots\cdots(2.9)$$

公式(2.8)两端对时间求导可得：

中国知网 https://www.cnki.net

$$\dot{l}=[\dot{\vec{x}}-\dot{\vec{r}}]^T\vec{N}_r+[\vec{x}-\vec{r}]^T\dot{\vec{N}}_r \quad\cdots\cdots(2.10)$$

其中：

$$\dot{\vec{x}}=\frac{d\vec{x}}{dt}=v_x\overrightarrow{T_x} \quad\cdots\cdots(2.11)$$

同样：

$$\dot{\vec{r}}=\dot{s}\overrightarrow{T_r} \quad\cdots\cdots(2.12)$$

同时有：

$$\dot{\vec{N}}_r=\frac{d\dot{\vec{N}}_r}{dt}=\frac{d\dot{\vec{N}}_r}{dt}\frac{ds}{dt}=-\kappa\dot{s}\overrightarrow{T_r} \quad\cdots\cdots(2.13)$$

综合上面的公式可得：

$$\dot{l}=[v_x\overrightarrow{T_x}-\dot{s}\overrightarrow{T_r}]^T\overrightarrow{N_r}-l\overrightarrow{N_r}\kappa\dot{s}\overrightarrow{T_r} \quad\cdots\cdots(2.14)$$

因为$\vec{T}_{\vec{r}}$和$\vec{N}_{\vec{r}}$正交，所以有：

$$\dot{l}=v_x\overrightarrow{T_x}^T\overrightarrow{N_r} \quad\cdots\cdots(2.15)$$

代入可得：

$$\dot{l}=v_x\overrightarrow{T_x}^T\overrightarrow{N_r}=v_x(\cos\theta_r\sin\theta_x-\sin\theta_r\cos\theta_x)=v_x\sin(\Delta\theta) \quad\cdots\cdots(2.16)$$

等式两边同时对时间求导可得：

$$\dot{\vec{x}}=\dot{\vec{r}}+\dot{l}\vec{N}_r+l\dot{\vec{N}}_r \quad\cdots\cdots(2.17)$$

将上述结果带入可得：

$$\dot{\vec{x}}=\dot{s}\overrightarrow{T_r}+\frac{dl}{dt}\overrightarrow{N_r}-l\kappa\dot{s}\overrightarrow{T_r}=\dot{s}(1-\kappa l)\overrightarrow{T_r}+\dot{l}\overrightarrow{N_r} \quad\cdots\cdots(2.18)$$

求解纵向速度可得：

$$v_x=\left\|\dot{\vec{x}}\right\|=\sqrt{\dot{s}^2(1-\kappa l)^2+\dot{l}^2} \quad\cdots\cdots(2.19)$$

对于公式(2.18)，两边同乘$\vec{T}_{\vec{r}}$可得：

$$\dot{\vec{x}}\overrightarrow{T_r}=\dot{s}(1-\kappa l)\overrightarrow{T_r}\overrightarrow{T_r}+\dot{l}\overrightarrow{N_r}\overrightarrow{T_r} \quad\cdots\cdots(2.20)$$

由公式(2.11)和公式(2.16)可得：

$$\dot{s}=\frac{v_x\cos(\Delta\theta)}{1-\kappa l} \quad\cdots\cdots(2.21)$$

最终，成功推导出从笛卡尔坐标系到 Frenet 坐标系的转换公式，具体如下所示：

中国知网 https://www.cnki.net

$$\begin{cases} \|l\| = \sqrt{(x-x_r)^2+(y-y_r)^2} \\ \dot{l} = v_x \sin(\Delta\theta) \\ \dot{s} = \dfrac{v_x \cos(\Delta\theta)}{1-\kappa l} \\ l' = (1-\kappa l)\tan(\Delta\theta) \\ l'' = (\kappa l' + \kappa' l)\tan(\Delta\theta) + \dfrac{1-\kappa l}{\cos^2(\Delta\theta)}\left(\dfrac{1-\kappa l}{\cos(\Delta\theta)}\kappa_x - \kappa_r\right) \\ \dot{v}_x = \ddot{s}\dfrac{1-\kappa l}{\cos(\Delta\theta)} - \dfrac{\dot{s}^2}{\cos^2(\Delta\theta)}\left[(1-\kappa l)\tan(\Delta\theta)(\Delta\theta)' - (\kappa l' + \kappa' l)\right] \end{cases} \quad \cdots\cdots\cdots\cdots (2.22)$$

关于从 Frenet 坐标系转换到笛卡尔坐标系的转换公式，可以通过类似的方法进行推导。鉴于篇幅有限，本文不再详细展开，直接提供相应的转换公式如下：

$$\begin{cases} x_x = x_r - l\sin(\theta_r) \\ y_x = y_r + l\cos(\theta_r) \\ \theta_x = \arctan\left(\dfrac{l'}{1-\kappa_r l}\right) + \theta_r \\ v_x = \sqrt{[\dot{s}(1-\kappa_r l)]^2 + (\dot{s}l')^2} \\ a_x = \ddot{s}\dfrac{1-\kappa_r l}{\cos(\theta_x-\theta_r)} + \dfrac{\dot{s}^2}{\cos(\theta_x-\theta_r)}\left[l'\left(\kappa_x\dfrac{1-\kappa_r l}{\cos(\theta_x-\theta_r)} - \kappa_r\right) - (\kappa_r' l + \kappa_r l')\right] \\ \kappa_x = \left(\left(l'' + (\kappa_r' l + \kappa_r l')\tan(\theta_x-\theta_r)\right)\dfrac{\cos^2(\theta_x-\theta_r)}{1-\kappa_r l} + \kappa_r\right)\dfrac{\cos(\theta_x-\theta_r)}{1-\kappa_r l} \end{cases} \quad \cdots\cdots (2.23)$$

在实现两个坐标系之间的相互转化后，规划开始时可以先将车辆的初始状态转换到 Frenet 坐标系下，规划结束后再将规划结果转换回笛卡尔坐标系。这样做可以降低算法的开发难度，但不能避免 Frenet 坐标系的缺点，因此在设计算法时应当结合两个坐标系的各自优点，以有效提升算法的适用性。

2.3 参考线生成

自动驾驶系统在车辆行驶过程中会通过感知模块和定位模块获取周围环境信息，如基础道路信息、交通参与者信息等。其中，基础道路信息包括车道的几何参数、交通标线、限行信息以及道路边缘特征等。

在城市道路场景中，为了保障交通秩序和行车安全，要求车辆在行驶过程中遵循道路方向，禁止倒车行驶。因此，车辆的行驶方向应与车道的切线方向保持一致，且行驶轨迹应尽量靠近车道中心线。当车辆在具有清晰明确的车道边界线的一般结构化道

中国知网 https://www.cnki.net

路场景中行驶时，车道中心线可以作为行驶的有效参考线。本节将详细介绍如何利用贝塞尔曲线在无明确车道线的特殊场景中生成参考线，以确保在行驶过程中始终为自动驾驶车辆提供一条连续且优质的行驶参考线。

2.3.1 贝塞尔曲线基本性质

贝塞尔曲线作为一种插值曲线，最初在汽车外形设计领域发挥了重要作用。得益于其卓越的数学特性，贝塞尔曲线后来被广泛应用于计算机图形学、动画制作和工业设计等多个领域。这种曲线通过一系列有序的控制点来定义形状，通过调整这些控制点的位置，可以灵活地改变曲线的形态，展现出极高的适应性。高阶贝塞尔曲线能够更精确地逼近控制点的位置，同时提供更精细的曲线形状调整能力。正因如此，近年来贝塞尔曲线也被引入到无人驾驶轨迹规划领域，并得到广泛应用。

贝塞尔曲线定义在固定的区间 $t \in [0,1]$ 上，其形状由一组固定的控制点 $P_i\ (i = 0,1,\ldots,n)$ 组成的特征多边形决定，其中 n 表示基底的度数，同时也与曲线的阶数有关。由这些控制点 P_i 确定的 n 阶贝塞尔曲线的数学表达式为：

$$B(t) = \sum_{i=0}^{n} p_i \cdot b_n^i(t), t \in [0,1] \quad \cdots\cdots (2.24)$$

式中 $B(t)$ 表示时间为 t 时曲线上对应点的坐标，p_i 为决定曲线形状的控制点，伯恩斯坦基函数 $b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$ 。

在轨迹规划中，一般会使用到贝塞尔曲线的如下性质：

（1）如图 2.6(a)所示，贝塞尔曲线的阶数取决于控制点的数量，n 阶贝塞尔曲线对应着 n+1 个控制点。

（2）如图 2.6(b)所示，如果决定贝塞尔曲线形状的控制点能够构成一个凸多边形，那么根据伯恩斯坦基函数的性质，由这些控制点生成的贝塞尔曲线一定处于对应的凸多边形内部。

（3）如图 2.6(c)所示，贝塞尔曲线始终经过首尾两个控制点，即曲线的起点和终点分别对应着特征多边形的第一个点 P_0 和最后一个点 P_n，并且曲线在起点和终点处的切线与特征多边形的第一条边和最后一条边相重合。贝塞尔曲线不直接通过非首尾控制点，但这些非首尾控制点会影响曲线的部分形状，可以通过改变控制点的位置来局

中国知网 https://www.cnki.net

部修改曲线而不影响整体结构。

（4）贝塞尔曲线 $B(t)$ 的 k 阶导数 $B^{(k)}(t)$ 仍然是贝塞尔曲线，且 $B^{(k)}(t)$ 所有控制点 $q_i^{(k)}$ 可由原始贝塞尔曲线的控制点 p_i 线性表示出来：

$$q_i^{(k)} = \frac{m!}{(m-k)!}\left(q_{i+1}^{(k-1)} - q_i^{(k-1)}\right) \quad (2.25)$$

（5）三阶及更高阶的贝塞尔曲线的曲率是连续的。证明过程如下：

根据定义，贝塞尔曲线的一阶导数和二阶导数可以表示为：

$$\dot{B}(t) = \sum_{i=0}^{n-1} b_{i,n-1}(t)(P_{i+1} - P_i) \quad (2.26)$$

$$\ddot{B}(t) = n(n-1)\sum_{i=0}^{n-2} b_{i,n-2}(t)(P_{i+2} - 2P_{i+1} + P_i) \quad (2.27)$$

贝塞尔曲线的曲率可以表示为：

$$\kappa(t) = \frac{\dot{B}_x(t)\ddot{B}_y(t) - \dot{B}_y(t)\ddot{B}_x(t)}{\left(\dot{B}_x^2(t) + \dot{B}_y^2(t)\right)^{3/2}} \quad (2.28)$$

如图 2.6(d)所示，由式(2.28)可知，当贝塞尔曲线存在二阶导数时，曲率是连续的。

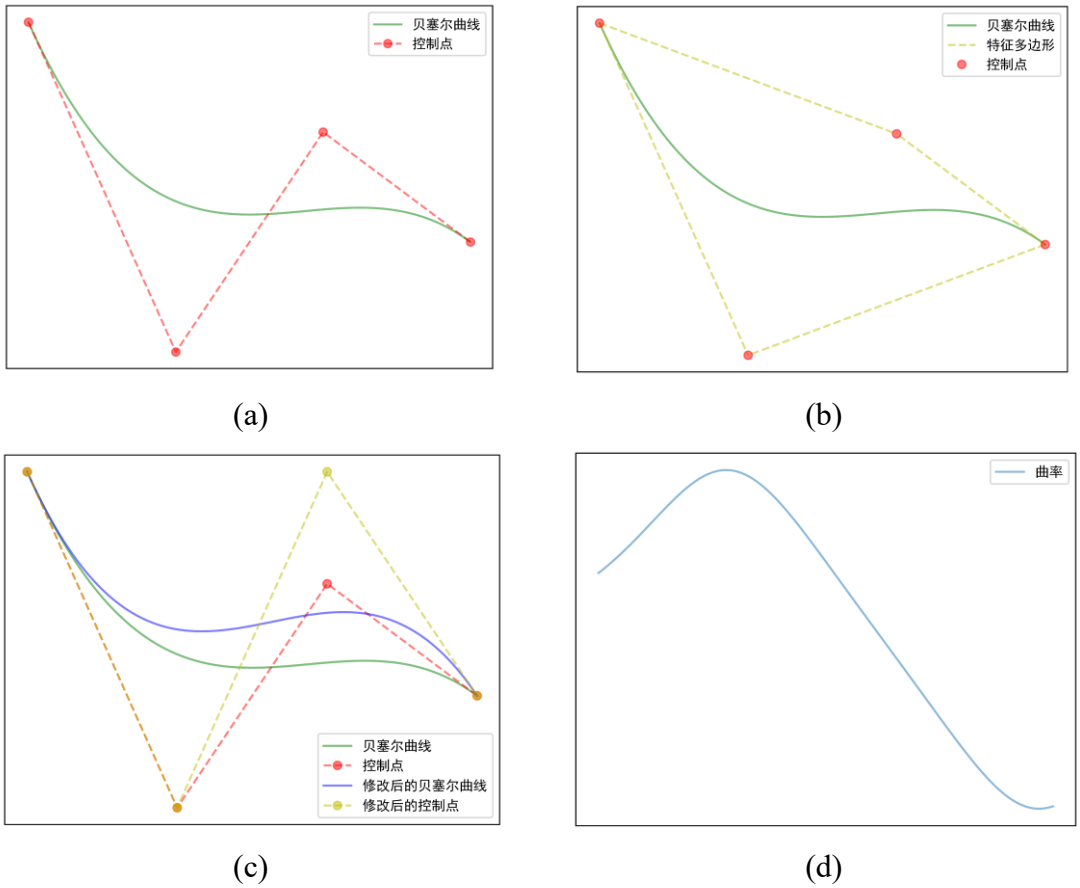


图 2.6 贝塞尔曲线的相关性质

中国知网 https://www.cnki.net

2.3.2 特殊场景参考线生成

在城市道路环境中，一般的结构化道路通常具有清晰且明确的车道边界，这使得车辆可以轻松地将车道中心线作为行驶时的参考线。然而，在交叉路口等特殊场景，由于缺乏明显可用的车道边界，车辆无法直接根据车道边界生成参考线使用。因此，本文将充分利用上文分析的贝塞尔曲线的数学特性，在特殊场景中生成平滑的参考线，并将其与一般结构化道路的车道中心线有机结合，以实现不同车道之间的连续过渡，确保车辆在行驶过程中始终有一条曲率连续且平顺的参考线作为轨迹规划的依据，从而保障车辆行驶过程中的平稳性与安全性。

如图 2.7 所示，当自车行驶至交叉路口将要转向、自车将要更换车道或自车处于掉头场景时，都需要生成一条连接前后车道的参考线。接下来，本文将详细阐述在上述场景中如何基于五阶贝塞尔曲线生成高质量的参考线，以实现不同车道之间的平滑过渡与连续衔接。

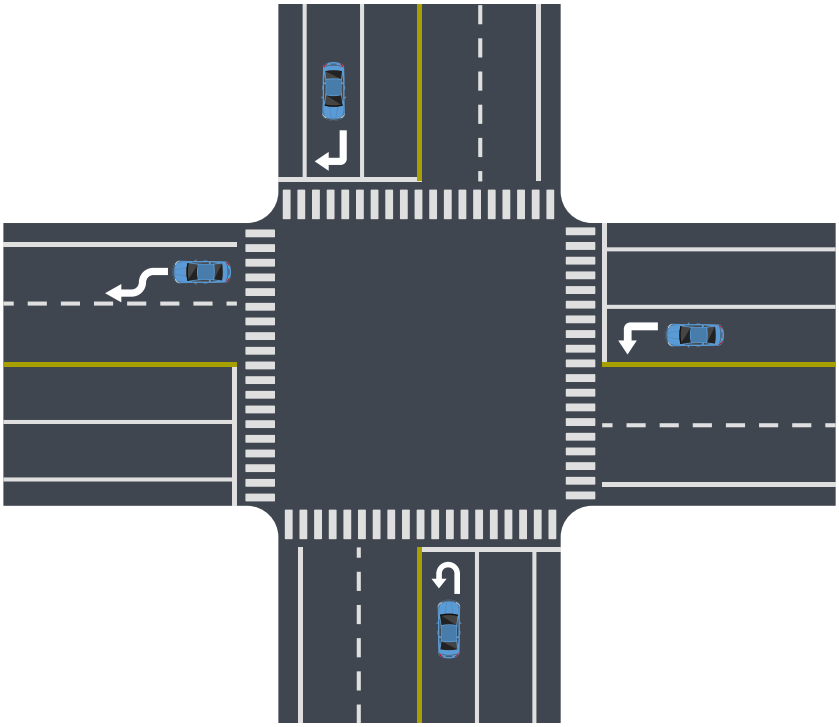


图 2.7 车辆特殊场景行为示意图

对于需要连接的两个车道，本文首先分别选取前一车道中心线末端的两个点和后一车道中心线前端的两个点，根据这些点，可以构造出两条分别描述前后车道中心线的直线，记为 L_f 和 L_r。根据两条车道不同的空间情况，表示前后车道中心线的两条直线之间也有着不同的几何位置关系。一般而言，当车辆进行换道或掉头操作时，两条车

中国知网 https://www.cnki.net

道中心线通常呈平行关系；而当车辆在交叉路口处进行转向时，两条车道中心线则通常不平行。因此，可以通过分析两条直线的斜率关系来确定车道中心线之间的几何关系，从而判断其是否平行。基于上述几何特征，本文将针对不同的典型驾驶场景，分别设计相应的参考线生成策略。

首先考虑两条车道中心线相互平行的情形。在确定车道中心线平行后，假设前一车道中心线的终止点为 R_{f0}，后一车道中心线的起始点为 R_{r0}。在前一车道中从 R_{f0} 开始沿着车道中心线前向延伸 L 米和 $2L$ 米处选取点 R_{f1} 和 R_{f2}，在后一车道中从 R_{r0} 开始沿着车道中心线后向延伸 L 米和 $2L$ 米处选取点 R_{r1} 和 R_{r2}。其中，L 的数值的选取需要考虑曲率变化和特定行驶场景灵活选择，在图 2.8 所示的换道场景和掉头场景中，本文均取 $L=5$ 来生成参考线，图中绿色实线均代表原有车道中心线。重新选取后的六个点 R_{f2}、R_{f1}、R_{f0}、R_{r0}、R_{r1} 和 R_{r2} 可依照车辆的行驶方向构成贝塞尔曲线的控制点，进而生成一条平滑连续的贝塞尔曲线，以连接两段车道中心线，共同构成车辆行驶过程中的参考线，如图 2.9 所示，其中，红色点表示用来生成贝塞尔曲线的控制点。

图 2.8 换道场景示意图（左）和掉头场景示意图（右）

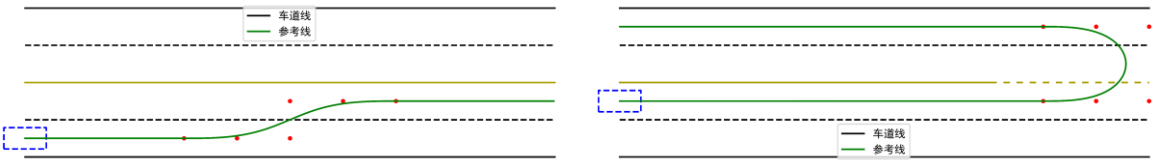


图 2.9 换道场景参考线（左）和掉头场景参考线（右）

其次，本文进一步考虑两条车道中心线非平行的情形。如图 2.10 所示，当车辆行驶至交叉路口将要左转时，除需已知的两段车道中心线的终点和起点外，为保证生成的参考线曲率连续，还需额外计算四个控制点，从而形成五阶贝塞尔曲线。具体方法为：首先求取前后两段车道中心线直线 L_1 与 L_2 的延长线交点，以此交点作为基准，将前段车道中心线的终点和后段车道中心线的起点分别与该交点进行加权计算，进而得到位于车道中心线延长线上的四个控制点。与车道中心线平行的情形类似，依据车辆

中国知网 https://www.cnki.net

行驶距离依次选取并利用这些控制点构成五阶贝塞尔曲线，以实现两段非平行车道中心线之间的平滑连接，生成车辆行驶时所需的连续参考线。如图 2.11 所示，本文所用的权重系数分别为 0.4 和 0.7，图中蓝色点表示两条车道中心线延长线的交点，红色点的含义与前文保持一致。

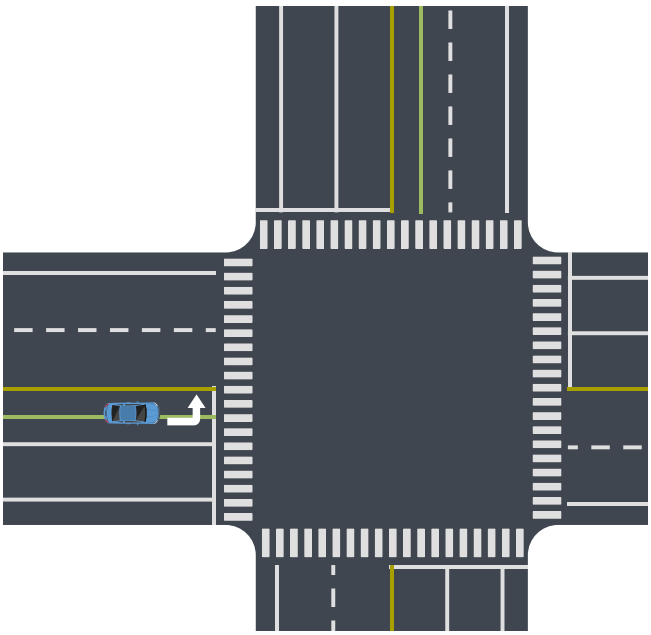


图 2.10 交叉路口左转场景

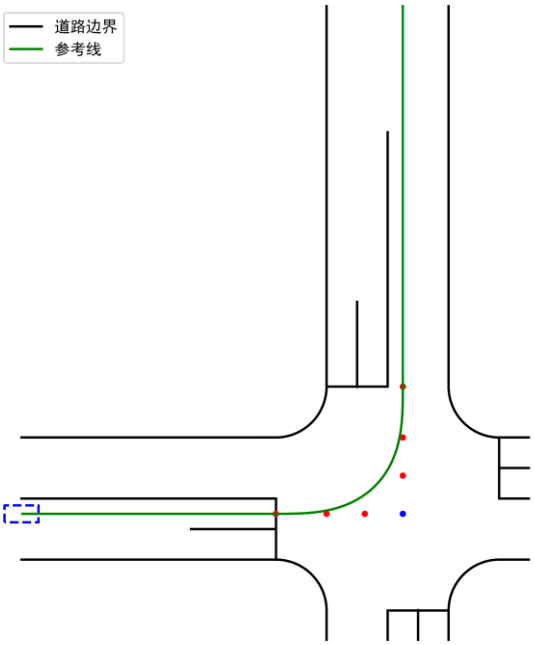


图 2. 11 交叉路口左转场景参考线

通过上述方法，可以在特殊场景下生成适用于车辆行驶的参考线。这些参考线与一般结构化道路场景的车道中心线相结合，能够构成一条连续且平滑的参考线，为车

中国知网 https://www.cnki.net

辆的安全行驶提供稳定可靠的指引。此外，生成的参考线还可作为构建 Frenet 坐标系的基础，为后续三维规划空间的建立提供有力支撑。

2.4 规划空间构建

传统的二维地图仅包含物体的空间位置信息，缺乏对时间维度和运动状态的描述，因此难以清晰地表达物体的运动轨迹。尽管基于图搜索和采样的路径规划算法在静态环境中得到了广泛应用，但由于这些算法忽略了时间因素，所以它们无法直接生成符合动态需求的可行驶轨迹。为了解决这一问题，许多轨迹规划算法采用将三维轨迹规划问题解耦为路径规划和速度规划两个子问题的策略。其核心思想是：首先，在二维地图中考虑静态障碍物的分布，通过路径规划算法生成一条从起点到终点的可行路径，该路径仅关注空间上的连通性，而不涉及时间因素。随后，在已规划出的路径上，考虑动态障碍物的运动以及车辆自身的动力学约束，通过速度规划算法确定车辆在路径上的行驶速度。最后，将路径和速度信息融合，形成一条包含时间和空间信息的完整轨迹，供自动驾驶车辆在实际行驶过程中使用。

这种分步规划策略能够将复杂的三维轨迹规划问题拆解为两个相对简单的低维子问题，从而显著降低任务难度与计算复杂度。该策略使得路径规划与速度规划能够分别针对静态环境与动态环境进行优化，进而提升算法的效率及可扩展性。然而，这种解耦方法同时也存在一定的局限性。由于路径规划与速度规划是分开进行的，其内在耦合关系被弱化，进而可能导致求解空间被压缩。这种压缩效应在面对复杂动态环境时，可能限制轨迹生成的灵活性与适应性，甚至可能产生次优或不可行的轨迹。

时空联合规划则在三维时空范围内将车辆的路径和速度同时作为优化变量，直接进行轨迹规划。这种方法能够在复杂的动态环境中生成更具灵活性和适应性的可行轨迹。与传统的时空解耦规划方法相比，时空联合规划由于问题维度的增加，可能导致求解效率下降，但其显著优势在于提供了更为广阔的求解空间。该规划方法能够同时考虑静态和动态障碍物的影响，避免了时空解耦方法中路径规划与速度规划分离所引发的局限性，从而能够在复杂场景下生成更为优越的轨迹。

本节基于前文生成的参考线，综合考虑道路中的交通信号与交通参与者，针对不同维度采用不同的分辨率参数，这些参数分别记为 s_{space}、l_{space}、t_{space}，并在后文中给出具体数值。根据这些分辨率参数，本文构建出相应的三维时空(S-L-T)规划空间。在该

中国知网 https://www.cnki.net

空间内，车辆的运动轨迹不仅受到道路几何信息的约束，还需考虑各类障碍物的位置以及交通信号的时序变化。车辆的运动状态在三维规划空间中通过纵向位置(S)、横向位置(L)和时间(T)三个维度进行建模。其中，S 代表车辆沿道路参考线的前进距离，L 代表车辆相对于道路参考线的横向偏移量，T 则代表车辆行驶过程中的时间进程。引入时间维度后，三维规划空间能够更全面地描述车辆与周围环境的动态交互，后续的时空参考轨迹也将在这一规划空间内生成。

如图 2.12 所示，在车辆行驶过程中，所有交通参与者均被视为障碍物，并根据其运动特性进一步被划分为静态障碍物和动态障碍物。静态障碍物指的是道路上速度为零的交通参与者，如静止的车辆、交通锥等。在三维规划空间中，静态障碍物在时间轴上占据固定的 S-L 区域，其位置和范围在时间维度上保持不变。与之相比，动态障碍物指的是道路上速度不为零的交通参与者，包括正在行驶的车辆、行人及非机动车等。这类障碍物在三维规划空间中呈现出明显的动态特征，其占据的 S-L 区域会随时间的推移而发生变化。

动态障碍物在未来一段时间内的位置应由预测模块进行预测生成。考虑到本文的研究重点在于轨迹规划算法，为简化问题，本文假设所有动态障碍物均沿既定方向以既定速度行驶，且不考虑多模态运动的可能性。

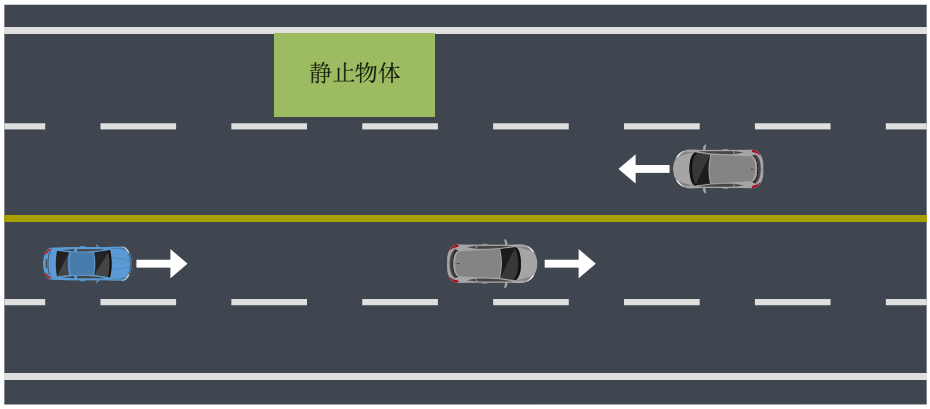


图 2.12 道路情况示意图

在车辆行驶过程中，除了需要关注其他交通参与者外，还应充分考虑各类交通信号的影响，如红绿灯和限行信息等。这些交通信号同样可以在三维规划空间中得到有效表示，从而为车辆轨迹规划提供更为精准的约束条件。以红绿灯为例，在三维规划空间中，红灯可被定义为在特定时间范围内，路口处的 S-L 空间被完全占据，车辆在此时间段内被禁止通行。而绿灯则不对规划空间产生通行约束，车辆可正常通过路口。类似地，当车道两侧存在实线等禁行区域时，其在三维规划空间中的表示方式与红灯的

中国知网 https://www.cnki.net

处理方法相似。具体而言，在所有时间范围内，这些禁行区域对应的 S-L 空间将被完全占据，车辆被明确禁止穿越这些区域。通过在规划空间中合理表示交通信号，可以清晰地为车辆标示出不可通行区域，从而有效避免闯红灯、跨越实线等违规行为的发生。

综合以上分析，以图 2.12 中交通参与者的运动情况及交通信号的约束条件为例，本文构建如图 2.13 所示的三维规划空间进行说明。在该规划空间中：绿色栅格表示图 2.12 中的静态障碍物；蓝色栅格代表图 2.12 中的自车；灰色栅格对应图 2.12 中的其他社会车辆；红色栅格则用于标识交通信号灯的影响区域。

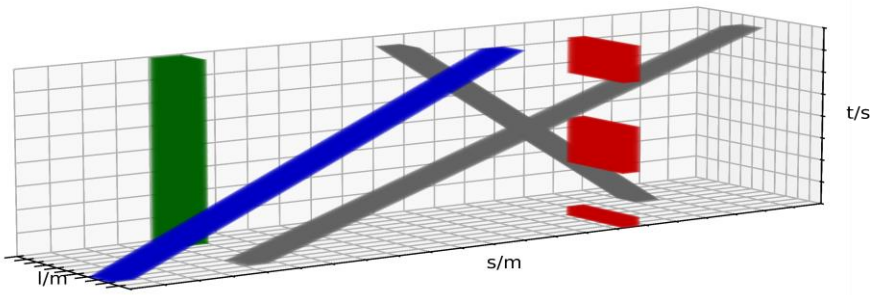


图 2.13 三维规划空间

图 2.13 清晰地展示了三维规划空间相较于传统二维空间的优势。通过引入时间维度，三维规划空间能够精确的描述未来一段时间内自车与障碍物之间的空间关系和相对运动状态。在该三维规划空间中，轨迹规划算法能够充分考虑交通信号的约束以及交通参与者的动态行为，为后续的时空参考轨迹生成与时空联合轨迹优化提供更加丰富和精准的信息。

2.5 本章小结

本章首先对轨迹规划领域中常用的笛卡尔坐标系与 Frenet 坐标系进行深入剖析，分别详细解析了二者的优势与不足。紧接着，推导了这两种坐标系之间的转换公式，为后续的轨迹规划工作提供了重要的数学基础。随后，对贝塞尔曲线进行了全面且细致的阐释，并推导论证了其在轨迹规划领域中常用的各项性质。针对城市交通场景的特点，在车辆换道、掉头以及通过交叉路口等复杂场景中，本文使用五阶贝塞尔曲线来生成参考线，并与一般道路的车道中心线有机结合，生成了平滑且曲率连续的参考线。最后，基于所生成的参考线构建三维规划空间，深入分析不同交通信号和交通参与者的特征属性，通过在三维规划空间中对这些特性进行投影表示，进而得到车辆可行驶空

中国知网 https://www.cnki.net

间，为后续章节中算法的开发奠定了坚实基础。

中国知网 https://www.cnki.net

第3章 时空参考轨迹生成算法研究

3.1 引言

当前，多数轨迹规划算法以 Frenet 坐标系为基础，采用先规划路径、后规划速度的解耦策略。然而，这种策略忽视了速度对路径的影响，容易导致规划结果陷入局部最优。本章将在上一章工作的基础上，对时空参考轨迹生成算法展开深入研究。首先，在传统混合 A*算法中融入时间维度，并针对节点扩展方式和代价函数进行设计，将其拓展为时空混合 A*算法。时空混合 A*算法能够在规划空间内直接搜索出一条连接起始节点与终止节点的可行轨迹，不过由于其高维特性，可能会面临计算耗时较长的问题。为保证算法的实时性，本章进一步融合 ARA*算法，对时空混合 A*算法进行优化，使得算法能够在指定时间范围内生成可行的参考轨迹，从而在效率与性能之间实现平衡。最后，本章将详细阐释如何依据时空参考轨迹匹配交通边界信息并构建约束条件，为后续章节中时空联合轨迹优化过程中安全性约束的构建奠定基础。

3.2 时空混合 A*算法设计

3.2.1 时空节点扩展方案设计

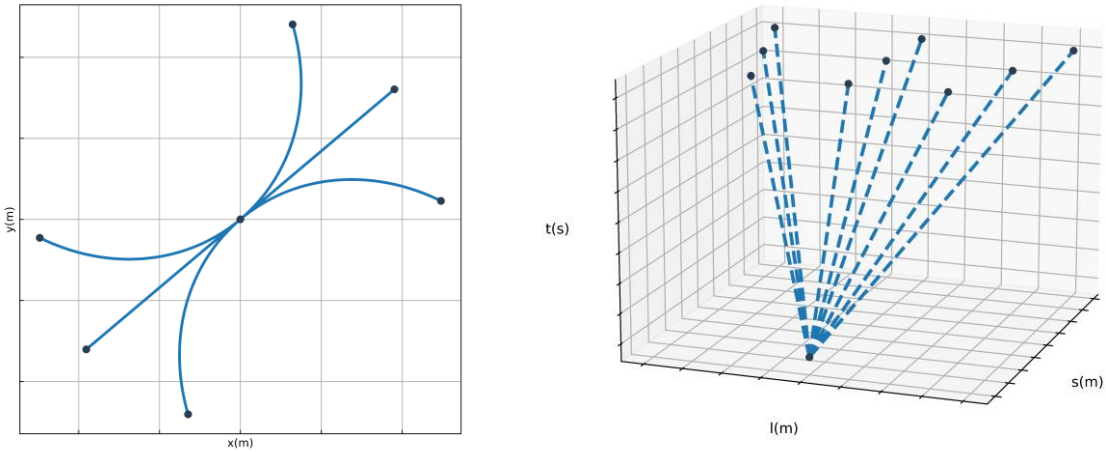
在 Frenet 坐标系下，传统的二维混合 A*算法中，节点的状态空间被定义为$\{s, l, \theta\}$，其中$\{s, l\}$分别刻画了车辆沿着参考线的纵向位置以及与之对应的横向位置，θ 则代表了车辆的航向角。节点的控制变量为前轮转角 δ，为了契合车辆的实际运动能力，并确保相邻节点之间的空间状态能够保持连续，δ 需要在区间$[\delta_{\min}, \delta_{\max}]$内离散取值。

如图 3.1(a)所示，不妨假设当前节点的状态为$\{s_0, l_0, \theta_0\}$。在此基础上，让车辆以固定速度行驶，并依据离散化后的前轮转角，对单位时间之后的运动结果进行模拟。通过分析可以发现，虽然基于固定速度的采样方式在一定程度上能够减少节点的扩展数量，但由于该方式未能充分考虑动态障碍物的影响，且固定速度本身缺乏足够的灵活性，导致在扩展节点时所采用的速度信息在实际应用中价值并不显著。在这种情况下，搜索得到的结果通常仅作为路径信息使用，若要生成完整的轨迹，则还需要进一步进行速度规划。

中国知网 https://www.cnki.net

为有效克服上述传统二维混合 A*算法存在的弊端，本文在前文所构建的三维规划空间的基础上，将原有算法改进为时空混合 A*算法，用以搜索生成时空参考轨迹。该算法在传统二维混合 A*算法的框架上进行了创新拓展，引入了时间信息，并相应地对节点的扩展方式做出了调整。

具体而言，时空混合 A*算法中节点对应的状态空间被定义为$\{s, l, \theta, v, t\}$。其中，前三项状态变量 s、l、θ 与传统混合 A*算法中的定义一致，分别表示车辆沿参考线的纵向位置、横向位置以及车辆的航向角；第四项 v 代表车辆的行驶速度；第五项 t 则用于表示扩展至当前节点时所对应的时间。为便于算法的实施和计算，本文假设本次搜索起始节点的时间状态为 t_0。同时，定义规划时域 T，用以表示一次规划所涵盖的未来时间范围。基于此设定，节点状态变量 t 的定义域被确定为$[t_0, t_0+T]$。



(a) 传统混合 A*节点扩展方式　　(b) 时空混合 A*节点扩展方式

图 3.1 传统混合 A*与时空混合 A*节点扩展方式对比

基于前文所定义的状态空间，本文依据具有非完整性约束的车辆模型，对时空混合 A*算法的控制空间进行了明确界定，将其定义为$\{\delta, a\}$。其中，δ 仍表示车辆的前轮转角，它决定了车辆的转向方向和程度，是控制车辆行驶轨迹的关键变量之一；a 代表车辆的加速度，用于控制车辆行驶速度的变化，对车辆的加速、减速过程起着决定性作用。基于前文对状态空间和控制空间的相关定义，节点的具体扩展方式可参照图 3.1(b)。具体的节点扩展过程如下：

在节点扩展阶段，本算法的节点扩展方式与传统混合 A*算法存在一定的相似性，同样需要对前轮转角 δ 和加速度 a 进行采样离散化处理。基于车辆的动力学参数，为了使车辆在节点扩展过程中能够均匀地向前行进，需要将前轮转角 δ 在区间$[\delta_{\min}, \delta_{\max}]$内进行均匀离散化。考虑到车辆转向过程中存在前轮转角为零这一特殊情况，为了保

中国知网 https://www.cnki.net

证零转角能够作为一个采样离散点被准确包含，前轮转角的离散数量应当设定为奇数。经过离散化处理后，前轮转角的集合Δ可以表示为：

$$\Delta=\left\{\delta_{\mathrm{i}}\,|\,\delta_{\max}+(\mathrm{i}\text{-}1)\times\frac{\delta_{\max}-(\delta_{\min})}{\mathrm{N}\text{-}1}\right\}\cdots\cdots(3.1)$$

式中，$i=1,2,\cdots,N$。

为了在节点扩展效率与扩展质量之间达成良好平衡，本文依据车辆的动力学参数，确定了车辆加速度所在区间为$[a_{\min},a_{\max}]$。在此区间内，采用独特的变区间间隔方式对加速度 a 进行离散化处理。具体而言，离散后的加速度以零为基准点，在该区间内向左右两侧逐步增大采样间隔来进行采样。经过上述离散化操作后，得到的加速度集 A 可以表示为：

$$\mathrm{A}=\{\mathrm{a}_{\min},\cdots,\ 0,\cdots,\ \mathrm{a}_{\max}\}\cdots\cdots(3.2)$$

在传统混合 A*算法中，搜索效果与步长紧密相关。步长设置过小，会显著增加算法的运行耗时；步长设置过大，又会降低搜索结果的质量。而本文所采用的时空混合 A*算法，能够借助离散化的加速度实现变步长的效果。结合离散后的前轮转角和加速度，对于任意一个父节点 $N_{\mathrm{p}}(s_{\mathrm{p}},l_{\mathrm{p}},\theta_{\mathrm{p}},v_{\mathrm{p}},t_{\mathrm{p}})$，本文将依据图 3.2 所示的车辆运动学模型来进行节点扩展。这样做的目的在于确保搜索结果严格符合车辆运动学约束条件，从而降低后续轨迹优化的难度。在本文中，两个节点之间的时间差被设定为一个恒定值，用 dt 来表示。经过父节点 N_{p} 扩展后得到的任一子节点 $N_{\mathrm{i}}(s_{\mathrm{i}},l_{\mathrm{i}},\theta_{\mathrm{i}},v_{\mathrm{i}},t_{\mathrm{i}})$与 N_{p} 之间均存在如下关系：

$$\begin{cases}s_i=s_p+v_p\times\cos\theta_i\times dt\\ l_i=l_p+v_p\times\sin\theta_i\times dt\\ \theta_i=\theta_p+v_i\times\dfrac{\tan\delta_i}{L}\times dt\\ v_i=v_p+a_i\times dt\\ t_i=t_p+dt\end{cases}\cdots\cdots(3.3)$$

式中 L 代表车辆的轴距；$a_{\mathrm{i}}\in\mathrm{A}$；$\delta_{\mathrm{i}}\in\Delta$。

中国知网 https://www.cnki.net

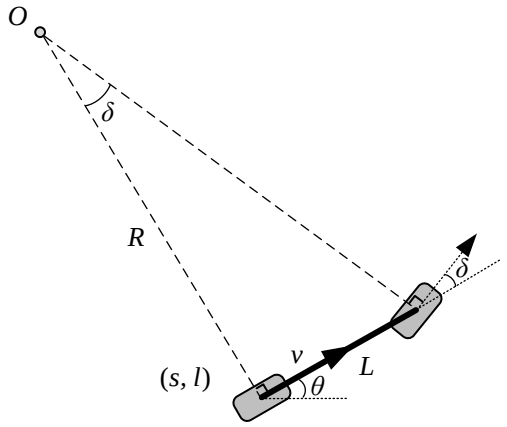


图 3.2 车辆运动学模型

对前轮转角集Δ和加速度集 A 进行全面遍历后，能够得到当前节点对应的一系列子节点。尽管这些子节点与父节点之间满足了车辆运动学约束条件，但它们未必能同时满足其他约束要求。所以，有必要对这些子节点进行进一步筛选，将那些物理上不可行的节点排除掉。

首先是速度约束检查。本算法主要应用于城市结构化道路场景。在城市道路中行车时，交通规则不允许车辆倒车行驶，所以子节点所对应的车辆速度必须大于或等于0。同时，城市道路通常会设置相应的限速标准，车辆行驶时需要严格遵守这些限速规定，这就要求子节点的速度不能超过规定的最高限速 $v_{\max}$。综上所述，子节点的速度 v_{i} 应当处于$[0, v_{\max}]$这个区间内。若子节点的速度不符合这一要求，便舍弃该节点。

其次是碰撞约束检查。在本算法中，每个子节点都对应着特定的时间状态。在三维规划空间里，一旦明确了时间状态，也就能够确定该时刻周围交通参与者的具体位置信息。基于此，如果某个子节点所代表的车辆位置与其他交通参与者在该时间状态下发生了碰撞，那么这个子节点所对应的状态在实际行驶中是不可行的，需要将该节点舍弃，以确保规划出的轨迹不会出现碰撞事故。

最后是道路边界约束检查。在实际行驶中，车辆必须在车道内行驶，这是保障交通安全与合规性的基本要求。根据子节点的横向坐标 l_{i}，算法能够精确检测车辆相对于参照线的位置，进而判断车辆是否处于车道边界内。如果发现子节点对应的车辆位置不在车道范围内，那么该节点所代表的状态是不符合实际道路行驶要求的，这种情况下需要舍弃该节点。

在舍弃那些未能通过上述约束检查的子节点后，对于剩余的子节点，需要开展下一步的节点代价评估工作。关于评估所使用的代价函数设计，将在下一小节详细阐述，

中国知网 https://www.cnki.net

接下来将对算法中使用的碰撞检查方法进行详细说明。

时空混合 A*算法主要用来完成三维空间内的轨迹搜索任务。由于空间复杂度较高，在执行碰撞约束检查时，所采用的碰撞检测方法会对算法的运行耗时产生显著影响。此外，车辆和障碍物轮廓的描述方式也至关重要，它会直接影响到碰撞检测算法的耗时和准确程度。因此，为了提高算法效率和准确性，选择既简单又完善的轮廓描述方式显得尤为关键。

轨迹规划领域中常见的用于描述车辆轮廓的方法有：轴对齐包围盒(Axis-Aligned Bounding Box, AABB)、有向包围盒(Oriented Bounding Box, OBB)和外接圆包围矩形框(Circle Enclosing Rectangle, CER)，分别如图 3.3 所示。

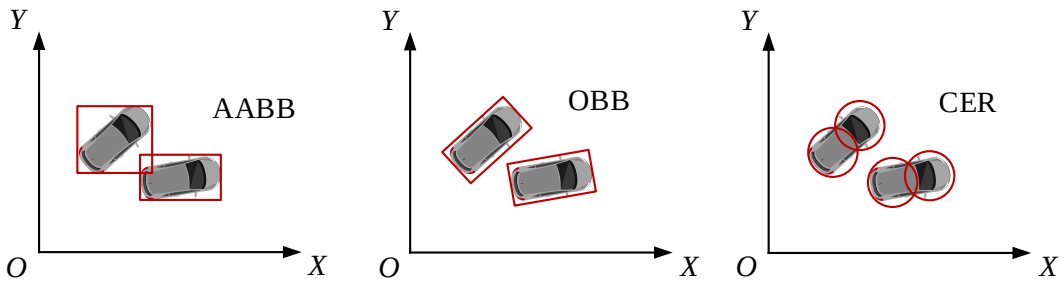


图 3.3 多种车辆模型轮廓构建方式

AABB 方法通过构建与坐标轴平行的边来形成一个包围车辆模型的矩形。随着车辆航向角的改变，此方法所构建的矩形的长度和宽度也会相应变化。该方法以($x_{\min}, x_{\max}, y_{\min}, y_{\max}$)的形式描述两个待检测物体的外接矩形，从而能够快速判断它们是否重叠。具体而言，如果这两个矩形在 X 轴或 Y 轴上不存在重叠区域，那么就可以判定这两个物体未发生碰撞。这种碰撞检测方式极为简洁，具有很高的计算效率。不过，AABB 方法在构建矩形时会额外占用一定的空间，且只能对物体的形状进行近似描述。尤其是当物体的长宽比过大时，这种构建方式会产生明显的失真，进而容易导致误判。所以，AABB 方法通常仅适用于进行粗略的碰撞检测。

OBB 方法能够根据车辆的航向角进行旋转，并且依据车辆的实际形状来构建矩形框。与 AABB 方法相比，OBB 方法对物体形状的描述更为精准，从而大大提高了几何碰撞检测的准确性。然而，如果 OBB 方法采用分离轴定理进行碰撞检测，会使得计算过程相对复杂，进而对算法的实时性造成一定的影响。因此，可以使用 GJK(Gilbert-Johnson-Keerthi)算法来加快碰撞检测的速度，但在障碍物数量较多的情况下，仍会产生较大的时间消耗。

中国知网 https://www.cnki.net

CER 方法的核心思路是根据物体的宽高比，使用一个或多个圆形来紧密包裹物体。在车辆碰撞检测场景中，只需计算代表车辆的圆心与其他边界之间的距离，判断其是否小于预先设定的安全距离即可。这种方法相对简单，但在某些情况下，由于圆形会额外占据一定空间，可能会引入误差。

综上所述，不同的碰撞检测方法各有优劣。本文将综合考虑上述方法的优缺点，以此来完成节点碰撞约束检查和道路边界约束检查，从而在保证检测准确性的同时，尽可能提高算法的效率和实时性。

在节点扩展过程中开展碰撞约束检查时，本文首先借鉴了 AABB 方法中的投影思想。当确定节点的时间状态后，便能在规划空间中明确道路上其他交通参与者的位置。接下来，将交通参与者的 s_{obs} 坐标与当前待检测节点的 s_i 状态进行对比。若满足$|s_{obs}-s_i|>s_{bound}$，则可判定该交通参与者与当前节点不存在碰撞风险，将其排除在进一步检测范围之外。而对于满足$|s_{obs}-s_i|\leqslant s_{bound}$的障碍物，由于在进行坐标系转换时物体形状可能会发生变形，因此需要特别关注。为确保碰撞检测的准确性，会把这些可能存在碰撞风险的障碍物记录下来，并在笛卡尔坐标系中针对自车和其他交通参与者的精细轮廓，运用 GJK 算法进行碰撞检测。经过前面的粗筛选步骤，需要进行精细碰撞检测的障碍物数量相对较少，所以不会对算法的实时性产生明显影响，能够在保证检测精度的同时，维持算法的高效运行。

在进行道路边界约束检查时，本文借鉴了 CER 模型的思想，假设自车及障碍物矩形框模型如图 3.4 所示。接下来，本文推导使用 N 个具有相同半径的圆覆盖车辆时的情形，如图 3.5 所示，当车辆的车身矩形框被 N 个半径相同的圆盘覆盖时，定义第 i 个圆的圆心坐标为(x_i, y_i)，那么这些圆心坐标可表示为：

$$\begin{cases} x_i = x + (\dfrac{2i-1}{2N}(L_R + L_W + L_F) - L_R)\cos\theta \\ y_i = y + (\dfrac{2i-1}{2N}(L_R + L_W + L_F) - L_R)\sin\theta \end{cases} \quad \cdots\cdots (3.4)$$

式中，$i = 1, 2, \cdots, N$；L_R 为车辆的后悬长度；L_W 为车辆的轴距；L_F 为车辆的前悬长度。每个圆的半径为：

$$R = \sqrt{\left(\frac{L_R + L_W + L_F}{2N}\right)^2 + \left(\frac{L_B}{2}\right)^2} \quad \cdots\cdots (3.5)$$

式中，L_B 为车辆的宽度。

中国知网 https://www.cnki.net

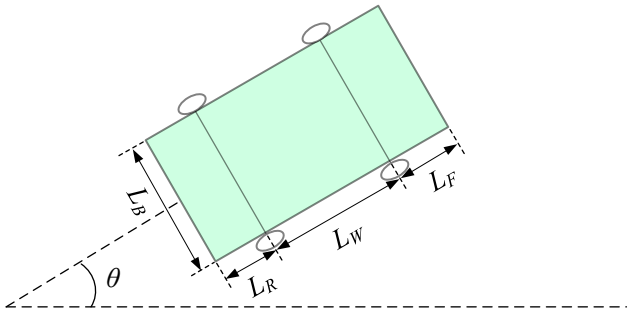


图 3.4 车辆矩形模型示意图

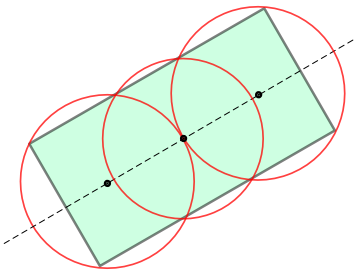


图 3.5 使用相同半径的圆盘来覆盖矩形车身的示意图(图中N=3)

基于上述推导，节点扩展中车辆与道路边界的碰撞检测问题可转化为：判断覆盖车辆矩形的所有圆盘的圆心到道路边界的距离是否大于安全距离。具体情形如图 3.6 所示，假设自车有 M 个包围圆，第 i 个包围圆的圆心坐标为(x_i, y_i)，转换到 Frenet 坐标系的坐标为(s_i, l_i)，半径为 r_i，道路宽度为 l_{road}，安全缓冲距离为 l_{buffer}。那么，自车与道路边界无碰撞需满足的条件为：对于任意的 $i \in M$，均有：

$$|l_i| < \frac{l_{road}}{2} - r_i - l_{buffer} \quad \cdots\cdots(3.6)$$

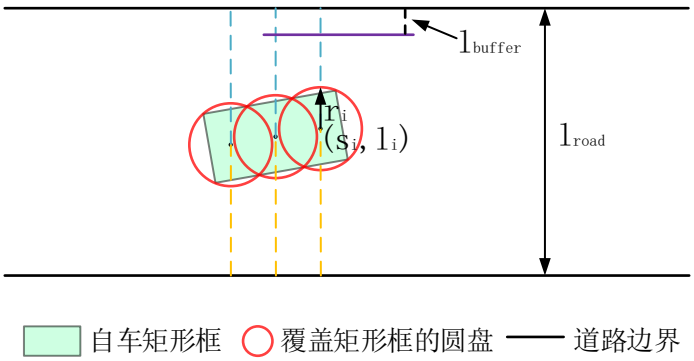


图 3.6 车辆与道路边界距离关系示意图

中国知网 https://www.cnki.net

3.2.2 时空节点启发式函数设计

在 A*系列算法中，节点的启发式函数与成本函数共同构成了该节点的代价函数。对于第 i 个节点，其代价函数可以表示为：

$$F_i = G_i + H_i \quad (3.7)$$

式中，F 代表节点的代价函数值，G 代表从起始节点到该节点的实际成本值，H 代表从该节点到目标节点的启发式估计成本值。

启发式函数在算法中扮演着至关重要的角色。其核心作用在于估算从当前节点抵达目标节点所需的最小代价，进而为搜索过程提供清晰的方向指引。借助启发式函数，算法能够更有针对性地开展搜索，尽可能的朝着目标点逼近，有效缩减搜索空间，显著提升算法的运行效率。在传统混合 A*算法中，通常会使用 A*搜索结果和 Reeds-Sheep 曲线来构建启发式函数，并且将二者中的最大值作为最终的启发信息。尽管这种组合方式在理论上能够为搜索过程提供较为可靠的引导，但在实际应用中却暴露出一个明显的弊端，即计算耗时过长。由于计算耗时的问题，传统混合 A*算法的应用场景受到了较大限制，主要适用于对实时性要求不高的低速非结构化场景。在这类场景中，由于环境相对简单、变化缓慢，算法有足够的时间完成复杂的计算。然而，复杂动态场景对算法的实时性提出了极高的要求，传统混合 A*算法因计算耗时过长，往往难以满足这样的实时性需求。

前文基于 Frenet 坐标系对城市道路场景进行三维规划空间构建，同时在规划空间内对车辆的运动进行了建模。鉴于搜索过程对实时性的严格要求，可以选取当前点到目标点的距离作为启发式函数的基础。在本文中，选用$\{s, l\}$坐标表示的欧式距离作为时空混合 A*算法搜索阶段的启发式函数。需要注意的是，采用$\{s, l\}$坐标，除了能够更好适应基于 Frenet 坐标系构建的规划空间外，还能避免在道路曲率较大的情况下，使用$\{x, y\}$坐标计算欧氏距离可能产生的误导搜索方向的不利影响。在计算时选择使用$\{s, l\}$坐标，可以避免道路形状因素的干扰，正确引导算法的搜索方向。

对于进入代价评估阶段的子节点 $N_i(s_i, l_i, \theta_i, v_i, t_i)$，其启发式函数的表达式为：

$$H_i = w_h \times \sqrt{(s_{end} - s_i)^2 + (l_{end} - l_i)^2} \quad (3.8)$$

式中，s_{end}、l_{end} 分别为目标节点的纵向位置和横向位置；w_h 为启发项中欧氏距离的权重。

中国知网 https://www.cnki.net

3.2.3 时空节点成本函数设计

在 A*系列算法中，成本函数同样占据着举足轻重的地位。成本函数的主要功能是衡量从起点到当前节点所产生的实际代价，在实际设计时，通常会将路径长度、转弯次数、能耗等诸多因素纳入其中，以此确保搜索结果既具备可行性又具备经济性。

一个节点的成本指的是从起始节点到该节点的累计成本。倘若在算法运行期间，每扩展一个节点都要从起始节点重新计算一次成本，这无疑会造成算力的极大浪费。因此，在算法设计时，一个节点的代价 G_{i} 可以由两部分组成，即其父节点的成本 G_{p} 与父节点扩展到子节点之间的成本ΔG_{pi} 之和，表达式如下所示：

$$G_i = G_p + \Delta G_{pi} \cdots\cdots (3.9)$$

为全面且精准地评估扩展节点的质量，本节将围绕通行效率、偏差程度、平滑性、舒适性等多个维度对成本函数进行设计。同时，会详细阐述从父节点扩展到子节点的成本的具体计算方式，具体公式如下所示：

$$\Delta G_{pi} = \Delta G_{pi}^{efficient} + \Delta G_{pi}^{offset} + \Delta G_{pi}^{smooth} + \Delta G_{pi}^{comfort} \cdots\cdots (3.10)$$

式中的各项分别代表通行效率成本、偏差程度成本、平滑性成本和舒适性成本。

在车辆行驶过程中，通常期望车辆能够保持较高的行驶速度。这是因为较高的车速有助于提升通行效率，能够显著减少行驶过程所耗费的时间。基于此，本文将通行效率的代价函数表示为：

$$\Delta G_{pi}^{efficient} = w_{efficient}(v_i - v_{\exp cted})^2 \cdots\cdots (3.11)$$

式中，$w_{efficient}$ 表示通行效率权重，$v_{\exp ected}$ 表示期望的车辆速度。

偏差程度主要体现在车辆的横向偏差上。在构建规划空间时，本文基于车道中心线和贝塞尔曲线生成了参考线。无论是车辆正常向前行驶，还是超车、变道等情况，我们都希望车辆能够沿着参考线行驶，这也符合人类驾驶员的驾驶习惯。同时，参考线上的点的 l 值始终为 0，所以偏差程度的代价函数可以表达为：

$$\Delta G_{pi}^{offset} = w_{offset}(l_i)^2 \cdots\cdots (3.12)$$

式中，w_{offset} 表示偏差程度权重。

对于父节点和子节点，它们之间的纵向偏差Δs 和横向偏差Δl 在某种程度上可以用来表示节点之间的平滑性。在父节点的状态量已经固定的情况下，由式(3.3)表示的节点扩展时遵循的运动学公式可知，子节点与父节点的纵向偏差Δs 和横向偏差Δl 主要与车

中国知网 https://www.cnki.net

辆航向角 θ_i 有关，而子节点的车辆航向角 θ_i 主要与行驶速度 v_i 和前轮转角 δ_i 有关，子节点的行驶速度 v_i 又与扩展节点时选取的加速度 a_i 有关。所以，对两个控制变量计算成本就相当于对纵向偏差Δs 和横向偏差Δl 计算成本。同时，更小的加速度 a_i 和前轮转角 δ_i 也意味着更小的能量消耗，这对车辆行驶的经济性也有帮助。因此，平滑性成本函数的计算公式可以表达为：

$$\Delta G_{pi}^{smooth} = w_{smooth}^{\delta}(\delta_i)^2 + w_{smooth}^{a}(a_i)^2 \quad \cdots\cdots(3.13)$$

式中，w_{smooth}^{δ} 和 w_{smooth}^{a} 分别为横向平滑性和纵向平滑性的权重。

在轨迹规划领域，对于车辆行驶舒适性的评价，一般使用加加速度 jerk 来表示。jerk 在数学上被定义为加速度对时间的导数，jerk 的绝对值越小，代表着加速度的变化越平缓，相应地也代表着乘坐越舒适。车辆纵向舒适性的计算相对简单，所以在这里直接根据定义给出车辆纵向加加速度 $jerk_{lon}$ 的近似计算公式：

$$jerk_i^{lon} = \frac{a_i - a_p}{dt} \quad \cdots\cdots(3.14)$$

接下来，本文先进行车辆横向加加速度的推导。在前文的坐标系转换公式中，针对 Frenet 坐标系，存在 l'' 可以表达横向加加速度，但在进行简化计算时，需要用到父节点的父节点的 l 状态。假设当前节点的父节点的父节点为 $N_g(s_g, l_g, \theta_g, v_g, t_g)$，则 l'' 的近似计算公式可以表达为：

$$l'' = \frac{l_i + l_g - 2l_p}{(dt)^2} \quad \cdots\cdots(3.15)$$

这种方法虽然原理简单，但也存在明显的缺点：一是这种计算方法在计算时涉及到的节点数量较多，数值精度相对来说会更差；二是在搜索刚开始时，若无法提供父节点的父节点信息，则无法计算 l'' 或导致计算结果失真。由于以上缺点，本文不选择使用这种计算方式，而将推导一种更优的计算方式。在单独考虑车辆的横向运动时，任意时刻车辆所受的横向力可以看作使车辆绕着当前前轮转角对应的转弯半径进行圆周运动，由车辆的运动学公式可知，当前前轮转角对应的转弯半径 R 的计算公式可以表达为：

$$\tan\delta = \frac{L}{R} \quad \cdots\cdots(3.16)$$

则当前前轮转角对应的曲率 κ 的计算公式可以表达为：

$$\kappa = \frac{1}{R} = \frac{\tan\delta}{L} \quad \cdots\cdots(3.17)$$

中国知网 https://www.cnki.net

因此，此时车辆横向加速度 a_{lat} 的计算公式可以表达为：

$$a_{lat} = \frac{v^2}{R} = v^2 \cdot \kappa = v^2 \frac{\tan\delta}{L} \quad \cdots\cdots (3.18)$$

定义前轮转角速度 γ 为前轮转角对时间的导数，则子节点的前轮转角速度 γ_i 的近似计算公式可以表达为：

$$\gamma_i = \frac{\delta_i - \delta_p}{dt} \quad \cdots\cdots (3.19)$$

车辆横向加速度对时间的导数即为车辆的横向加加速度，子节点对应的横向加加速度 $jerk_i^{lat}$ 计算公式可以表达为：

$$jerk_i^{lat} = \frac{2v_i \cdot a_i \cdot \tan\delta_i}{L} + \frac{v_i^2 \cdot \gamma_i}{L\cos^2\delta_i} \quad \cdots\cdots (3.20)$$

所以舒适性的成本函数表示为：

$$\Delta G_{pi}^{comfort} = w_{comfort}^{a}(a_i)^2 + w_{comfort}^{alat}(a_i^{lat})^2 + w_{comfort}^{jlon}(jerk_i^{lon})^2 + w_{comfort}^{jlat}(jerk_i^{lat})^2 \quad \cdots (3.21)$$

式中，$w_{comfort}^{a}$ 和 $w_{comfort}^{alat}$ 为纵向加速度和横向加速度的权重，$w_{comfort}^{jlon}$ 和 $w_{comfort}^{jlat}$ 为纵向加加速度和横向加加速度的权重。

综上所述，本文在设计节点成本函数的过程中进行了全面的综合考虑。由于前文已经完成了节点的碰撞检测，因此在成本函数中未对碰撞问题进行额外设计。关于这一部分，将在交通边界信息匹配之后，在后续的轨迹优化过程中进一步考虑。

3.2.4 时空混合 A*算法流程

经过节点扩展方案设计、节点约束检测、成本函数设计、启发式函数设计后，时空混合 A*算法能够在给定终止节点的情况下，总是选择代价值最小的节点优先进行搜索，最终在规划空间内搜索出行驶至终止节点的轨迹。

时空混合 A*算法在搜索过程中以自车当前状态构造搜索的起始节点 N_{start}，基于公式(3.3)进行时空节点扩展。在扩展过程中，按照章节 3.2.1 中的描述，对节点进行速度合理性检查、障碍物碰撞检查和道路边界约束检查，以筛选满足约束的节点。算法运行时会维护一个开列表(open list)和一个关列表(closed list)。筛选出的节点都将基于公式(3.7)进行代价评估，计算出具体的代价值，并放入开列表中。然后，选取代价值最小的节点作为当前节点，此节点被选中后应立即移入关列表，同时基于此节点对未曾扩展过的节点进行扩展。如此循环往复，直至搜索至目标节点 N_{end}，完成时空混合 A*搜索

中国知网 https://www.cnki.net

过程。最后，从 N_{end} 倒序输出轨迹。算法流程如表 3.1 所示：

表 3.1 时空混合 A*算法流程

时空混合 A*算法流程
输入：车辆配置参数、车辆当前状态、目标节点、三维规划空间
输出：连接车辆当前状态和目标节点的参考轨迹

```
初始化 open list 和 closed list
以车辆当前状态构造起始节点 N_start，将其加入 open list，并初始化节点代价
while (open list 不为空)
    从 open list 中选择综合代价 F 最小的节点作为当前节点 N_i
    将当前节点 N_i 添加至 closed list
    根据节点扩展方式扩展 N_i 的所有子节点 N_child
    for 遍历所有子节点 N_child
        if N_child 位于 closed list 中 or 未通过筛选
            continue
        if N_child is N_end
            结束所有循环
        if N_child 不在 open list 中
            计算 N_child 的综合代价 F，并将 N_child 添加到 open list 内
    end for
end while
初始化一个空节点容器 Ω 和一个临时节点 N_temp
将 N_end 赋值给 N_temp，并添加到容器 Ω 中
while (N_temp != N_start)
    将 N_temp 的父节点 N_parent 赋值给 N_temp，并将 N_temp 添加入容器 Ω
end while
reverse 节点容器 Ω，生成轨迹
return 搜索轨迹
```


中国知网 https://www.cnki.net

3.3 时空混合 ARA*算法设计

虽然时空混合 A*算法能搜索出从初始节点到终止节点的轨迹，但由于规划空间具有高维特性，在特殊场景下可能无法满足实时性要求。为了解决这一问题，需要对现有的时空参考轨迹生成算法做进一步的优化和改进。基于此，本文将前文的时空混合 A*算法改进为时空混合 ARA*算法，旨在提升算法在复杂场景下的实时性能和适应性。

时空混合 A*算法中的启发式函数用于估计当前节点到目标节点的代价，并引导算法的搜索方向。启发式函数需要满足一致性条件，即对任意节点 N_i 及其后继节点 N_{child} 而言：

$$H(N_i) \le G(N_i, N_{child}) + H(N_{child}) \quad \cdots\cdots (3.22)$$

式中，$G(N_i, N_{child})$ 表示从 N_i 到 N_{child} 的实际代价。

一致性启发式函数可以保证算法能够找到最优解，同时确保每个节点在扩展时只被扩展一次。加权 A*算法则通过在启发式函数 $H(N_i)$前乘以一个大于 1 的系数 ϵ 来加快搜索速度，减少节点扩展数量，但同时牺牲了算法的最优性，即搜索结果可能不是最优的。参数 ϵ 决定了最优性和搜索效率之间的权衡：当 ϵ=1 时，算法保证找到最优解；当 ϵ>1 时，算法找到次优解，解的质量由 ϵ 决定，其长度低于最优解的 ϵ 倍。

为了平衡最优性和搜索效率，可以使用 ARA*算法，通过分阶段调整启发式函数的膨胀系数 ϵ 来寻找最优路径。初始阶段，选择一个较大的 ϵ 值以快速获得一个初始解。随后，在迭代过程中逐渐减小 ϵ 的值，直至 ϵ=1，以期望得到更接近最优的解。在每次减小 ϵ 后，都基于已有的结果重新进行计算和改进，从而保证搜索效率和解的最优性。

当一个节点的成本函数值降低时，该节点及其后继节点之间的代价关系可能不再满足之前的等价关系。具体来说，如果节点 N_p 是节点 N_i 的最优前继节点，那么理论上应该满足以下关系：

$$G(N_i) = \min_{N'_p \in \mathrm{pred}(N_i)} (G(N'_p) + \Delta G(N'_p, N_i)) = G(N_p) + \Delta G(N_p, N_i) \quad \cdots\cdots (3.23)$$

其中，$G(N_i)$表示节点 N_i 的成本函数值，$G(N_p, N_i)$表示从节点 N_p 到节点 N_i 的成本，$\mathrm{pred}(N_i)$表示 N_i 的所有前继节点集合。当节点 N_p 的成本函数值降低时，如果 N_p 与其后继节点之间存在局部不一致，就会出现：

$$G(N_i) > \min_{N'_p \in \mathrm{pred}(N_i)} (G(N'_p) + \Delta G(N'_p, N_i)) \quad \cdots\cdots (3.24)$$

从局部不一致的定义可以看出，开列表可以被视作那些需要将局部不一致传递的节点的集合：当节点成本函数值降低时，则将其加入开列表；当节点被扩展时，则将其

中国知网 https://www.cnki.net

移出开列表，直到它的成本函数值再次下降。

当 ϵ 大于 1 时，启发式函数的一致性可能会被破坏，即无法确保每个节点在扩展时只被扩展一次，这会影响最终搜索结果的有效性。如果简单地加入每个节点仅扩展一次的限制，那么开列表将无法保证包含所有的非一致节点，此时开列表将不具备完备性，因为必须保留所有非一致节点的信息作为后续迭代搜索的基础。因此，本文在算法中加入了不一致列表(inconsistent list)，用于存储不在开列表中的非一致节点。通过将开列表和不一致列表结合，能够准确地存储所有局部不一致节点，并在每次新的搜索中将其作为非一致性传播的起点。

由于搜索过程中需要利用过去的搜索结果，而目标节点可能永远不会出现局部不一致，即可能不会被放入开列表中。这种情况下，时空混合 A*算法的终止条件将变得无效，导致搜索过程可能无法终止。为了解决这一问题，在时空混合 ARA*算法中可以将终止条件设置为：当终止节点的代价值小于等于开列表中最小代价值时停止。这样的终止条件确保目标节点不会被扩展，同时与目标节点代价值相同的其他节点也可能不会被扩展。综上所述，时空混合 ARA*算法的单次搜索流程如表 3.2 所示：

表 3.2 时空混合 ARA*算法单次搜索流程

时空混合 ARA*算法单次搜索流程
while $(G(N_{end})+\epsilon H(N_{end})) > \min_{N_i \in open}(G(N_i)+\epsilon H(N_i))$
从 open list 中选择并移除 F 最小的节点 N_i
将节点 N_i 移入 closed list
根据节点扩展方式获取节点 N_i 的所有子节点 N_{child}
for 遍历所有子节点 N_{child}
if N_{child} 没有被扩展过
$G(N_{child})$设置为∞
if $G(N_{child}) > G(N_i) + \Delta G(N_i, N_{child})$
$G(N_{child}) = G(N_i) + \Delta G(N_i, N_{child})$
if N_{child} 不在 closed list 中
将 N_{child} 加入到 open list 中
else
将 N_{child} 加入到 inconsistent list 中

中国知网 https://www.cnki.net

ARA*算法在开始初始化之后，会基于设定好的 ϵ 值进行一次搜索。之后在每次搜索之前，都会将不一致列表中的数据移入开列表中。由于开列表中的节点是按照之前的代价值排序的，所以在与不一致列表合并后，必须重新计算代价值并进行排序。这样，在每次调用单次搜索函数后，得到的次优解的代价都不会大于最优解的 ϵ 倍。在一次搜索结束后，ϵ 应当按照下式缩小至次优解的实际边界：

$$\epsilon' = \min(\epsilon, G(N_{end}) / \min_{N_i \in open \cup inconsistent\ list}(G(N_i) + H(N_i))) \cdots\cdots\cdots\cdots (3.25)$$

在减小 ϵ 后重新进行搜索的过程中，可以再减少一个$\Delta\epsilon$ 来提升求解质量。当算法运行时间达到时间阈值 t_{total} 或者 ϵ 减小到 1 时，算法将输出搜索结果。将表 3.2 表示的单次搜索流程记为 SingleSearch()，则时空混合 ARA*算法的具体流程如表 3.3 所示：

表 3.3 时空混合 ARA*算法流程

时空混合 ARA*算法流程
输入：车辆配置参数、车辆当前状态、目标节点、三维规划空间 输出：连接车辆当前状态和目标节点的时空参考轨迹

初始 open list、closed list、inconsistent list
以车辆当前状态构造起始节点 N_{start} 加入 open list，并初始化节点代价
SingleSearch()
按公式(3.25)更新 ϵ
存储当前次优解
while ($\epsilon > 1$)
  $\epsilon = \epsilon - \Delta\epsilon$
  将 inconsistent list 中的节点转移到 open list 中
  重新计算 open list 中所有节点的 F 值并按照 F 值重新排序，初始化 closed list
  SingleSearch()
  按公式(3.25)更新 ϵ
  if ($t \geqslant t_{\text{total}}$)
  break
  存储当前次优解
return 最后存储的解

中国知网 https://www.cnki.net

3.4 边界信息匹配

相较于 Frenet 坐标系，在笛卡尔坐标系所表征的求解空间内，基于时空参考轨迹进行轨迹优化，可以更为灵活地找到最优解。然而，此过程需考虑障碍物的非凸几何特性以及道路边界的碰撞约束等。这些约束条件的引入，极易使整个优化问题转变为非线性优化问题。这不仅会显著降低求解效率，还可能导致算法收敛于局部最优解，从而难以获取全局范围内的最优轨迹规划结果。

为解决上述问题，常见的解决方案包括边界框、可达集及时空走廊等方法。前文在时空混合 A*算法节点扩展过程中，对基于边界框的检测方法进行了详细阐述。该方法的优势在于能简便且精准地判断车辆是否与周边障碍物发生碰撞，从而确定轨迹的可行性；但其弊端在于无法清晰界定可通行区域，导致在评估安全风险时更为复杂。时空走廊方法是依据参考轨迹以及环境障碍物约束来生成驾驶走廊。此方法可将原本复杂且多源的约束信息转化为数值优化中的凸约束，但会导致解空间收缩。此外，在时空走廊的衔接部位，还可能出现路径不连续或转弯过度急促等问题。可达集是指自动驾驶车辆从一系列初始状态出发，随时间推移能够达到的状态集合。基于可达集的方法可以在不缩减求解空间的前提下，综合考虑车辆动力特性、视野可见性等因素，生成自动驾驶车辆的可通行区域，进而有效探索驾驶任务的可用状态空间及拓扑结构。综合以上分析，本文将在笛卡尔坐标系内融入边界框和可达集的核心思想，对道路边界与障碍物边界进行精确有效的约束构建，为后续轨迹优化奠定基础。

首先，针对道路边界进行分析。如图 3.7 所示，当车辆行驶至时空参考轨迹上的某一特定点时，采用三个圆形来完整包裹车辆的矩形框。为了清晰阐述如何基于时空参考轨迹点将车辆位置与道路边界进行精准匹配，本文以最末端的圆形为例展开说明。

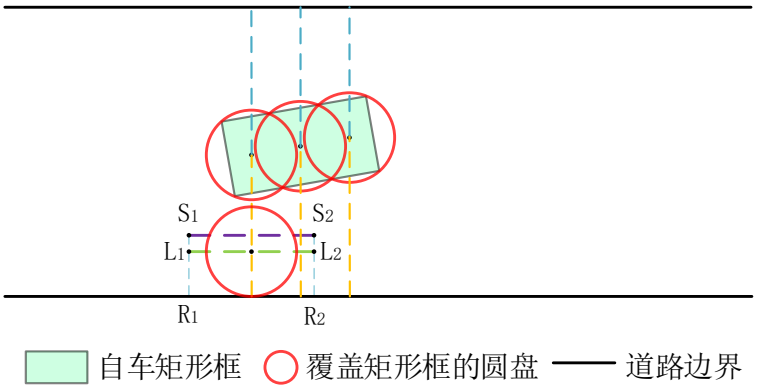


图 3.7 车辆与道路边界示意图

中国知网 https://www.cnki.net

首先，将该圆形的圆心向参考线进行投影，从而得到对应的投影点。基于这个投影点，能够准确找到与之相对应的道路边界点。以右侧道路边界为例，沿着边界分别向前和向后延伸一定距离，采集两个扩展边界点 R_1 和 R_2。以这两个扩展边界点 R_1 和 R_2 为基础，可以构建出一个法向量指向车道内部的半平面。然而，由点 R_1 和 R_2 所确定的半平面并不能直接作为道路边界的约束条件，还需对其进行膨胀处理。具体的操作方法为：将该半平面朝着法向量方向平移一段距离，这段距离与覆盖车身的圆的半径相同，平移后的半平面与图 3.7 中由点 L_1 和 L_2 确定的半平面位置一致。

假设由点 L_1 和 L_2 所确定的半平面对应的参数为$\{a, b, c\}$，圆心坐标为(x_0, y_0)。为确保距离表示的一致性和规范性，还进一步处理半平面参数使其满足特定的条件如下：

$$\sqrt{a^2+b^2+c^2}=1 \tag{3.26}$$

此时，分割半平面的直线的表达式如下所示：

$$ax+by+c=0 \tag{3.27}$$

将覆盖圆的圆心坐标带入直线表达式可得：

$$ax_0+by_0+c \tag{3.28}$$

式(3.28)计算结果的符号，能够直观反映出车辆与道路边界之间的位置关系。如前文所述，半平面的法向量均指向车道内部，所以当式(3.28)的计算结果大于 0 时，表明车辆正行驶在道路内侧的安全区域；当式(3.28)的计算结果小于 0 时，则意味着车辆已行驶至道路外侧，与道路边界发生了碰撞。

为了进一步提升车辆行驶的安全性，除了要确保车辆不与道路边界发生直接碰撞之外，还期望车辆在行驶过程中能与道路边界保持一定的安全距离。因此，可将原有的半平面沿法向量方向进一步平移，从而得到一个更为保守的半平面，以此来提供充足的安全缓冲空间。如图 3.7 所示，由 S_1 和 S_2 确定的半平面，相较于 L_1 和 L_2 确定的半平面，在位置上更加靠近车道内侧，对车辆与道路边界的位置关系提出了更高要求。同样地，可以使用式(3.28)来准确计算车辆与这个保守半平面之间的位置关系，进而保障行车安全。

接下来，将对障碍物边界进行分析。在确定时间 t 后，规划空间内的每一个交通参与者都具有固定的空间位置。与车辆和道路边界的相对位置关系有所不同，尽管障碍物自身的位置是固定的，但其相对于自车的方向和位置却存在不确定性。

为了准确判断障碍物相对于自车的方向和位置，首先构建一个由车辆后轴中心和

中国知网 https://www.cnki.net

航向角确定的单位向量，再构造一个由车辆后轴中心与障碍物位置确定的向量。通过对这两个向量进行内积和外积运算，便能够精准地确定障碍物相对于自车的具体方向和位置信息。

如图 3.8 所示，在确定障碍物相对于自车的方向和位置之后，进一步找到障碍物靠近自车的那一条边。将这条边朝着自车一侧偏移一定距离，同时沿着障碍物的运动方向前后扩展一定距离，从而得到两个端点。随后，把障碍物的位置向道路边界进行投影，并在投影点的基础上前后扩展，进而得到道路边界上的两个点。通过组合这些点，能够构建出一个用于描述障碍物空间位置的梯形。在车辆行驶过程中，需要避免车辆的行驶轨迹与这个梯形空间产生交集，以此来有效避免车辆与障碍物发生碰撞。

在构建障碍物边界约束时，其思路与构建道路边界的方式大致相似。同样采用三个圆形来完整覆盖车辆的矩形框，并为每个圆心匹配法向量指向车道内部的半平面。通过判断圆心与这些半平面之间的位置关系，来确定车辆与障碍物之间的相对位置。在图 3.8 中，所选取的车辆位置极具代表性，将覆盖车辆形状的三个圆的圆心分别标记为 C_1、C_2 和 C_3。当圆心 C_1 向道路边界投影时，会与梯形空间的上底相交。此时，由上底两个端点构成的半平面沿法向量方向平移一个半径距离后生成的半平面 L_1，即为 C_1 所对应的障碍物边界。圆心 C_2 向道路边界投影时，会与梯形空间的腰相交。倘若直接使用由梯形空间的腰构成的半平面作为障碍物边界约束，将会极大地压缩求解空间。因此，在这种情况下，直接采用与上底重合的半平面 L_2 作为对应的障碍物边界。而圆心 C_3 向道路边界投影时，不会与梯形空间相交，此时的障碍物边界与前文设计的道路边界一致，由半平面 L_3 表示。

在得到三个圆心所对应的障碍物边界之后，同样出于对行车安全性的考虑，本文希望车辆在行驶过程中能够与障碍物保持一定的安全距离。为此，可以将这些半平面沿法向量方向平移一段距离，从而得到更为保守的半平面。如图 3.8 所示，S_1、S_2 和 S_3 便是平移后得到的半平面。对于所有表示障碍物边界的半平面，同样可以运用式(3.28)精确地计算出圆心与各个半平面之间的位置关系，从而为车辆的安全行驶提供可靠的判断依据。

中国知网 https://www.cnki.net

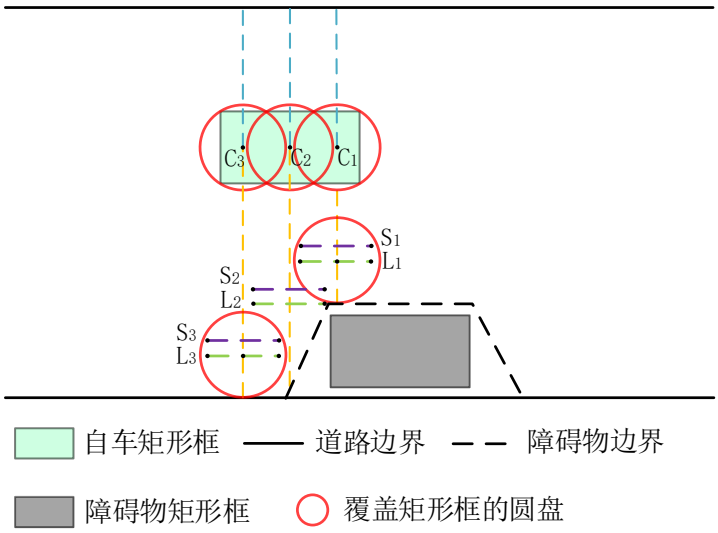


图 3.8 车辆与障碍物边界示意图

3.5 本章小结

本章基于上一章生成的参考线和规划空间，对时空参考轨迹生成算法进行研究。首先，针对传统混合 A*算法的局限性进行分析，并引入时间维度，对节点的扩展方式和代价函数进行了改进，将传统混合 A*算法改进为时空混合 A*算法。该算法能够从自车当前状态出发，搜索得到一条连接目标节点的可行驶轨迹。为了避免高维空间带来的耗时严重问题，保证算法的实时性，本文又将时空混合 A*算法进一步改进为时空混合 ARA*算法，使得算法能够在规定的时间范围内生成可行的参考轨迹，从而在效率和性能之间取得平衡。最后，本文融合了可达集和边界框的思想，提出了一种根据时空参考轨迹构建道路边界约束和障碍物边界约束的方法，为后续章节中时空联合轨迹优化中安全性约束的构建奠定了基础。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第4章 时空联合轨迹优化算法研究

4.1 引言

在上一章中，本文对时空参考轨迹的生成算法进行了研究，但由于对车辆运动学特性考虑较少，没有对车辆状态进行限制，也没有进一步考虑平滑性、安全性等因素，因此参考轨迹不能直接作为轨迹规划的结果输出使用。本章将在前文的基础上，在笛卡尔坐标系中构造时空联合轨迹优化问题，并优化算法流程进行求解。首先，在 ILQR 算法框架下构建轨迹优化问题，介绍 ILQR 算法的求解流程，并在求解过程中引入正则化、线搜索、增广拉格朗日方法，将其优化为具有更好求解效果的 CILQR 算法。之后，进一步完善本文的轨迹优化问题，包括推导高阶车辆状态方程、确定初始轨迹的生成方式、设计目标函数等，以确保在保证求解实时性的前提下，能够提升轨迹优化结果的安全性与舒适性。

4.2 基于 CILQR 算法的优化问题求解

4.2.1 ILQR 问题定义

相较于传统的时空解耦式轨迹规划算法，时空联合轨迹规划算法能够更高效地处理复杂环境中的障碍物。它通过同时规划路径和速度，生成了更具灵活性的轨迹。这种一体化的优化方式不仅能够提升规划算法对不同道路场景的适应能力，还能增强规划结果的鲁棒性和实用性。

从前文对轨迹规划领域中常用的两种坐标系的分析来看，虽然 Frenet 坐标系在某些场景下具有独特优势，但也存在一些显著的局限性，如投影失真以及坐标映射不唯一等问题。更为关键的是，车辆在真实世界中的行驶轨迹本质上是基于笛卡尔坐标系来描述的。因此，在前文已经生成的时空参考轨迹的基础上，本章选择基于笛卡尔坐标系来构建时空联合轨迹优化问题。

时空联合轨迹优化问题本质上是一个带约束的最优控制问题，其目标是求解一条连续最优，且包含位置信息和速度信息的轨迹。然而，由于车辆运动学模型的非线性特性以及障碍物分布的非凸性质，求解过程中需要同时处理非线性等式约束和非凸不等

中国知网 https://www.cnki.net

式约束。这些复杂的约束条件显著增加了问题求解的难度，导致现有的求解器存在计算复杂度高和求解效率低的问题。这直接导致求解耗时大幅增加，而在对实时性要求极高的轨迹规划场景中，这种延迟是难以容忍的。

ILQR 算法是一种基于线性二次调节器的优化算法。它通过在每一步迭代中应用动态规划的思想，将复杂的非线性最优控制问题分解为一系列线性二次型调节器问题，从而高效地求解最优控制序列。ILQR 算法凭借其简洁的数学形式，展现出更高的计算效率和更小的问题规模。这一特性使其在处理时空联合轨迹规划问题时，能够显著降低计算耗时，从而满足对实时性的要求。

本文中的时空联合轨迹优化问题可以表述为：

$$\arg\min_{U} \; J = \sum_{k=0}^{N-1} \ell_k(x_k, u_k) + \ell_N(x_N) \quad \cdots\cdots (4.1)$$

$$s.t. \quad x_0 = x_{start} \quad \cdots\cdots (4.2)$$

$$x_{k+1} = f(x_k, u_k) \quad \mathrm{k} = 0,1,\cdots,N-1 \quad \cdots\cdots (4.3)$$

$$g_i(x_k) \le 0 \quad \mathrm{k} = 0,1,\cdots,N \quad i = 0,1,\cdots,m \quad \cdots\cdots (4.4)$$

$$X = \{x_0, x_1, \cdots, x_N\} \quad \cdots\cdots (4.5)$$

$$U = \{u_0, u_1, \cdots, u_{N-1}\} \quad \cdots\cdots (4.6)$$

其中，式(4.1)表示需要找到一组控制序列，使得目标函数的值最小。目标函数由两部分组成：前一部分 $\ell_k(x_k, u_k)$ 表示除终点车辆状态外，车辆状态量和控制量产生的代价成本；后一部分 $\ell_N(x_N)$ 表示轨迹终点的车辆状态量产生的代价成本。目标函数不要求一定为二次型，但需要保证二次可微。$x_k = [x_k^c, y_k^c, \theta_k, \delta_k, \gamma_k, v_k, a_k, jerk_k]^T$ 表示第 k 个轨迹点的车辆状态量，分别代表着采样时间 t_k 时车辆后轴中心在笛卡尔坐标系下的横纵坐标、航向角、前轮转角、前轮转角速度、速度、加速度和加加速度，$u_k = [snap_k, djerk_k]^T$ 表示第 k 个轨迹点的车辆控制量，分别代表着采样时间 t_k 时车辆前轮转角速度变化率和加加速度变化率。式(4.2)到(4.4)表示优化问题中的约束条件，其中，式(4.2)是轨迹初始状态量与车辆当前状态量相等的等式约束；式(4.3)是由车辆状态空间方程构成的非线性等式约束，式(4.4)是对状态量的不等式约束。式(4.5)表示优化问题中的状态变量，式(4.6)表示优化问题中的控制变量，它们共同构成了优化问题的优化向量。

中国知网 https://www.cnki.net

4.2.2 ILQR 算法求解流程

ILQR 算法的本质思想是运用动态规划原理来求解由式(4.1)所表达的优化问题。对于给定的控制序列，定义第 i 个轨迹点状态的值函数为：

$$V(x_i)=\min_{u_i}[\ell_i(x_i,u_i)+V(x_{i+1})] \quad \cdots\cdots(4.7)$$

式中，$\ell_i(x_i,u_i)$ 为轨迹上第 i 个状态控制对产生的代价（中间项），$V(x_{i+1})$ 为第 i+1 个状态的值函数。

在有限状态转移的离散系统中，根据动态规划思想中的贝尔曼最优原理，可以将求解整个最优控制序列的问题转化为单步上控制变量 u_i 的最优化问题，即只需要在每一步找到满足式(4.7)的控制量，就可以组合得到最优的控制序列。

定义式(4.7)中的最小化运算符内的函数为：

$$\begin{aligned}P(x_i,u_i)&=\ell_i(x_i,u_i)+V(x_{i+1})\\&=\ell_i(x_i,u_i)+V(f(x_i,u_i))\end{aligned} \quad \cdots\cdots(4.8)$$

在运行 ILQR 算法前需要给定一组初始解，ILQR 算法将在这组初始解的基础上进行迭代优化。假设在第 i 步时系统对应的状态量和控制量分别为 x_i 和 u_i，并分别引入微小的变动 δx 和 δu。值得说明的是，δx 的产生是由上一步的控制量变化引起的，与当前控制量并无直接关系。在式(4.9)中定义函数 P 的变化量为 $Q(\delta x, \delta u)$，则有：

$$\begin{aligned}Q(\delta x,\delta u)&=P(x_i+\delta x,u_i+\delta u)-P(x_i,u_i)\\&=\ell_i(x_i+\delta x,u_i+\delta u)-\ell_i(x_i,u_i)\\&\quad+V(f(x_i+\delta x,u_i+\delta u))-V(f(x_i,u_i))\end{aligned} \quad \cdots\cdots(4.9)$$

将 $Q(\delta x, \delta u)$二阶泰勒展开可得：

$$Q(\delta x,\delta u)\approx\frac{1}{2}\begin{bmatrix}1\\ \delta x\\ \delta u\end{bmatrix}^T\begin{bmatrix}0 & Q_x^T & Q_u^T\\ Q_x & Q_{xx} & Q_{xu}\\ Q_u & Q_{ux} & Q_{uu}\end{bmatrix}\begin{bmatrix}1\\ \delta x\\ \delta u\end{bmatrix} \quad \cdots\cdots(4.10)$$

式(4.10)中的各项系数分别为：

$$\begin{aligned}Q_x&=\ell_x+f_x^\top V_x'\\Q_u&=\ell_u+f_u^\top V_x'\\Q_{xx}&=\ell_{xx}+f_x^\top V_{xx}'f_x+V_x'\cdot f_{xx}\\Q_{uu}&=\ell_{uu}+f_u^\top V_{xx}'f_u+V_x'\cdot f_{uu}\\Q_{ux}&=\ell_{ux}+f_u^\top V_{xx}'f_x+V_x'\cdot f_{ux}\end{aligned} \quad \cdots\cdots(4.11)$$

式中，ℓ 代表目标函数；f 代表车辆的系统动力学方程；V' 代表第 i+1 步的价值函数；

中国知网 https://www.cnki.net

角标代表对相应的状态变量 x 或控制变量 u 求导；角标数量为一时，代表求取相应的雅可比矩阵；角标数量为二时，代表求取相应的海森矩阵。

其中二阶项 Q_{xx}, Q_{ux} 和 Q_{uu} 中包含有车辆的系统动力学模型的二阶微分项 f_{xx}，f_{ux} 和 f_{uu}。对于 ILQR 算法，假设车辆模型的二阶微分项为零，以简化计算。通过最小化二次展开函数 $Q(\delta x, \delta u)$来求解局部的最优控制增量 δu^*可得：

$$\delta u^* = \arg\min_{\delta u} Q(\delta x, \delta u) = -Q_{uu}^{-1}(Q_u + Q_{ux}\delta x) \quad \cdots\cdots(4.12)$$

令：

$$\begin{aligned} k &= -Q_{uu}^{-1} \cdot Q_u \\ K &= -Q_{uu}^{-1} \cdot Q_{ux} \end{aligned} \quad \cdots\cdots(4.13)$$

式(4.12)可写成：

$$\delta u^* = k + K\delta x \quad \cdots\cdots(4.14)$$

将式(4.14)带入式(4.10)的二次展开方程中，并且计算价值函数关于状态量的雅可比矩阵和海森矩阵，为下一次最优控制增益的求解做准备可得：

$$\begin{aligned} \Delta V &= -\frac{1}{2}k^T Q_{uu} k \\ V_x &= Q_x - K^T Q_{uu} k \\ V_{xx} &= Q_{xx} - K^T Q_{uu} K \end{aligned} \quad \cdots\cdots(4.15)$$

在上述计算过程中，可以注意到式(4.12)中需要对矩阵 Q_{uu} 求逆，当 Q_{uu} 非正定时，其逆矩阵 Q_{uu}^{-1} 的计算会变得不稳定。因此在算法迭代过程中，首先要对 Q_{uu} 进行正定性检查，如果 Q_{uu} 是非正定的，那么还需要对其进行正则化处理。

对于矩阵正定性的检查，一般有特征值分解法、LU 分解法和 Cholesky 分解法。特征值分解法的计算过程通常比较复杂，时间复杂度较高；LU 分解法是判断矩阵是否正定的必要条件，但对于一些特殊情况可能会出现数值不稳定的现象，导致判断结果不准确。

Cholesky 分解法是一种适用于正定对称矩阵的特殊分解方法。对于一个对称矩阵 A，其正定的充要条件是：可以将对称矩阵 A 分解为 $A=LL^T$ 的形式，并且分解过程中不会出现数值问题，其中 L 是一个下三角矩阵。Cholesky 分解法的优点是能够高效且准确地进行正定性判断，计算复杂度相对较低。尽管该方法的适用范围存在一定局限性，但在 ILQR 算法中，Q_{uu} 为对称矩阵，恰好满足 Cholesky 分解法的应用条件，因此，本文将使用 Cholesky 分解法来判断 Q_{uu} 的正定性。

中国知网 https://www.cnki.net

如果判断 Q_{uu} 是非正定的，为了避免在迭代求解过程中出现数值不稳定的情况，需要对其进行正则化处理。本文中使用的正则化方法为：

$$\tilde{Q}_{uu} = Q_{uu} + \rho I \quad (4.16)$$

式中，ρ 为正则化系数，要求大于 0；I 表示与 Q_{uu} 维度相同的单位矩阵；$\tilde{Q}_{uu}$ 表示正则化后的矩阵。

这种正则化方法会在局部控制代价矩阵 Q_{uu} 中增加一个对角矩阵项，这相当于在当前控制序列周围增加了一个二次成本，使得步骤更加保守。Q_{uu} 更改后，式(4.13)所表示的控制增益也会随之更改为：

$$\begin{aligned} k &= -\tilde{Q}_{uu}^{-1} \cdot Q_u \\ K &= -\tilde{Q}_{uu}^{-1} \cdot Q_{ux} \end{aligned} \quad (4.17)$$

在 ILQR 算法的后向传递过程中引入正则化策略时，正则化参数 ρ 的设定需要满足三个相互矛盾的要求：（1）在算法迭代计算的初始阶段，初始轨迹与局部最小值的偏离程度相对较大。此时，如果后向传递过程失败（即 $\tilde{Q}_{uu}$ 矩阵非正定），则期望正则化参数 ρ 能够迅速增大。这是因为在初始的不稳定阶段，为确保轨迹计算的收敛性，所需的最小 ρ 值通常较大。只有通过增大 ρ 值，才能引导算法朝着收敛的方向发展。（2）在算法迭代的中间阶段，正则化参数 ρ 需要维持一个大于 0 但不趋近于 0 的状态。在此阶段，期望能够精确调整 ρ，使其尽可能接近一个最小的正值。这一调整旨在保证算法稳定性的同时，尽可能减少正则化对算法性能的过度影响，从而平衡收敛性与计算精度之间的关系。（3）在算法迭代接近完成时，轨迹已趋近于局部最小值。此时，期望正则化参数 ρ 能够迅速趋近于零。由于在该阶段目标函数具有良好的二次逼近性质，通过让 ρ 趋近于零，可以充分利用这种二次逼近特性，从而实现算法的快速收敛并提高计算效率。

综上所述，根据不同的迭代阶段快速准确地调整正则化参数 ρ 的值至关重要。本文使用两步修改方法，定义一个较小的正值 $\rho_{\min}$ 和一个修正因子 Δ_0，按如下准则对 ρ 的值进行调整：

如果当前这次迭代的后向传递过程中出现非正定的 $\tilde{Q}_{uu}$，则按如下规则增大 ρ 值：

$$\begin{cases} \Delta = \max(\Delta_0, \Delta \cdot \Delta_0) \\ \rho = \max(\rho_{\min}, \rho \cdot \Delta) \end{cases} \quad (4.18)$$

如果当前这次迭代的后向传递过程中的 $\tilde{Q}_{uu}$ 都是正定的，则按如下规则减小 ρ 值：

中国知网 https://www.cnki.net

$$\Delta = \min\left(\frac{1}{\Delta_0}, \frac{\Delta}{\Delta_0}\right)$$
$$\rho = \begin{cases} \rho \cdot \Delta, & \rho \cdot \Delta > \rho_{\min} \\ 0, & \rho \cdot \Delta < \rho_{\min} \end{cases} \quad \cdots\cdots(4.19)$$

完成 ILQR 算法的后向传递过程后，可以得到一系列的控制增益。利用这些控制增益对原有的控制序列进行更新后，就可以进行前向传递过程。将更新后的控制序列输入到车辆模型中，并结合车辆的初始状态 x_0 进行前向积分，可以得到新的状态序列。前向传递过程可以用如下公式表示：

$$\begin{aligned} \hat{x}_0 &= x_0 \\ \hat{u}_i &= u_i + k_i + K_i(\hat{x}_i - x_i) \\ \hat{x}_{i+1} &= f(\hat{x}_i, \hat{u}_i) \end{aligned} \quad \cdots\cdots(4.20)$$

式中，$i = 0, 1, 2, \cdots, N\text{-}1$；$x_0$ 表示车辆的初始状态；x_i 和 u_i 分别表示第 i 步的车辆状态量和控制量；$\hat{x}$ 和 $\hat{u}$ 分别表示更新后的车辆状态量和控制量。

式(4.20)所表示的前向传递过程是 ILQR 算法快速收敛的关键步骤。对于常见的非线性系统而言，当新轨迹大幅偏离线性化模型的有效区域时，目标函数值可能不会下降，甚至还可能出现发散的情况。为了解决这个问题，本文在前向传递过程中引入了一个线搜索参数 α，用以控制生成新轨迹的步幅：

$$\hat{u}_i = u_i + \alpha k_i + \mathrm{K}_i(\hat{x}_i - x_i) \quad \cdots\cdots(4.21)$$

式中，α 的取值范围为[0, 1]。

当 $\alpha = 0$ 时，轨迹将不会得到更新。当 $\alpha \neq 0$ 时，由于反馈项的存在，最终的控制步长并非是全步长的简单缩放版本。可以利用式(4.15)中ΔV 的累加来预估新轨迹总代价的下降值（在加入了线搜索过程的情况下）：

$$\Delta J(\alpha) = \alpha \sum_{i=0}^{N-1} k(i)^T Q_u(i) + \frac{\alpha^2}{2} \sum_{i=0}^{N-1} k(i)^T Q_{uu}(i) k(i) \quad \cdots\cdots(4.22)$$

计算实际的轨迹代价下降与预估的轨迹代价下降之比为：

$$z = \frac{\left[J(U) - J(\hat{U})\right]}{\Delta J(\alpha)} \quad \cdots\cdots(4.23)$$

式中，U 表示原有的控制序列，$\hat{U}$ 表示更新后的控制序列。

仅当实际的代价下降与预估的代价下降比较接近时，才接受这次迭代，即需满足如下条件：

$$0 < c_1 < z \quad \cdots\cdots(4.24)$$

中国知网 https://www.cnki.net

式中，c_1 是一个较小的常数。当实际的代价下降与预估的代价下降比较接近时，表明目标函数的二次展开处于模型的有效区域，具有良好的近似效果。

在前向传递过程中加入线搜索后，后向传递与前向传递共同构成了一次 ILQR 算法的迭代过程。在每一次迭代结束后，应当对算法是否满足收敛条件进行判断，如果不满足收敛条件，则继续迭代。

本文基于两次迭代轨迹的目标函数值的变化来制定算法的收敛条件：

$$\Delta J = \left| \frac{J_{new} - J_{old}}{J_{old}} \right| \cdots\cdots (4.25)$$

式中，J_{new} 为轨迹迭代后的目标函数值，J_{old} 为轨迹迭代前的目标函数值。若$\Delta J < \varepsilon$ 或迭代次数达到上限，则算法终止，否则算法继续进行优化计算。其中，ε 为收敛阈值，满足 $0 < \varepsilon << 1$。

4.2.3 CILQR 算法问题定义与求解

从上文 ILQR 算法的求解过程可以看出，该算法的局限性在于仅适用于带有系统状态方程约束的非线性优化问题，难以严格遵循不等式约束。而 CILQR 算法在 ILQR 算法的基础上进行了改进，通过将不等式约束转化为目标函数，并对非线性约束进行线性化处理，有效克服了 ILQR 算法的这一局限，从而能够更高效地解决自动驾驶轨迹优化问题。因此，本文将基于 CILQR 算法构建并求解时空联合轨迹优化问题，以充分发挥其优势。

为有效处理不等式约束，本文将不等式约束引入目标函数，将 ILQR 算法扩展为 CILQR 算法，使其具备严格遵循硬约束的能力，从而提升轨迹优化的可行性和精确性。将不等式约束引入目标函数的方法包括二次规划求解法、屏障函数法、增广拉格朗日法等。二次规划求解法通过将 ILQR 算法的每次迭代转化为一个带约束的二次规划问题，并借助二次规划求解器来求解线性或二次型约束问题。虽然这种方法能够严格满足不等式硬约束的要求，但同时也牺牲了 ILQR 算法原有的部分优良特性。屏障函数法通过将不等式约束转化为指数型或对数型屏障函数的形式，并将其嵌入目标函数中。当约束被轻微违反时，屏障函数增长缓慢，惩罚力度较轻；而当约束被严重违反时，屏障函数迅速增长，惩罚力度显著增强，从而促使优化结果快速满足不等式约束。以指数型屏障函数为例，其数学形式为：

$$B(x) = q_1 \exp(q_2 g(x)) \cdots\cdots (4.26)$$

中国知网 https://www.cnki.net

式中，q_1 和 q_2 为指数型屏障函数的调节参数，具体函数效果如图 4.1 所示：

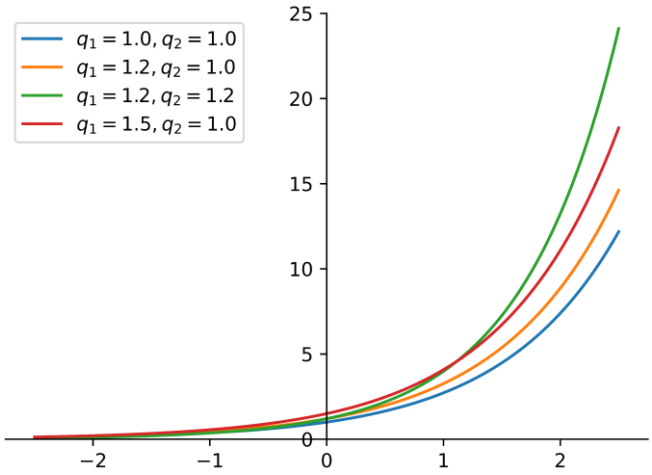


图 4.1 指数型屏障函数取不同调节参数时的图像

由指数函数的性质可知，当优化变量超过约束边界时，屏障函数的值会迅速增大。这种特性虽然有助于严格约束优化变量，但也可能导致求解过程中出现数值不稳定的情况。此外，从图像中可以看出，屏障函数中调节参数的选择对其惩罚效果有着显著影响。如何合理整定这些参数，使其在多种场景下均能取得良好的优化效果，也是该方法面临的问题。

增广拉格朗日方法通过引入拉格朗日乘子和惩罚项，将约束条件嵌入到代价函数中。在优化过程中，该方法通过迭代更新拉格朗日乘子，逐步逼近满足约束条件的状态。这不仅确保了原目标函数的优化，同时严格满足了约束条件，从而实现了约束优化问题的高效求解。增广拉格朗日方法的数学表达公式为：

$$\mathcal{L}(x) = \ell(x) - \lambda^T c(x) + \mu \frac{1}{2} c(x)^T c(x) \quad \cdots\cdots (4.27)$$

式中，$\mathcal{L}(\mathrm{x})$为增广拉格朗日函数，$\ell(\mathrm{x})$为目标函数或代价函数，λ 是拉格朗日乘子，μ 是惩罚系数，$c(x)$是等式约束或不等式约束。

增广拉格朗日方法将原问题转化为无约束最小化问题后，可以在保持拉格朗日乘子 λ 和惩罚系数 μ 值不变的情况下，对转化后的问题进行求解，求解后再更新 λ 和 μ 的值。一般采取的更新方法是根据下式增加惩罚系数 μ 的值，并更新拉格朗日乘子 λ 的值：

$$\begin{cases} \mu = m \cdot \mu \\ \lambda = \lambda - \mu c(x) \end{cases} \quad \cdots\cdots (4.28)$$

中国知网 https://www.cnki.net

式中，m 表示 μ 的增长系数。

式(4.27)相当于在对偶问题上执行梯度上升步骤。增广拉格朗日方法在理论上能够达到超线性的收敛速度，但在实践中，当参数 μ 增加时，增广拉格朗日方法往往在接近最优解时收敛速度会变慢。为了改善上述问题，本文使用增广拉格朗日方法来执行一般的圆锥约束。式(4.27)可以改写为：

$$\mathcal{L}(x)=\ell(x)+\frac{1}{2\mu}\left(\|\lambda-\mu c(x)\|^2-\|\lambda\|^2\right) \cdots\cdots (4.29)$$

将式(4.29)中的第一个二次惩罚项 $\lambda-\mu c(x)$ 与式(4.27)中标准的对偶上升步长进行比较，可以看到式(4.29)中的表述有效地惩罚了当前和更新后的拉格朗日乘子估计之间的差异。

基于以上对不等式约束解决方案的分析，本文选用增广拉格朗日方法来处理优化问题中的不等式约束。原有时空联合轨迹优化问题在固定拉格朗日乘子和惩罚系数后可以重新表述为标准的 ILQR 问题形式：

$$\arg\min_{U} J|_{\lambda,\mu}=\sum_{k=0}^{N-1}\mathcal{L}_k(x_k,u_k)|_{\lambda,\mu}+\mathcal{L}_N(x_N)|_{\lambda,\mu} \cdots\cdots (4.30)$$

$$s.t.\quad x_0=x_{start} \cdots\cdots (4.31)$$

$$x_{k+1}=f(x_k,u_k)\quad \mathrm{k}=0,1,\cdots,N-1 \cdots\cdots (4.32)$$

$$X=\{x_0,x_1,\cdots,x_N\} \cdots\cdots (4.33)$$

$$U=\{u_0,u_1,\cdots,u_{N-1}\} \cdots\cdots (4.34)$$

在保持 λ 和 μ 不变的情况下，按照上一节描述的 ILQR 求解方法进行求解后，遵循式(4.28)更新 λ 和 μ，当由不等式约束产生的代价值小于阈值（本文设为 0.05）时，结束迭代。

本文基于 CILQR 算法实时帧求解最优轨迹的整体流程如图 4.2 所示。

中国知网 https://www.cnki.net

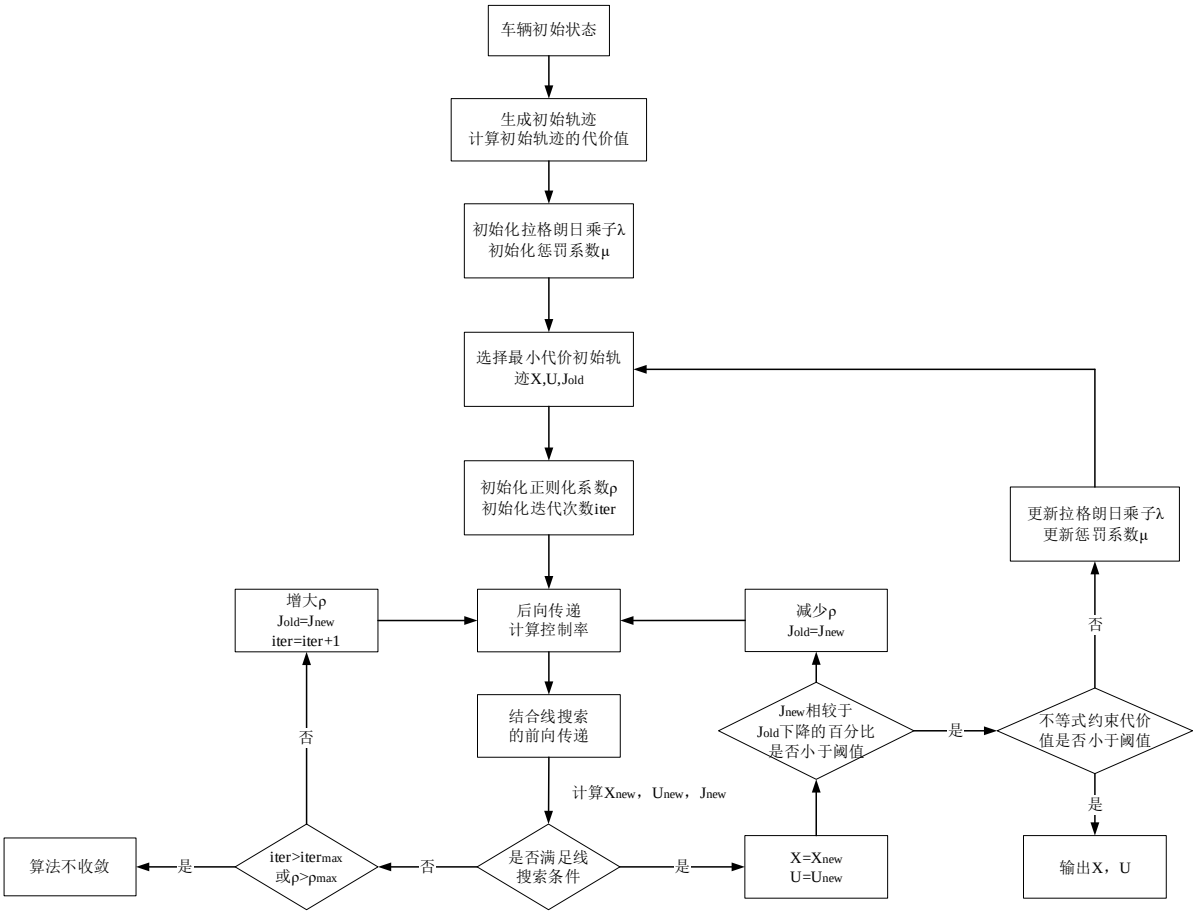


图 4.2 CILQR 算法流程图

4.3 时空联合轨迹规划优化问题构建

4.3.1 高阶车辆状态方程

车辆运动学模型在轨迹规划中扮演着至关重要的角色。在上一章的时空混合 ARA* 算法节点扩展过程中，在 Frenet 坐标系中结合车辆运动学模型来进行时空节点扩展，确保扩展生成的节点之间严格满足车辆的运动学约束，从而保证了搜索生成的轨迹具备可行性。

在 CILQR 算法中，基于车辆的初始状态和控制序列，通过前向模拟可以生成所有轨迹点的状态信息。在此基础上，采用更加精确的车辆运动学模型，能够显著提升状态信息的准确性和轨迹的平顺性。因此，本文在传统车辆运动学模型的基础上引入更高阶的状态变量和控制变量，以达到更平滑准确的效果。高阶车辆运动学模型的微分方程如下所示：

中国知网 https://www.cnki.net

$$\begin{cases} \dot{x} = v\cos\theta \\ \dot{y} = v\sin\theta \\ \dot{\theta} = \dfrac{v\tan\delta}{L} \\ \dot{\delta} = \gamma \\ \dot{v} = a \\ \dot{a} = jerk \\ \dot{jerk} = djerk \\ \dot{\gamma} = snap \end{cases} \quad \cdots\cdots (4.35)$$

出于实际工程的需要，为了保证前向模拟时的精度，本文在式(4.35)所示的高阶车辆运动学状态方程的基础上，使用四阶龙格库塔积分法对连续系统模型进行离散化：

$$x_{k+1} = x_k + \frac{dt}{6}(K_1 + 2K_2 + 2K_3 + K_4) \quad \cdots\cdots (4.36)$$

式中 dt 代表规划时间步长，即两个轨迹点的时间间隔，系数 K_1、K_2、K_3 和 K_4 可分别定义如下：

$$\begin{cases} K_1 = f(x_k, u_k) \\ K_2 = f(x_k + \dfrac{dt}{2}K_1, u_k) \\ K_3 = f(x_k + \dfrac{dt}{2}K_2, u_k) \\ K_4 = f(x_k + dt\ K_3, u_k) \end{cases} \quad \cdots\cdots (4.37)$$

在问题定义时的式(4.32)可以被看作式(4.35)、(4.36)、(4.37)三者的结合。从式(4.35)不难看出，该车辆模型是非线性的，而 ILQR 算法在迭代过程中需要对车辆状态方程进行线性化处理，即将车辆模型在(x_k, u_k)处一阶泰勒展开处理有：

$$\begin{aligned} \overline{f}(\delta x_k, \delta u_k) &= f(x_t + \delta x_t, u_t + \delta u_t) - f(x_t, u_t) \\ &= f_x^T \cdot \delta x_k + f_u^T \cdot \delta u_k \end{aligned} \quad \cdots\cdots (4.38)$$

式中 f_x^T 代表 $f(x_k, u_k)$ 对 x_k 的雅可比矩阵，f_u^T 代表 $f(x_k, u_k)$ 对 u_k 的雅可比矩阵。值得说明的是，由于在前文中使用四阶龙格库塔法求取车辆状态，因此在求微分时也需要按照四阶龙格库塔法求取。由于公式展开较为复杂且篇幅所限，在此处不再展开表示。

4.3.2 初始轨迹生成

在 CILQR 算法中，初始轨迹扮演着至关重要的角色，具有非常重要的作用。初始轨迹本质上是由车辆的初始状态和控制序列构成的，剩下的状态量均可以由初始状态

中国知网 https://www.cnki.net

和控制序列前向模拟得到，其中控制序列也被称为初始解。CILQR 算法本质上是一种迭代优化算法，它通过不断迭代来逐步优化控制策略，以找到使目标函数最小的最优解。初始解为迭代过程提供了一个起始点，从这个起始点出发，根据前文描述的规则和策略进行迭代更新，逐步逼近最优解。没有初始解和车辆的初始状态，算法的迭代过程就无法开始。同样的，高质量的初始解可以有效避免迭代结果陷入局部最优，还可以帮助算法更快收敛，减少迭代次数，提高求解效率。

为了确保优化算法在大多数情况下都能表现出色，需要在算法迭代开始前提供合理数量的初始解进行代价评估，并从中选择代价最小的初始解作为迭代的起始解。本文提出了三种生成初始解的方法，接下来将详细介绍这些方法。

首先，一种直观的方法是将算法上一帧迭代生成的最优解直接作为当前帧的初始解。这种方法的优点在于简单便捷，无需额外计算。然而，它也存在一个明显的局限性：在优化算法处于第一帧时，由于没有上一帧的最优解可供使用，因此无法直接应用此方法。为了解决这一问题，本文在第一帧时采取了特殊策略，将横向控制量固定为 0，纵向控制量固定为 0.5。

其次，在上一章中已经生成了时空参考轨迹，参考轨迹的每个点中均包含了相应的状态信息，除起始节点外，其余节点还包含了扩展结点时所用的控制信息，控制信息包括前轮转角 δ 和加速度 a。可以注意到，在上一节的高阶车辆运动学模型中，横向控制量是转角 δ 对时间的二阶导数，纵向控制量是加速度 a 对时间的二阶导数。因此，可以使用有限差分法，借助当前参考轨迹点的控制量和前后参考轨迹点的控制量来进行中心差分，近似求取前轮转角和加速度的二阶导数，分别构建横向控制量和纵向控制量，具体公式如下所示：

$$snap_i = \frac{\delta_{i+1} - 2\delta_i + \delta_{i-1}}{(dt)^2} \quad \cdots\cdots(4.39)$$

$$djerk_i = \frac{a_{i+1} - 2a_i + a_{i-1}}{(dt)^2} \quad \cdots\cdots(4.40)$$

值得一提的是，本文在初始解生成方法的设计中，曾尝试使用插值曲线和拟合曲线的方式来生成前轮转角曲线与加速度曲线，之后通过求取曲线上对应点的二阶导数来分别获取横、纵向控制量。然而，由于参考轨迹中控制量的数量较多，高阶曲线容易出现龙格现象，而低阶曲线的拟合精度又不尽如人意，所以通过生成曲线求导的方法来构造初始解的效果并不理想。相比之下，前文提到的有限差分法作为一种数值计算

中国知网 https://www.cnki.net

方法，更适合处理离散数据的场景，尤其是在数据量有限的情况下具有良好的表现。有限差分法不仅操作简便，而且计算效率极高，因此本文将其作为一种初始解的生成方案。

最后，初始解的本质是找到一组行之有效的控制量，所以可以通过使用较为简单的横向控制算法和纵向控制算法来分别生成横向和纵向的控制量。出于算法实时性的考虑，本文横向控制算法选择使用纯跟踪算法，纵向控制算法选择使用 PID 算法。

纯跟踪算法的原理如图 4.3 所示，基于当前车辆的后轮中心位置，在参考路径上向前方 l_d（称为预瞄距离）处匹配一个预瞄点，车辆以一定的前轮转角和速度向该预瞄点行驶。

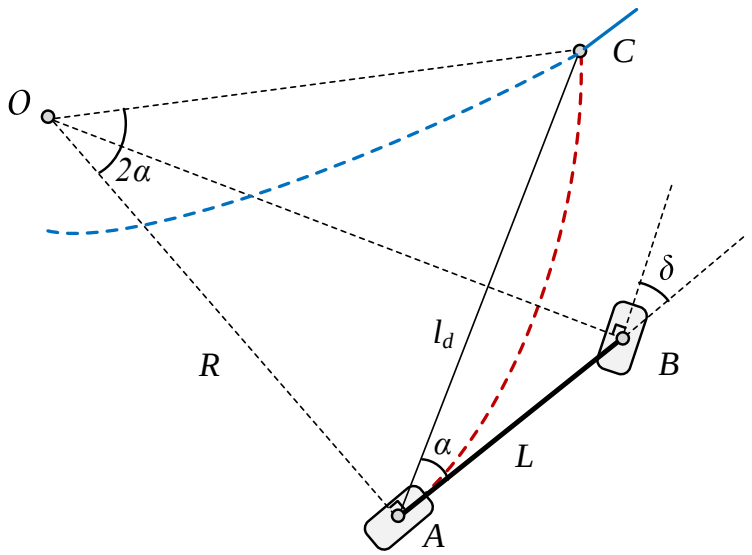


图 4.3 纯跟踪算法示意图

假设车辆可以按照一定的转弯半径 R 行驶至该预瞄点，然后根据预瞄距离 l_d、转弯半径 R、车辆坐标系下预瞄点的朝向角 α 之间的几何关系来计算前轮转角 δ，如图 4.3 所示的三角形 OAC 中，根据正弦定理可得：

$$\frac{l_d}{\sin(2\alpha)} = \frac{R}{\sin\left(\frac{\pi}{2} - \alpha\right)} \quad \cdots\cdots (4.41)$$

化简可得：

$$R = \frac{l_d}{2\sin(\alpha)} \quad \cdots\cdots (4.42)$$

根据车辆的运动学方程，有：

$$\delta = \tan^{-1}\frac{L}{R} \quad \cdots\cdots (4.43)$$

中国知网 https://www.cnki.net

所以，前轮转角为：

$$\delta = \tan^{-1}\frac{2L\sin(\alpha)}{l_d} \cdots\cdots (4.44)$$

得到需要的前轮转角后，可以依次求出 γ 和 $snap$：

$$\gamma = \frac{\delta - \delta_0}{dt} \cdots\cdots (4.45)$$

$$snap = \frac{\gamma - \gamma_0}{dt} \cdots\cdots (4.46)$$

式中，δ_0 表示车辆当前的前轮转角，γ_0 表示车辆当前的前轮转角速度。

PID 算法具有简单有效、响应速度快等特点，在工程中被广泛应用。其相关理论比较成熟，控制效果稳定有效。PID 控制算法通过调整比例系数 K_p、积分系数 K_i、微分系数 K_d 的大小来调整控制效果，其中：K_p 的作用是按比例调节系统的偏差；K_i 的作用是使系统消除稳态误差；K_d 的作用是减少超调和调节时间，改善系统的动态性能。

在连续系统中，PID 控制算法的表达式为：

$$u(t) = k_p \cdot e(t) + k_i\int_0^t e(t)dt + k_d\frac{de(t)}{dt} \cdots\cdots (4.47)$$

在离散系统中，PID 控制算法的表达式为：

$$u(i) = k_p \cdot e(i) + k_i T\sum_{j=0}^{k} e(j) + k_d\frac{e(i) - e(i-1)}{T} \cdots\cdots (4.48)$$

在上面的公式中，$e(i)$表示输入偏差，其数值为输入量与输出量之间的差，i 为采样序号，T 为采样时间。

在前文生成的参考轨迹中已经包含了各个节点之间的速度状态量，因此本文将设计一个三闭环 PID 控制算法来生成车辆的纵向控制量。最外环的控制器会根据车辆的速度与目标速度生成目标加速度，中间环的控制器会根据车辆当前的加速度与目标加速度生成目标加加速度，最内环的控制器会根据车辆当前的加加速度与目标加加速度生成纵向控制量，具体步骤如图 4.4 所示：

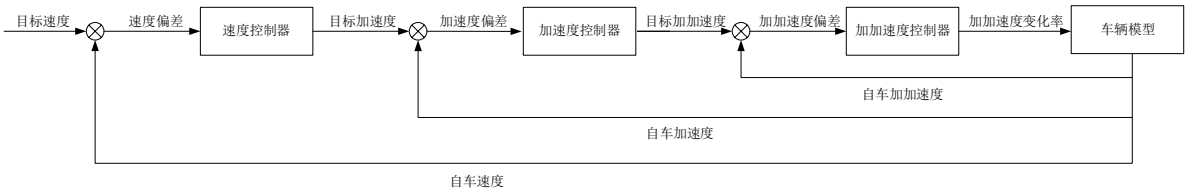


图 4.4 纵向三闭环 PID 控制器

将基于纯跟踪算法生成的横向控制量与基于 PID 算法生成的纵向控制量结合起来，

中国知网 https://www.cnki.net

就构成了 CILQR 算法的初始解。

4.3.3 目标函数设计

本文对于时空联合轨迹优化问题的目标函数$\ell_k(x_k,u_k)$的设计，将会从以下几个方面进行考虑，分别是变量优化目标、驾乘舒适性优化目标、避障安全性优化目标，接下来将依次进行介绍：

首先是变量本身的优化目标，在前文生成的时空参考轨迹中，存在车辆的部分状态变量。对于这部分状态量，其优化目标定义为与时空参考轨迹点的偏移量尽可能小，即期望最优轨迹和时空参考轨迹在位置、航向角、前轮转角、速度和加速度间的偏差尽可能小。因此，本文将部分状态量的目标函数设计成如下二次函数的形式：

$$\ell^{ref}=(x-x^r)^T w_{ref}(x-x^r) \quad (4.49)$$

式中$x^r=[x_c^r,y_c^r,\theta^r,\delta^r,v^r,a^r]^T$为时空参考轨迹对应采样时间间隔的轨迹点的状态信息。

$w_{ref}=\begin{bmatrix} w_{pos} & & & & & \\ & w_{pos} & & & & \\ & & w_{\theta} & & & \\ & & & w_{\delta} & & \\ & & & & w_v & \\ & & & & & w_a \end{bmatrix}$，$w_{\mathrm{pos}}$，$w_{\theta}$，$w_{\delta}$，$w_{\mathrm{v}}$，$w_{\mathrm{a}}$分别代表车辆相应状态量与参考轨迹点位置、航向角、前轮转角、速度和加速度的偏移量的代价权重值。

除了时空参考轨迹中体现的状态量外，还有部分车辆状态量未在时空参考轨迹中体现，这些高阶的状态量通常与安全性和舒适性相关，因此希望这些状态量尽可能小。在目标函数设计的过程中，本文将其余状态量的优化目标设计成如下二次函数的形式：

$$\ell^{state}=x^T w_{state} x \quad (4.50)$$

式中$w_{state}=\begin{bmatrix} w_{\gamma} & \\ & w_{jerk} \end{bmatrix}$，$w_{\gamma}$和$w_{\mathrm{jerk}}$分别代表车辆状态量中前轮转角速度和加加速度优化目标项的代价权重值。

对于控制量，出于舒适性和能量消耗的考虑，同样希望这些控制量尽可能小。因此，在目标函数设计的过程中，本文将控制量的优化目标设计成如下二次函数的形式：

$$\ell^{control}=u^T w_{control} u \quad (4.51)$$

中国知网 https://www.cnki.net

式中 $w_{control}=\begin{bmatrix} w_{snap} & \\ & w_{djerk} \end{bmatrix}$，$w_{\mathrm{snap}}$ 和 w_{djerk} 分别代表车辆状态量中前轮转角速度变化率和加加速度变化率优化目标项的代价权重值。

其次是舒适性优化目标。车辆行驶过程中的舒适性往往与加速度和加加速度相关，在车辆行驶过程中，一般希望减少速度和加速度的突变，进而希望加速度和加加速度的数值尽可能小。在式(4.50)中对车辆的纵向加加速度添加了舒适性约束。同时，在上一章进行时空节点成本函数设计时，利用公式(3.16)至(3.20)推导了车辆在行驶过程中横向加速度和横向加加速度的计算方式，本章将对其添加约束，以保证驾乘人员的舒适性。本文将部分舒适性优化目标设计成如下二次函数的形式：

$$\ell^{comfort}=x^{T}w_{comfort}x \cdots\cdots(4.52)$$

式中，$x=[a,a_{lat},jerk_{lat}]^{T}$ 为车辆行驶过程中的纵向加速度、横向加速度、横向加加速度；$w_{confort}=\begin{bmatrix} w_{alon} & & \\ & w_{alat} & \\ & & w_{jlat} \end{bmatrix}$，$w_{\mathrm{alon}}$，$w_{\mathrm{alat}}$，$w_{\mathrm{jlat}}$ 分别代表纵向加速度、横向加速度、横向加加速度优化目标项的代价权重值。

关于舒适性的考虑，除了将横纵向加速度和加加速度本身的大小设置为优化目标外，当加速度和加加速度在一定范围内时，驾乘人员并不会感到不舒适；而当它们超过一定范围时，驾乘人员才会感到不适。因此，还希望在加速度和加加速度超过一定界限时能够额外产生代价。本文将其余舒适性优化目标设计成如下二次函数的形式：

$$\begin{aligned} A&=diag(max(|x_i|-x_i^{limit},0)) \\ \ell^{limit}&=A^{T}w_{limit}A \end{aligned} \cdots\cdots(4.53)$$

式中，$x^{limit}=[a^{limit},a_{lat}^{limit},jerk^{limit},jerk_{lat}^{limit}]^{T}$ 为车辆行驶过程中纵向加速度、横向加速度、纵向加加速度、横向加加速度的舒适性边界；$w_{limit}=\begin{bmatrix} w_{al} & & & \\ & w_{alatl} & & \\ & & w_{jl} & \\ & & & w_{jlatl} \end{bmatrix}$，$w_{\mathrm{al}}$，$w_{\mathrm{alatl}}$，$w_{\mathrm{jl}}$，$w_{\mathrm{jlatl}}$ 分别代表纵向加速度、横向加速度、纵向加加速度、横向加加速度的舒适性边界优化目标项的代价权重值。

最后是避障安全性约束，在上一章中细致地讲述了如何生成代表道路边界和障碍物边界的半平面，并且可以根据式(3.28)的计算结果获得车辆与这些边界的位置关系。

中国知网 https://www.cnki.net

在安全性优化目标中，将车辆与各个边界之间根据式(3.28)的计算结果记为 $d=[d_{vsoft},d_{vhard},d_{rsoft},d_{rhard}]^T$，$d_{\mathrm{vsoft}}$，$d_{\mathrm{vhard}}$，$d_{\mathrm{rsoft}}$，$d_{\mathrm{rhard}}$ 分别表示车辆与保守的障碍物边界、障碍物边界、保守的道路边界、道路边界之间的计算结果。根据车辆与边界之间的位置关系来添加约束，以保证驾乘人员的避障安全性。本文将部分安全性优化目标设计成如下二次函数的形式：

$$\begin{aligned} A &= diag(max(-d_i,0)) \\ \ell^{safe} &= A^T w_{safe} A \end{aligned} \quad \cdots\cdots (4.54)$$

式中，$w_{safe}=\begin{bmatrix} w_{vsoft} & & & \\ & w_{vhard} & & \\ & & w_{rsoft} & \\ & & & w_{rhard} \end{bmatrix}$，$w_{\mathrm{vsoft}}$，$w_{\mathrm{vhard}}$，$w_{\mathrm{rsoft}}$，$w_{\mathrm{rhard}}$ 分别代表车辆与保守的障碍物边界、障碍物边界、保守的道路边界、道路边界之间的位置关系优化目标项的代价权重值。

4.3.4 车辆性能约束

本文在前一节中设计了与安全性相关的目标函数，但尚未将车辆性能限制纳入考虑。实际上，轨迹规划的结果将由控制模块执行，这就要求在规划过程中必须确保不超出车辆的性能极限。此外，在车辆行驶过程中，还需考虑功能安全，以确保在系统出现故障时，车辆状态仍能保持在安全可控的范围内。鉴于交通环境的不确定性，本文仅对车辆自身状态施加不等式约束，并通过前文所述的增广拉格朗日方法，将这些约束整合进目标函数中。这样的处理方式既考虑了车辆的动态性能，也兼顾了安全性要求。

本文中，车辆状态量的性能约束由以下边界构成：速度的上下界约束、加速度的上下界约束、加加速度的上下界约束、前轮转角的上下界约束以及前轮转角速度的上下界约束。

轨迹优化问题中的车辆速度边界约束如下：

$$v_{\min} \le v \le v_{\max} \quad \cdots\cdots (4.55)$$

式中，$v_{\min}$ 和 $v_{\max}$ 分别是自动驾驶车辆在轨迹规划任务中所允许的最小速度和最大速度。根据式(4.29)，将其写为惩罚函数有如下表示：

$$\lambda-\mu c(v)=\min\left[0,\lambda-\mu(v-v_{\max})\right]+\min\left[0,\lambda-\mu(v_{\min}-v)\right] \quad \cdots\cdots (4.56)$$

轨迹优化问题中的车辆加速度边界约束如下：

中国知网 https://www.cnki.net

$$a_{\min} \le a \le a_{\max} \cdots\cdots (4.57)$$

式中，$a_{\min}$ 和 $a_{\max}$ 分别代表自动驾驶车辆在执行轨迹规划任务时，为确保功能安全所允许的最小加速度和最大加速度。对应的惩罚函数有如下表示：

$$\lambda - \mu c(a) = \min\left[0, \lambda - \mu(a - a_{\max})\right] + \min\left[0, \lambda - \mu(a_{\min} - a)\right] \cdots\cdots (4.58)$$

轨迹优化问题中的车辆加加速度边界约束限制如下所示：

$$jerk_{\min} \le jerk \le jerk_{\max} \cdots\cdots (4.59)$$

式中，$jerk_{\min}$ 和 $jerk_{\max}$ 分别代表自动驾驶车辆在执行轨迹规划任务时，为确保功能安全所允许的最小加加速度和最大加加速度。对应的惩罚函数有如下表示：

$$\lambda - \mu c(jerk) = \min\left[0, \lambda - \mu(jerk - jerk_{\max})\right] + \min\left[0, \lambda - \mu(jerk_{\min} - jerk)\right] \cdots (4.60)$$

轨迹优化问题中的车辆前轮转角边界约束如下：

$$\delta_{\min} \le \delta \le \delta_{\max} \cdots\cdots (4.61)$$

式中，$\delta_{\min}$ 和 $\delta_{\max}$ 分别代表自动驾驶车辆在执行轨迹规划任务时，为确保功能安全所允许前轮转角的最小值和最大值。对应的惩罚函数表示如下：

$$\lambda - \mu c(\delta) = \min\left[0, \lambda - \mu(\delta - \delta_{\max})\right] + \min\left[0, \lambda - \mu(\delta_{\min} - \delta)\right] \cdots\cdots (4.62)$$

轨迹优化问题中的车辆前轮转角速度边界约束如下：

$$\gamma_{\min} \le \gamma \le \gamma_{\max} \cdots\cdots (4.63)$$

式中，$\gamma_{\min}$ 和 $\gamma_{\max}$ 分别代表自动驾驶车辆在执行轨迹规划任务时，为确保功能安全所允许的最小前轮转角速度和最大前轮转角速度。对应的惩罚函数有如下表示：

$$\lambda - \mu c(\gamma) = \min\left[0, \lambda - \mu(\gamma - \gamma_{\max})\right] + \min\left[0, \lambda - \mu(\gamma_{\min} - \gamma)\right] \cdots\cdots (4.64)$$

综合考虑车辆性能和功能安全后，得到了关于车辆状态量的不等式约束，并将其转化为惩罚函数的形式，嵌入到目标函数中作为惩罚项，最终确保在算法迭代完成后的优化结果中，相应的车辆状态量既不会超越其性能极限，也不会违反功能安全的标准。

4.4 本章小结

本章首先在笛卡尔坐标系中对时空联合轨迹优化问题进行了定义与阐述。随后，介绍了如何利用 ILQR 算法求解该问题，并在求解过程中对该算法进行了改进，以提升其性能。为了弥补 ILQR 算法的不足，本章综合分析了多种 CILQR 算法的优缺点，并最终采用改进后的增广拉格朗日方法来实现不等式约束的处理。之后，本章针对时空

中国知网 https://www.cnki.net

联合轨迹优化问题的细节进行了深入设计与补充，提出了一种更高阶的车辆运动学模型，以增强优化过程计算结果的平滑性和准确性。同时，设计了多种初始轨迹生成算法，以加快求解速度。从安全性、舒适性等多个维度出发，构建了合理的目标函数。最后，结合车辆的性能限制和功能安全要求，将不等式约束纳入目标函数中，进一步完善了时空联合轨迹优化问题的建模与求解过程。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第5章 城区场景轨迹规划算法仿真验证

5.1 引言

前文针对城市道路场景的轨迹规划问题进行了系统研究，对参考线生成算法进行详细说明，并在特殊场景中进行验证。设计了时空混合 ARA*算法来搜索生成时空参考轨迹，之后又基于 CILQR 算法框架，对时空参考轨迹进行优化。为了验证前文设计的轨迹规划算法的可行性，本文在 Ubuntu 系统下基于 ROS1 搭建了相应的仿真实验平台，选取有代表性的交通场景进行仿真，并对仿真结果进行分析，综合验证本文提出的轨迹规划算法的效果。

5.2 仿真实验平台搭建与参数设置

本文选用的 Ubuntu 系统版本为 20.04，基于与该系统对应的 ROS1 Noetic 版本搭建仿真实验平台，对轨迹算法进行验证，同时使用 RViz 作为可视化工具。为了提高算法的运行效率与实时性，本文算法的核心代码均使用 C++语言编写，算法运行过程中的数据存入 csv 文件中，并通过 Python 语言读取运行数据，进行可视化绘制与显示。仿真平台的硬件配置如下：处理器型号为英特尔 i9-13900，处理器运行频率为 5.60×2GHz，内存为 64GB。

仿真过程中用到的车辆配置参数、时空参考轨迹生成算法中使用到的主要参数和时空联合轨迹优化算法中使用到的主要参数分别在表 5.1、表 5.2 和表 5.3 中给出，由于篇幅所限，其余参数不再赘述。在给出使用到的主要参数后，本节将对前文的部分关键内容进行仿真说明。

表 5.1 车辆配置参数

参数符号	参数描述	参数值	参数单位
L_{length}	车辆长度	4.69	m
L_{B}	车辆宽度	1.92	m
L_{W}	车辆轴距	2.80	m
L_{F}	车辆前悬长度	0.96	m
L_{R}	车辆后悬长度	0.93	m

中国知网 https://www.cnki.net

表 5.2 时空参考轨迹生成算法主要参数

参数符号	参数描述	参数值	参数单位
s_{space}	规划空间纵向分辨率	1.0	m
l_{space}	规划空间横向分辨率	0.2	m
t_{space}	规划空间时间分辨率	0.1	s
δ_{max}，δ_{min}	最大最小前轮转角	0.7，-0.7	rad
a_{max}，a_{min}	最大最小加速度	2.0，-3.0	m/s^2
v_{max}，v_{min}	最大最小速度	16.67，0.0	m/s
A	离散加速度集	{-3，-1，-0.2，0，0.1，0.3，0.6，1，1.5，2}	-
N	前轮转角离散数量	15	-
dt	节点扩展时间间隔	0.2	s
s_{bound}	检测范围边界	4.0	m
ϵ	启发函数膨胀系数	4.0	-
$\Delta\epsilon$	膨胀系数差值	0.6	-
t_{total}	时间阈值	15	ms

如图 5.1 所示，本文设计了一个非常特殊的场景来对时空混合 ARA*算法的搜索效果进行说明。在图 5.1 中，蓝色矩形框代表自车位置，绿色矩形框代表车道中的静止车辆障碍物。在此场景中，本文希望自车能够搜索得到一条顺利穿行至右下方车道的轨迹。图中颜色不同的各曲线代表着时空混合 ARA*算法中启发函数膨胀系数不同时算法搜索得到的轨迹结果。从图中结果和前文对算法代价函数的设计中可以看出，当 ϵ=1 时的轨迹结果是当前场景的最优解；当 ϵ 大于 1，特别是取 5、7、10 时，为了搜索效率，牺牲了一定最优性，得到的轨迹结果是次优的。值得说明的是，此场景是为了展示时空混合 ARA*算法的搜索效果而特殊设计的，ϵ 的取值在此场景下有一定的不合理性，本文在实际使用中的取值如表 5.2 所示。从图中可以看出，当 ϵ 的取值从大到小变化时，时空混合 ARA*算法会在之前搜索结果的基础上不断完善，提升轨迹结果的质量，在时间允许的情况下最终生成最优参考轨迹。

中国知网 https://www.cnki.net

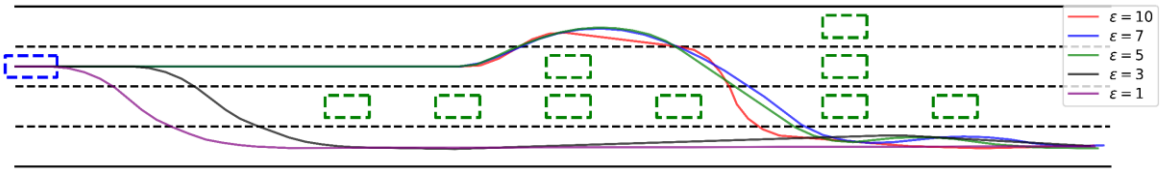


图 5.1 时空混合 ARA*算法效果图

表 5.3 时空联合轨迹优化算法主要参数

参数符号	参数描述	参数值	参数单位
ρ	初始正则化系数	1	-
$\rho_{\max}$	最大正则化系数	1024	-
$\rho_{\min}$	最小正则化系数	0.01	-
Δ_0	修正因子	2	-
c_1	接受迭代系数	0.25	-
ε	收敛阈值	0.02	-
μ	惩罚系数	10	-
m	惩罚系数增长系数	20	-
λ	拉格朗日乘子	100	-
$iter_{\max}$	最大迭代次数	10	-
dt	规划时长采样间隔	0.2	s
N	规划时长采样数量	20	-
a_{limit}	加速度边界	1.5	m/s^2
$a_{\text{limit}}^{\text{lat}}$	横向加速度边界	3.5	m/s^2
$jerk_{\text{limit}}$	加加速度边界	5.0	m/s^3
$jerk_{\text{limit}}^{\text{lat}}$	横向加加速度边界	7.0	m/s^3
$v_{\max}$，$v_{\min}$	最大最小速度	16.67，0.0	m/s
$a_{\max}$，$a_{\min}$	最大最小加速度	5.0，-5.0	m/s^2
$jerk_{\max}$，$jerk_{\min}$	最大最小加加速度	10.0，-10.0	m/s^3
$\delta_{\max}$，$\delta_{\min}$	最大最小前轮转角	0.7，-0.7	rad
$\gamma_{\max}$，$\gamma_{\min}$	最大最小前轮转角速度	0.6，-0.6	rad/s

中国知网 https://www.cnki.net

如图 5.2 所示，本文设计了一个较为简单的场景，用以说明 CILQR 算法迭代开始前初始轨迹的生成情况。在图 5.2 中，蓝色矩形框代表自车位置，绿色矩形框代表车道中的静止车辆障碍物。红色轨迹是由时空混合 ARA*算法搜索生成的时空参考轨迹。由于是第一帧，本应由上一帧控制量生成的黄色初始轨迹此时由固定控制量生成，因此相较于其余初始解，其效果相对较差。蓝色轨迹是根据时空参考轨迹的控制量，通过有限差分法计算得出的。可以看到，这种构建方法生成的初始轨迹与时空参考轨迹具有很高的相似性。紫色轨迹则是通过纯跟踪算法和 PID 算法计算得出的初始轨迹，它在贴近时空参考轨迹的同时，还考虑了车辆的实际控制情况，因此变化更加平缓。通过目标函数对这些初始轨迹分别进行计算后，最终选取由控制算法生成的初始解作为迭代起始解。经过多次迭代后的轨迹优化结果如图中绿色轨迹所示，相较于初始解，优化后的轨迹与障碍物的距离更远，具有更好的安全性。

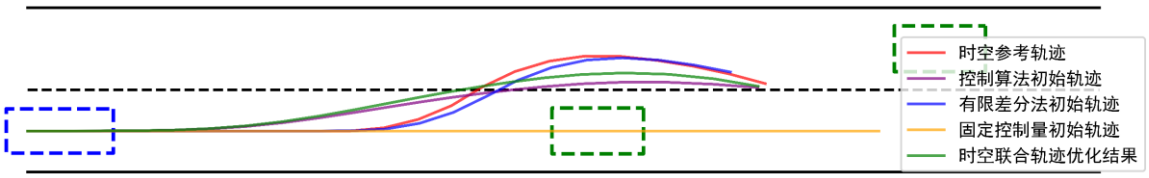


图 5.2 初始轨迹生成效果图

5.3 多场景仿真实验结果分析

由于城市道路中存在交叉路口多、道路狭窄、车辆行驶速度较低、部分道路弯曲程度大等情况，本文选取借道超车、无信号灯交叉路口、大曲率 U 形弯等场景，对本文提出的时空联合轨迹规划算法进行仿真验证。

5.3.1 借道超车场景分析

在城市道路中行驶时，经常会出现前方车速较慢的情形。为了提高通行效率，自车一般会借助旁边车道进行超车。如图 5.3 所示，本文在 RViz 中模拟了一个常见的借道超车场景。在这个场景中，本文将单个车道宽度设定为 3.6m，自车如图中黄色矩形框所示，并设定其初始速度为 8.0m/s，方向为向右行驶。本文在自车前方 30m 处设置了一辆与自车同向行驶的障碍物车辆，如图中粉色矩形框所示，其速度设置为 4.0m/s，在自车旁边车道 100m 处设置了一辆与自车反向行驶的障碍物车辆，如图中绿色矩形框

中国知网 https://www.cnki.net

所示，其速度设置为 3.0m/s。

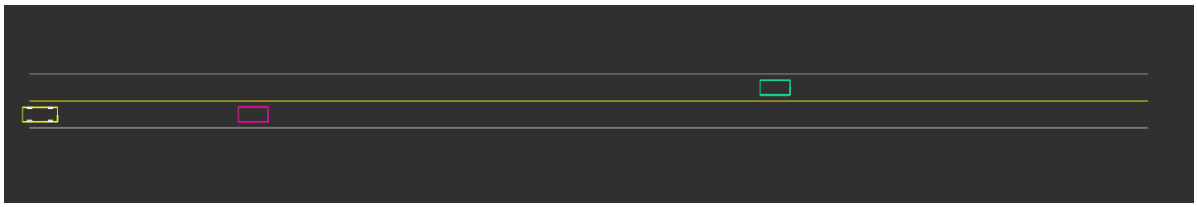


图 5.3 借道超车仿真场景

如图 5.4 所示，图中黄色轨迹为本文第三章中提出的时空混合 ARA*算法输出的参考轨迹。从参考轨迹的搜索结果来看，在进行超车前，由于前方车道被慢速车占据不可通行，算法会借用旁边车道进行超车。超车结束后，为了避免与对向车道车辆发生碰撞，算法又重新搜索出一条回到原先车道的轨迹。由此可以看出，时空混合 ARA*算法能够很好地同时考虑障碍物车辆的空间和时间信息，进而处理动态障碍物的情景，最终生成一条既保证安全性又保证通过性的轨迹。同时，在图 5.4 中粉色轨迹为时空联合优化算法对参考轨迹优化后的结果，这说明本文第四章中提出的基于 CILQR 算法的时空联合轨迹优化算法能够优化得到超越慢速车且避开对向行驶障碍物车辆的轨迹，优化结果在贴合时空参考轨迹的前提下足够平滑，同时满足道路边界和车辆边界的安全性约束。

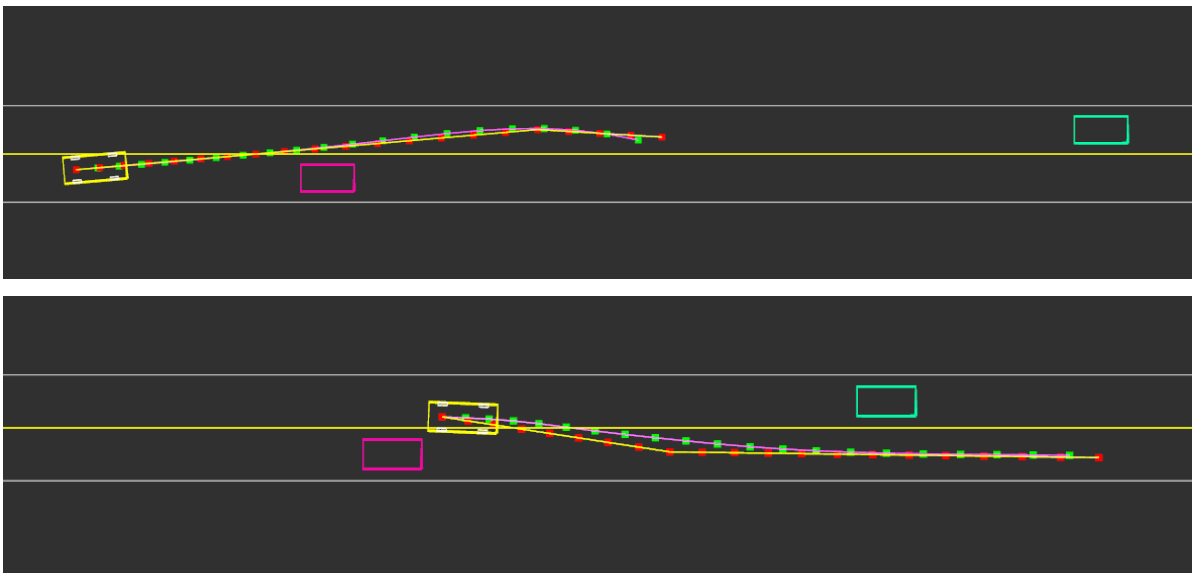


图 5.4 借道超车场景不同时刻轨迹规划结果

为了更加直观地观察借道超车时自车的轨迹状态，本文将自车与障碍物车辆的历史轨迹分别在图中进行绘制，结果如图 5.5 所示。为了便于清晰观察，每隔两秒绘制一次车辆位置，以免过于稠密。同时，在图片中使用蓝色矩形框来表示自车位置，其余障碍物车辆的颜色与仿真环境中的颜色相同。从图 5.5 中车辆不同时刻的坐标构成的行驶

中国知网 https://www.cnki.net

轨迹可以看出，规划出的路径在借道超车任务中平滑变化，且未与周围障碍物车辆发生碰撞。本场景中，时空参考轨迹生成算法的平均单帧耗时为 8.556ms，时空联合轨迹优化算法的平均单帧耗时为 1.296ms，满足轨迹规划算法的实时性要求。

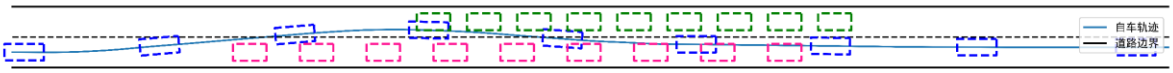


图 5.5 借道超车场景轨迹图

图 5.6 和图 5.7 分别为借道超车轨迹规划过程中的横向状态量和纵向状态量信息。从图 5.6 中可以看出，在借道超车过程中以及借道超车结束后车辆返回原车道的过程中，航向角变化平滑，前轮转角和前轮转角速度的数值均不大，这说明规划出的轨迹在借道超车过程中能让驾乘人员具有较好的横向舒适性。从图 5.7 中可以看出，在借道超车过程中，自车速度相对稳定，加速度和加加速度在 0 附近波动，说明此阶段车辆行驶较为平稳。当借道超车结束后，车辆在返回原车道过程后速度有所增加，此时加速度和加加速度的数值也均在合理范围内，有效地保证了驾乘人员的纵向舒适性。

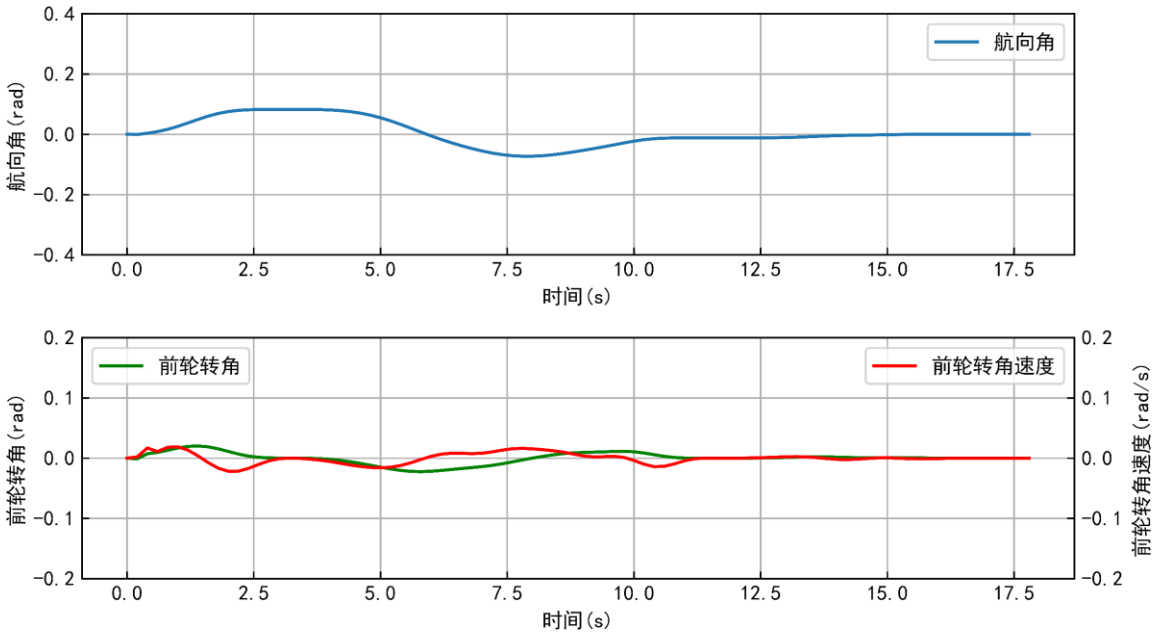


图 5.6 借道超车场景横向状态量

中国知网 https://www.cnki.net

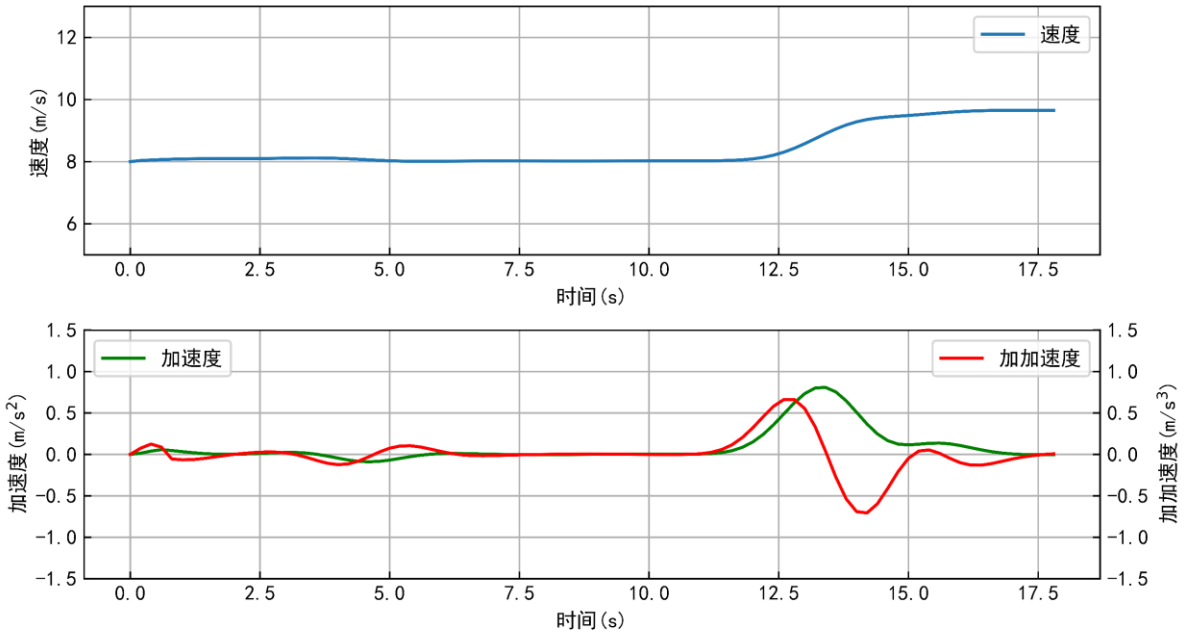


图 5.7 借道超车场景纵向状态量

5.3.2 交叉路口左转场景分析

在城市道路中，交叉路口是一种非常常见的交通场景。在没有交通信号灯的路口，左转车辆需要综合考虑其余往来车辆、行人等多种因素，以规划出一条平滑且安全的行驶轨迹，这对轨迹规划算法来说具有很大挑战。如图 5.8 所示，本文在 RViz 中构建了交叉路口场景，此场景与图 2.11 中的道路情况相同，因此参考线的生成效果在此不再赘述。在此场景中，自车将在右侧存在直行车辆的情况下并入车流进行左转。本文将单个车道宽度设定为 3.6m，自车如图中黄色矩形框所示，并设定其初始速度为 8.0m/s，当在行驶到交叉路口时，车辆将左转行驶。障碍物车辆如图中粉色和绿色矩形框所示，其车速分别为 4.5m/s 和 4.0m/s，均为向前直线行驶。

中国知网 https://www.cnki.net

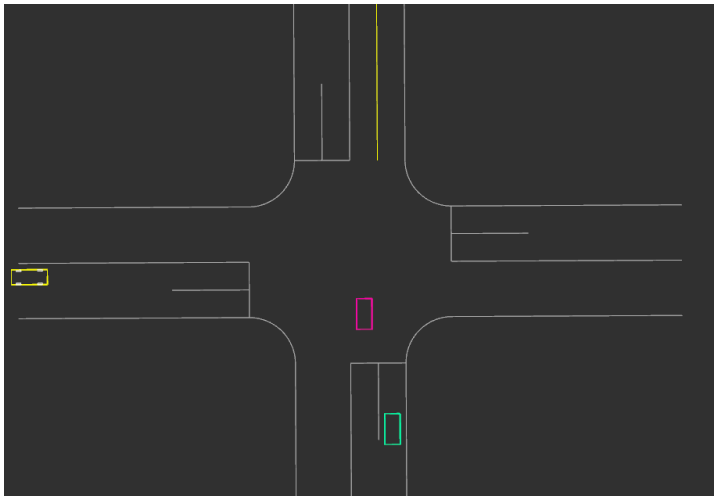
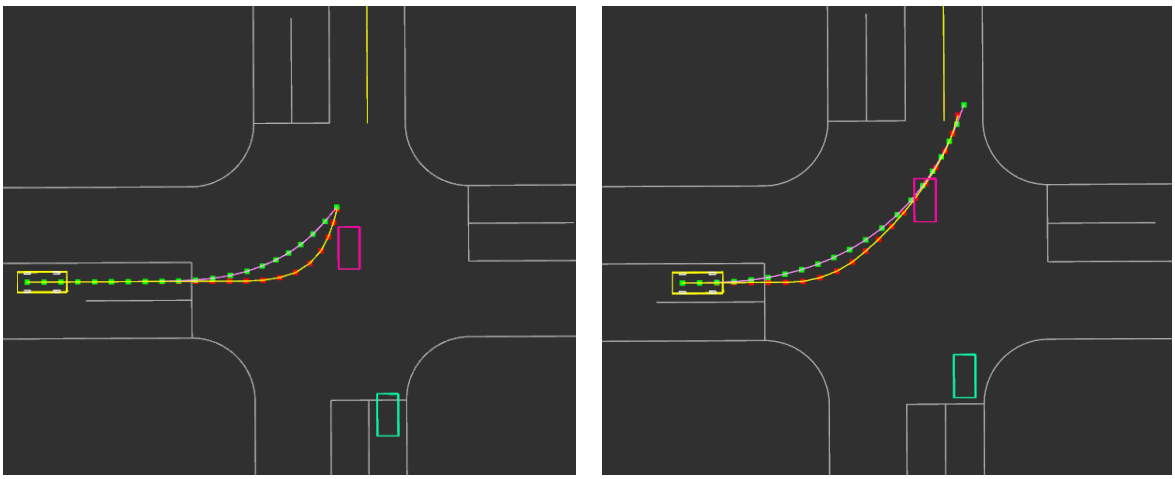


图 5.8 交叉路口左转仿真场景

如图 5.9 所示，图中黄色轨迹为轨迹规划算法输出的时空参考轨迹。从 5.9(a)可以看出，在车辆进行左转时，自车先搜索出跟随障碍物车辆行驶的参考轨迹，但由于前方障碍物车辆行驶速度低，长时间占据前方道路，在 5.9(b)中可以看出，轨迹规划算法又搜索生成了借助两障碍物车辆行驶间隙驶向旁边车道的参考轨迹，在保证安全性的同时，还保证了通行效率。图中粉色轨迹为时空联合优化算法对参考轨迹优化后的结果，从 5.9(a)可以看出，优化后的轨迹相较于参考轨迹会更早地开始转向，车轮转角的变化幅度也更小，这说明轨迹优化算法在保证车辆的运动学约束和安全性的前提下，还可以提升轨迹的平滑性与舒适性。



(a) (b)

图 5.9 交叉路口左转场景不同时刻轨迹规划结果

为了更加直观地观察左转时自车的轨迹状态，同样将自车与障碍物车辆的历史轨迹分别在图中进行绘制，结果如图 5.10 所示。为了便于清晰观察，每隔两秒绘制一次车辆位置，以免过于稠密影响观察效果。在图中，使用蓝色矩形框来表示自车位置，其

中国知网 https://www.cnki.net

余障碍物车辆的颜色与仿真环境中的车辆颜色相同。从图 5.10 中自车的行驶路径可以看出，在交叉路口左转任务中，算法规划出的路径是平滑无突变的，满足车辆的运动要求，同时未与其他障碍物车辆发生碰撞，满足安全性要求。本场景中，时空参考轨迹生成算法的平均单帧耗时为 10.539ms，时空联合轨迹优化算法的平均单帧耗时为 1.312ms，满足轨迹规划算法的实时性要求。

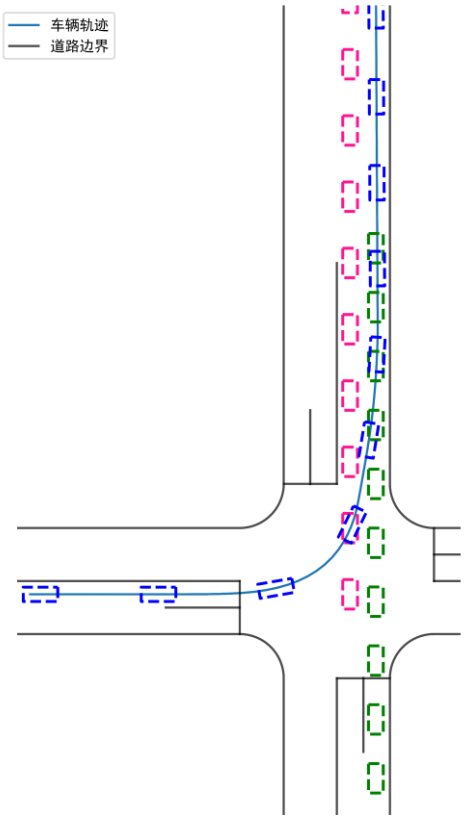


图 5.10 交叉路口左转场景轨迹图

图 5.11 至图 5.12 为交叉路口左转场景轨迹规划过程中的横纵向状态量信息。从图 5.11 中可以看出，在左转过程中及左转完成后，车辆航向角和前轮转角平滑变化，前轮转角和前轮转角速度的数值均在合理范围内，未出现极端突变情况。从图 5.12 中可以看出，在自车开始左转并与障碍物车辆开始交互后，自车车速降低，这在保障安全性的同时还提升了驾乘人员的横向舒适性。在整个左转任务中，加速度和加加速度的数值均在合理范围内，保证了驾乘人员的纵向舒适性。

中国知网 https://www.cnki.net

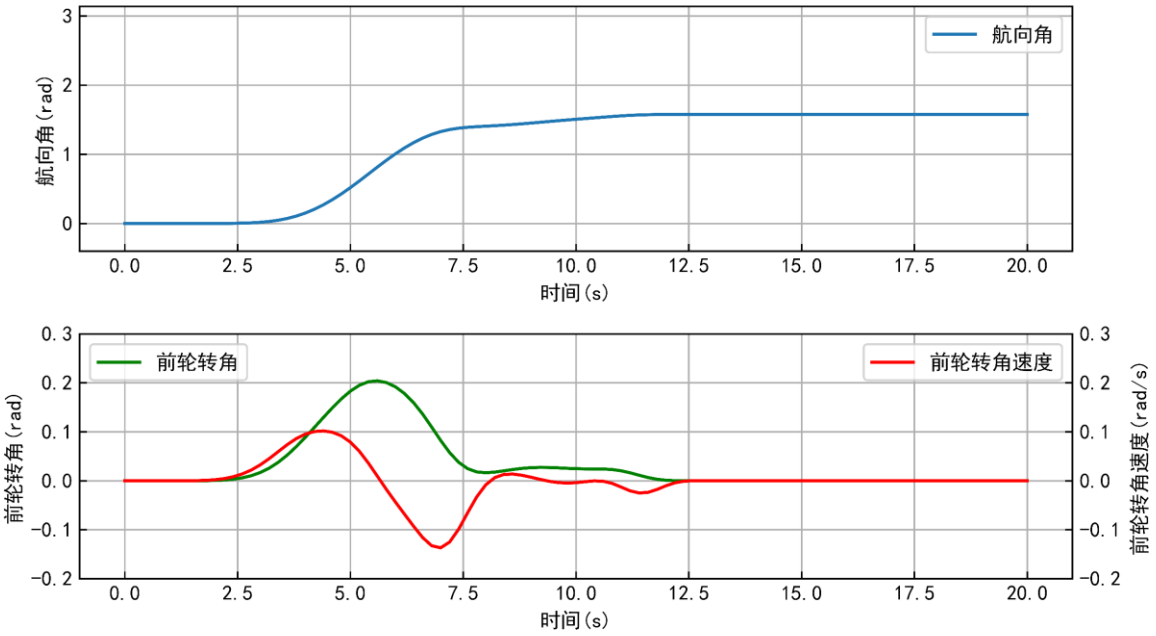


图 5.11 交叉路口左转场景横向状态量

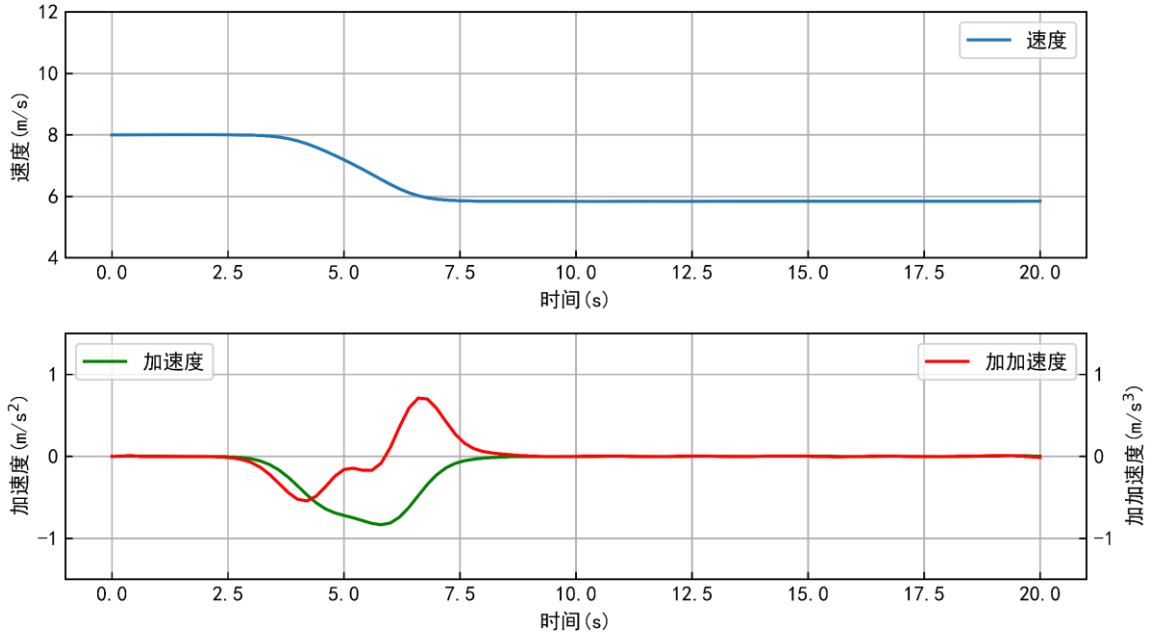


图 5.12 交叉路口左转场景纵向状态量

5.3.3 大曲率 U 形弯场景分析

车辆在城市道路中行驶时，经常会遇到掉头、U 形弯等大曲率场景。此时，车辆的行驶轨迹需要满足车辆转弯半径的限制，同时还要在保证安全的前提下避免频繁的加减速以及转向操作。相较于其他结构化场景，这对整个轨迹规划算法的求解稳定性有

中国知网 https://www.cnki.net

着更高的要求。如图 5.13 所示，本文在 RViz 中构建了一个双车道的大曲率 U 形弯场景，在此场景中，单个车道宽度设定为 3.6m，自车如图中黄色矩形框所示，并设定其初始速度为 3.0m/s，方向为向右行驶。转弯处的道路由一段圆弧构成，其中最内侧的道路中心线对应的曲率为 $0.22m^{-1}$。

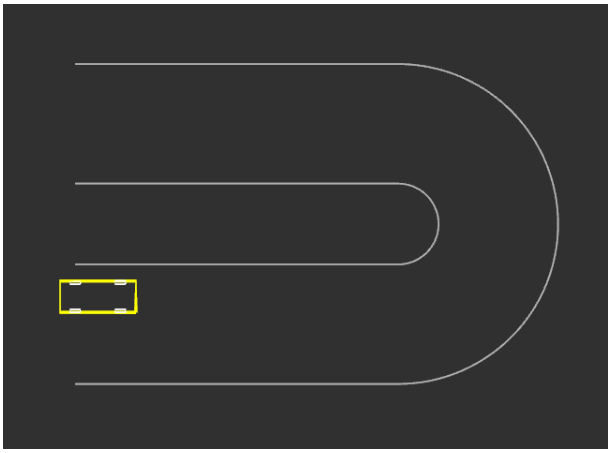


图 5.13 大曲率 U 形弯仿真场景

如图 5.14 所示，图中黄色轨迹为轨迹规划算法输出的时空参考轨迹。从图中可以看出，时空混合 ARA*算法搜索得到的参考轨迹能够顺利通过大曲率 U 形弯场景。图中粉色轨迹则是时空联合轨迹优化算法在综合考虑各种约束后，对参考轨迹进行优化所得的结果，相较于参考轨迹，优化后的结果更加平滑。

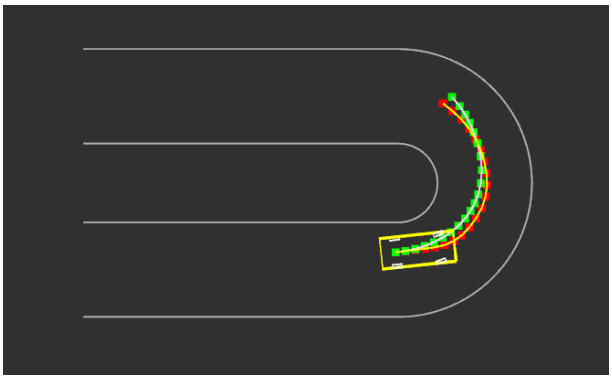


图 5.14 大曲率 U 形弯仿真场景单帧轨迹规划结果

如图 5.15 所示，本文将车辆在 U 形弯通行的轨迹在图中进行展示，每间隔一秒绘制一次车辆位置。从车辆的行驶轨迹可以看出，在通过 U 形弯时，车辆的转弯半径约为 5.6m 左右，这证明本文提出的轨迹规划算法能够规划出安全且平滑的轨迹，以顺利通过大曲率场景。本场景中，时空参考轨迹生成算法的平均单帧耗时为 7.855ms，时空联合轨迹优化算法的平均单帧耗时为 1.236ms，满足轨迹规划算法的实时性要求。

中国知网 https://www.cnki.net

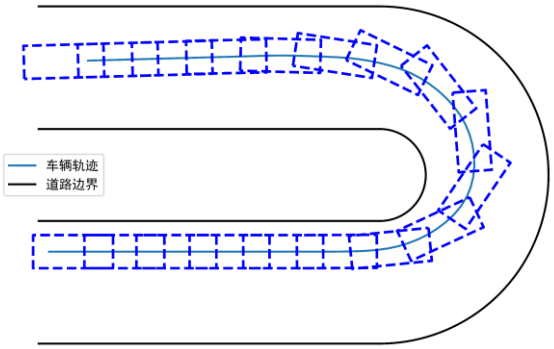


图 5.15 大曲率 U 形弯场景轨迹图

图 5.16 和图 5.17 分别为大曲率 U 形弯场景轨迹规划过程中的横向状态量和纵向状态量信息。从图 5.16 中可以看出，在车辆转向通过 U 形弯的过程中，自车的航向角和前轮转角平滑无突变，且前轮转角始终没有超过车辆自身性能和功能安全要求的限制，前轮转角和前轮转角速度均在合理范围内。从图 5.17 中可以看出，在转弯过程中，自车速度平顺稳定，加速度和加加速度始终在 0 附近波动，说明车辆行驶较为平稳，未出现频繁加减速现象，保证了驾乘人员的安全和纵向舒适性。

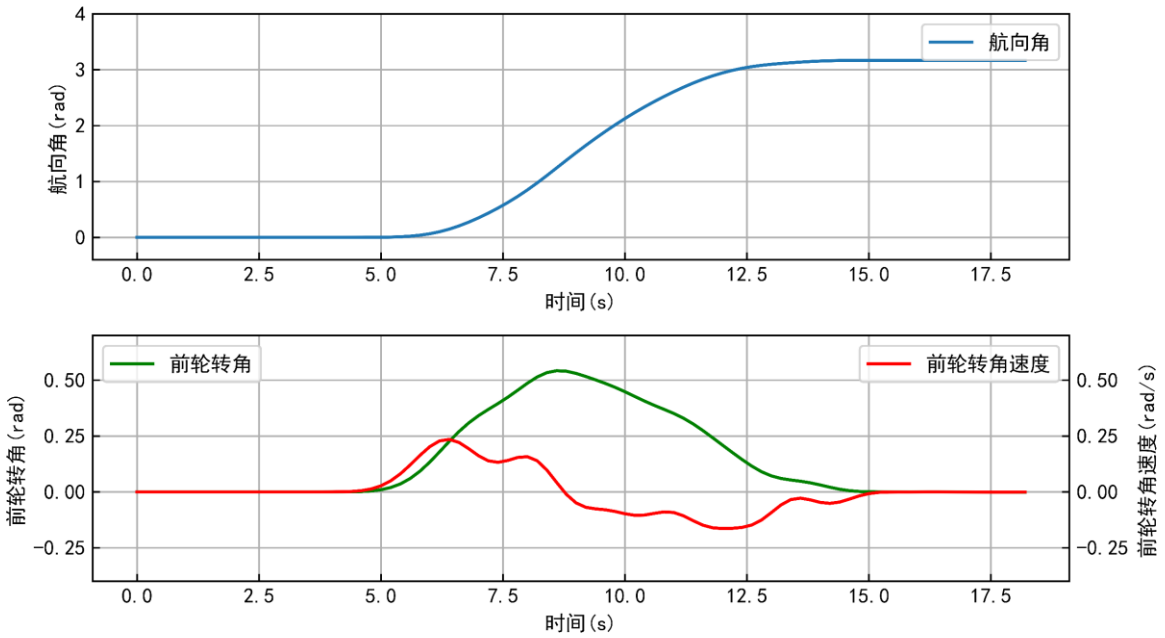


图 5.16 U 形弯场景横向状态量

中国知网 https://www.cnki.net

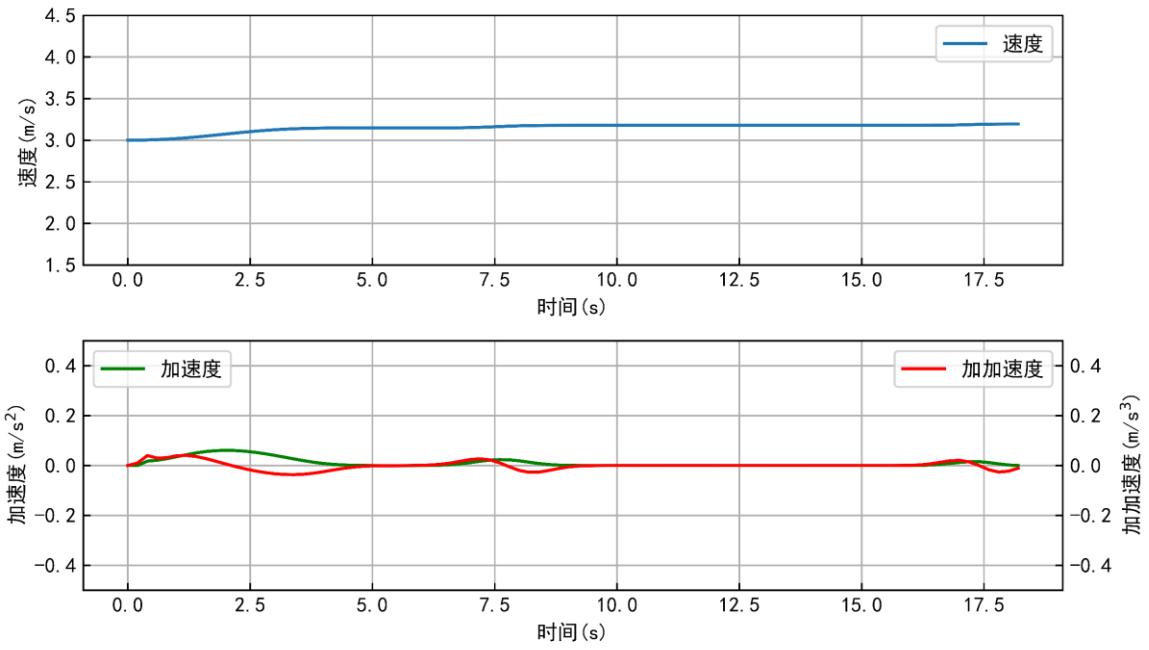


图 5.17 U 形弯场景纵向状态量

5.4 本章小结

本章在 Ubuntu 系统下，借助 ROS1 和 RViz 搭建了城市道路仿真场景，并设计了典型工况，对本文提出的时空联合轨迹规划算法框架进行仿真验证。在换道超车场景中，验证了时空联合轨迹规划算法对于障碍物的避让能力；在交叉路口左转场景中，验证了轨迹规划算法在复杂场景中的交互能力；在 U 形弯场景中，验证了轨迹规划算法在大曲率场景中的通行能力。实验结果表明，本文提出的时空联合轨迹规划算法框架在各个场景中均能生成合理且安全的轨迹，同时展现出高效性和良好的实时性，充分证明了其在城市道路场景下的适用性。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第6章 总结与展望

6.1 工作总结

轨迹规划算法在自动驾驶功能的实现中占据着重要地位，对自动驾驶车辆的舒适性、安全性、通行效率、能耗表现等多个方面有着重要影响。本文以城市结构化道路为背景，为了最大化利用求解空间，建立了一套安全高效的时空联合轨迹规划算法框架。本文的研究内容总结如下：

（1）参考线生成与规划空间构建。首先，介绍轨迹规划中常用的 Frenet 坐标系和笛卡尔坐标系，并针对二者在轨迹规划中的优缺点进行分析，同时推导了二者之间的坐标转换公式。其次，在一般结构化道路场景中使用车道中心线作为行驶参考线，在交叉路口等特殊场景中则使用五阶贝塞尔曲线生成参考线，这两种方式共同构成了平滑且曲率连续的行驶参考线。最后，以参考线为基础建立 Frenet 坐标系，构建三维规划空间，在该规划空间中对交通信号和交通参与者进行表示，进一步构建出车辆的可行驶区域。

（2）时空参考轨迹生成算法研究。针对传统混合 A*算法较少考虑速度因素，只能在二维空间进行路径规划的问题，本文在节点扩展阶段引入加速度和时间信息，筛选出安全可行的扩展节点，对成本函数和启发式函数进行重新设计，将算法改进为时空混合 A*算法，以在三维规划空间内进行轨迹搜索。为进一步提升算法的实时性，又将其优化为时空混合 ARA*算法。最终，基于搜索得到的时空参考轨迹，对道路边界和障碍物边界进行匹配，将其作为后续时空联合轨迹优化中安全性约束的重要组成部分。

（3）时空联合轨迹优化算法研究。首先对城市道路场景下的时空联合轨迹优化问题进行了明确定义，并详细介绍了 ILQR 算法求解该问题的步骤。针对 ILQR 算法的缺点，本文将不等式约束添加至目标函数中，将其转换为 CILQR 算法来求解最优轨迹问题，并给出算法的整体流程图。随后，提出了一种高阶车辆运动学模型，以提升轨迹的平滑性和前向仿真结果的准确性，并结合时空参考轨迹设计了多种初始解的生成策略。最后，根据车辆性能特点对目标函数进行了精细化设计，以确保优化结果的平滑性和舒适性。

（4）城区场景轨迹规划算法仿真验证。为了验证本文提出的轨迹规划算法的有效性，本文基于 Ubuntu 系统和 ROS1 构建了仿真实验平台，并采用 C++语言进行代码开

中国知网 https://www.cnki.net

发。针对城市道路环境，本文精心设计了借道超车、交叉路口左转以及大曲率 U 形弯三种典型场景，以全面评估算法性能。实验结果表明，本文提出的时空联合轨迹规划算法框架在各个场景中均能生成合理且安全的轨迹，同时展现出高效性和良好的实时性，充分证明了其在复杂城市道路场景下的适用性。

6.2 研究展望

本文提出了一套适用于城市道路场景的时空联合规划算法框架，但由于时间限制，未来仍可基于本文研究成果从以下几方面开展更深入的研究工作：

（1）本文在规划过程中比较依赖感知模块和预测模块的结果，但在自动驾驶车辆实际运行过程中，感知模块和预测模块的结果可能会出现前后矛盾的现象。在后续研究中可以针对上游结果跳变的情况设计相应的解决方案。同时，感知模块和预测模块的结果可能与真实世界存在一定偏差，因此在后续的研究过程中可以引入交通参与者多模态的特性，进一步提升轨迹的质量。

（2）本文聚焦于轨迹规划算法的研究，对决策算法的考虑相对较少。后续可以将本文的轨迹规划算法框架与决策算法相结合，在确保车辆轨迹安全舒适的前提下，提升轨迹的拟人化程度。

中国知网 https://www.cnki.net

参考文献

[1] 编辑部. 公安部: 全国新能源汽车保有量达 2472 万辆, 占比 7.18%[J]. 汽车与配件, 2024, (13): 48.

[2] LINDSAY BROOKE. SAE and ISO refine the Levels of Driving Automation[J]. Automotive engineering, 2021, 8(5): 30-32.

[3] 国家市场监督管理总局. 汽车驾驶自动化分级: GB/T 40429-2021[S]. 2021.

[4] Invernizzi D, Giurato M, Gattazzo P, et al. Comparison of control methods for trajectory tracking in fully actuated unmanned aerial vehicles[J]. IEEE Transactions on Control Systems Technology, 2020, 29(3): 1147-1160.

[5] 李雅琪. 从 Waymo 最大单笔融资看当前全球自动驾驶产业发展[J]. 智能网联汽车, 2024, (06): 6-7.

[6] 赵云帆. 没有方向盘、没有踏板特斯拉揭面 Robotaxi 鸿图[N]. 21 世纪经济报道, 2024-10-14(002).

[7] 杜明博. 基于人类驾驶行为的无人驾驶车辆行为决策与运动规划方法研究[D]. 中国科学技术大学, 2016.

[8] 实现可规模化的 L4, 最关键的是安全[J]. 汽车与配件, 2024, (21): 56.

[9] 文远知行自动驾驶小巴(Robobus)[J]. 汽车观察, 2024, (03): 133.

[10] Dijkstra E W. A note on two problems in connexion with graphs[M]//Edsger Wybe Dijkstra: his life, work, and legacy. 2022: 287-290.

[11] Chen R, Hu J, Xu W. An RRT-Dijkstra-based path planning strategy for autonomous vehicles[J]. Applied Sciences, 2022, 12(23): 11982.

[12] Sundarraj S, Reddy R V K, Basam M B, et al. Route planning for an autonomous robotic vehicle employing a weight-controlled particle swarm-optimized Dijkstra algorithm[J]. IEEE Access, 2023, 11: 92433-92442.

[13] Pearl J. Heuristic Search Theory: Survey of Recent Results[C]//IJCAI. 1981, 1: 554-562.

[14] Rowold M, Ögretmen L, Kerbl T, et al. Efficient spatiotemporal graph search for local trajectory planning on oval race tracks[C]//Actuators. MDPI, 2022, 11(11): 319.

[15] Likhachev M, Gordon G J, Thrun S. ARA*: Anytime A* with provable bounds on sub-

中国知网 https://www.cnki.net

optimality[J]. Advances in neural information processing systems, 2003, 16: 767-774.

[16] Islam F, Narayanan V, Likhachev M. A*-connect: Bounded suboptimal bidirectional heuristic search[C]//2016 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2016: 2752-2758.

[17] Furcy D A. Speeding up the convergence of online heuristic search and scaling up offline heuristic search[M]. Georgia Institute of Technology, 2004.

[18] Likhachev M, Ferguson D I, Gordon G J, et al. Anytime dynamic A*: An anytime, replanning algorithm[C]//ICAPS. 2005, 5: 262-271.

[19] Koenig S, Likhachev M, Furcy D. Lifelong planning A*[J]. Artificial Intelligence, 2004, 155(1-2): 93-146.

[20] Dolgov D, Thrun S, Montemerlo M, et al. Practical search techniques in path planning for autonomous driving[J]. ann arbor, 2008, 1001(48105): 18-80.

[21] Meng T, Yang T, Huang J, et al. Improved hybrid a-star algorithm for path planning in autonomous parking system based on multi-stage dynamic optimization[J]. International journal of automotive technology, 2023, 24(2): 459-468.

[22] Zhang Y, Wang Y, Wu M, et al. Two-Stage Dichotomy Hybrid A* for Path Planning of Unmanned Mining Trucks in Large-Scale Open-Pit Mine[C]//2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM). IEEE, 2024: 296-301.

[23] Sheng W, Li B, Zhong X. Autonomous parking trajectory planning with tiny passages: A combination of multistage hybrid A-star algorithm and numerical optimal control[J]. IEEE Access, 2021, 9: 102801-102810.

[24] Hu K, Cheng K. Trajectory planning for an articulated tracked vehicle and tracking the trajectory via an adaptive model predictive control[J]. Electronics, 2023, 12(9): 1988.

[25] Fan H, Zhu F, Liu C, et al. Baidu apollo em motion planner[J]. arXiv preprint arXiv:1807.08048, 2018.

[26] Bohlin R, Kavraki L E. Path planning using lazy PRM[C]//Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065). IEEE, 2000, 1: 521-528.

[27] Karaman S, Frazzoli E. Incremental sampling-based algorithms for optimal motion

中国知网 https://www.cnki.net

planning[J]. Robotics Science and Systems VI, 2010, 104(2): 267-274.

[28] Kuffner J J, LaValle S M. RRT-connect: An efficient approach to single-query path planning[C]//Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065). IEEE, 2000, 2: 995-1001.

[29] Wu B, Zhang W, Chi X, et al. A novel AGV path planning approach for narrow channels based on the Bi-RRT algorithm with a failure rate threshold[J]. Sensors, 2023, 23(17): 7547.

[30] Webb D J, Van Den Berg J. Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics[C]//2013 IEEE international conference on robotics and automation. IEEE, 2013: 5054-5061.

[31] Wang J, Li B, Meng M Q H. Kinematic Constrained Bi-directional RRT with Efficient Branch Pruning for robot path planning[J]. Expert Systems with Applications, 2021, 170: 114541.

[32] Swedeen J, Droge G, Christensen R. Fillet-based RRT＊: a rapid convergence implementation of RRT* for curvature constrained vehicles[J]. Journal of Intelligent & Robotic Systems, 2023, 108(4): 68.

[33] Huang G, Ma Q. Research on path planning algorithm of autonomous vehicles based on improved RRT algorithm[J]. International journal of intelligent transportation systems research, 2022: 1-11.

[34] Esmaiel H, Zhao G, Qasem Z A H, et al. Double-Layer RRT* Objective Bias Anytime Motion Planning Algorithm[J]. Robotics, 2024, 13(3): 41.

[35] Li B, Kong Q, Zhang Y, et al. On-road trajectory planning with spatio-temporal RRT* and always-feasible quadratic program[C]//2020 IEEE 16th International Conference on Automation Science and Engineering (CASE). IEEE, 2020: 942-947.

[36] Xia T, Chen H. A Survey of Autonomous Vehicle Behaviors: Trajectory Planning Algorithms, Sensed Collision Risks, and User Expectations[J]. Sensors, 2024, 24(15): 4808.

[37] Shi Y, Wang P, Wang X. An autonomous valet parking algorithm for path planning and

中国知网 https://www.cnki.net

tracking[C]//2022 IEEE 96th Vehicular Technology Conference (VTC2022-Fall). IEEE, 2022: 1-7.

[38] Ran K, Wang Y, Fang C, et al. Improved RRT* Path-Planning Algorithm Based on the Clothoid Curve for a Mobile Robot Under Kinematic Constraints[J]. Sensors, 2024, 24(23): 7812.

[39] Kim D J, Chung C C. Automated perpendicular parking system with approximated clothoid-based local path planning[J]. IEEE Control Systems Letters, 2020, 5(6): 1940-1945.

[40] Gu T, Snider J, Dolan J M, et al. Focused trajectory planning for autonomous on-road driving[C]//2013 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2013: 547-552.

[41] Arbitmann M, Stählin U, Schorn M, et al. Method and device for performing a collision avoidance maneuver: U.S. Patent US8,209,090[P]. 2012.

[42] Heilmeier A, Wischnewski A, Hermansdorfer L, et al. Minimum curvature trajectory planning and control for an autonomous race car[J]. Vehicle System Dynamics, 2020, 58(10): 1497-1527.

[43] Cao H, Zhao S, Song X, et al. An optimal hierarchical framework of the trajectory following by convex optimisation for highly automated driving vehicles[J]. Vehicle System Dynamics, 2019, 57(9): 1287-1317.

[44] Zhang Z, Wei C, Ma B, et al. An optimization-based trajectory planning method with polynomial curves[C]//2023 IEEE International Conference on Unmanned Systems (ICUS). IEEE, 2023: 1-6.

[45] Li Y, Li L, Ni D. Dynamic trajectory planning for automated lane changing using the quintic polynomial curve[J]. Journal of advanced transportation, 2023, 2023(1): 6926304.

[46] Zhou B, Gao F, Wang L, et al. Robust and efficient quadrotor trajectory generation for fast autonomous flight[J]. IEEE Robotics and Automation Letters, 2019, 4(4): 3529-3536.

[47] Li J, Xie X, Ma H, et al. Speed planning using bezier polynomials with trapezoidal corridors[J]. arXiv preprint arXiv:2104.11655, 2021.

[48] Ding W, Zhang L, Chen J, et al. Safe trajectory generation for complex urban environments using spatio-temporal semantic corridor[J]. IEEE Robotics and Automation

中国知网 https://www.cnki.net

Letters, 2019, 4(3): 2997-3004.

[49] Chen J, Zhan W, Tomizuka M. Autonomous driving motion planning with constrained iterative LQR[J]. IEEE Transactions on Intelligent Vehicles, 2019, 4(2): 244-254.

[50] Cao K, Cao M, Yuan S, et al. DIRECT: A differential dynamic programming based framework for trajectory generation[J]. IEEE Robotics and Automation Letters, 2022, 7(2): 2439-2446.

[51] Stano P, Montanaro U, Tavernini D, et al. Model predictive path tracking control for automated road vehicles: A review[J]. Annual reviews in control, 2023, 55: 194-236.

[52] Li B, Acarman T, Zhang Y, et al. Optimization-based trajectory planning for autonomous parking with irregularly placed obstacles: A lightweight iterative framework[J]. IEEE Transactions on Intelligent Transportation Systems, 2021, 23(8): 11970-11981.

[53] Zhang W, Wang Z, Drugge L, et al. Evaluating model predictive path following and yaw stability controllers for over-actuated autonomous electric vehicles[J]. IEEE Transactions on Vehicular Technology, 2020, 69(11): 12807-12821.

[54] Wang Z, Zha J, Wang J. Flatness-based model predictive control for autonomous vehicle trajectory tracking[C]//2019 IEEE Intelligent Transportation Systems Conference (ITSC). IEEE, 2019: 4146-4151.

[55] Xiang C, Peng H, Wang W, et al. Path tracking coordinated control strategy for autonomous four in-wheel-motor independent-drive vehicles with consideration of lateral stability[J]. Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering, 2021, 235(4): 1023-1036.

[56] Morato M M, Normey-Rico J E, Sename O. Model predictive control design for linear parameter varying systems: A survey[J]. Annual Reviews in Control, 2020, 49: 64-80.

[57] Huang C, Lv C, Hang P, et al. Toward safe and personalized autonomous driving: Decision-making and motion control with DPF and CDT techniques[J]. IEEE/ASME Transactions on Mechatronics, 2021, 26(2): 611-620.

[58] Qie T, Wang W, Yang C, et al. An improved model predictive control-based trajectory planning method for automated driving vehicles under uncertainty environments[J]. IEEE Transactions on Intelligent Transportation Systems, 2022, 24(4): 3999-4015.

中国知网 https://www.cnki.net

[59] Chen J, Zhan W, Tomizuka M. Constrained iterative lqr for on-road autonomous driving motion planning[C]//2017 IEEE 20th International conference on intelligent transportation systems (ITSC). IEEE, 2017: 1-7.

[60] Li W, Todorov E. Iterative linear quadratic regulator design for nonlinear biological movement systems[C]//First International Conference on Informatics in Control, Automation and Robotics. SciTePress, 2004, 2: 222-229.

[61] Da F. Comprehensive reactive safety: No need for a trajectory if you have a strategy[C]//2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2022: 2903-2910.

[62] Li T, Zhang L, Liu S, et al. Marc: Multipolicy and risk-aware contingency planning for autonomous driving[J]. IEEE Robotics and Automation Letters, 2023, 8(10): 6587-6594.

[63] Hardy J, Campbell M. Contingency planning over probabilistic obstacle predictions for autonomous road vehicles[J]. IEEE Transactions on Robotics, 2013, 29(4): 913-929.

[64] Zhang Y, Sun H, Chai R, et al. Optimal Vehicle Trajectory Planning for Static Obstacle Avoidance using Nonlinear Optimization[J]. arXiv preprint arXiv:2307.09466, 2023.

[65] Han Z, Wu Y, Li T, et al. Differential flatness-based trajectory planning for autonomous vehicles[J]. arXiv preprint arXiv:2208.13160, 2022.

[66] Zhang Y, Sun H, Zhou J, et al. Optimal vehicle path planning using quadratic optimization for baidu apollo open platform[C]//2020 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2020: 978-984.

[67] Zhang Z, Tian R, Sherony R, et al. Attention-based interrelation modeling for explainable automated driving[J]. IEEE Transactions on Intelligent Vehicles, 2022, 8(2): 1564-1573.

[68] Zhu Z, Zhao H. A survey of deep RL and IL for autonomous driving policy learning[J]. IEEE Transactions on Intelligent Transportation Systems, 2021, 23(9): 14043-14065.

[69] Zhu Z, Zhao H. Multi-task conditional imitation learning for autonomous navigation at crowded intersections[J]. arXiv preprint arXiv:2202.10124, 2022.

[70] Hoque R, Balakrishna A, Novoseller E, et al. Thriftydagger: Budget-aware novelty and risk gating for interactive imitation learning[J]. arXiv preprint arXiv:2109.08273, 2021.

[71] Yan C, Qin J, Liu Q, et al. Mapless navigation with safety-enhanced imitation learning[J].

中国知网 https://www.cnki.net

IEEE Transactions on Industrial Electronics, 2022, 70(7): 7073-7081.

[72] Phan-Minh T, Howington F, Chu T S, et al. Driving in real life with inverse reinforcement learning[J]. arXiv preprint arXiv:2206.03004, 2022.

[73] Ronecker M P, Zhu Y. Deep Q-network based decision making for autonomous driving[C]//2019 3rd international conference on robotics and automation sciences (ICRAS). IEEE, 2019: 154-160.

[74] Kaushik M, Singhania N, S P, et al. Parameter sharing reinforcement learning architecture for multi agent driving[C]//Proceedings of the 2019 4th International Conference on Advances in Robotics. 2019: 1-7.

[75] Ye F, Cheng X, Wang P, et al. Automated lane change strategy using proximal policy optimization-based deep reinforcement learning[C]//2020 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2020: 1746-1752.

[76] Wu Y, Liao S, Liu X, et al. Deep reinforcement learning on autonomous driving policy with auxiliary critic network[J]. IEEE transactions on neural networks and learning systems, 2023, 34(7): 3680-3690.

中国知网 https://www.cnki.net